==========

PlayList Name : CORE JAVA FULL COURSE

Channel: India Free Internship

Live Notes with Live coding

NotesLink:
https://github.com/indiafreeinternship/NOTES/tree/main/core%20java%20notes

==========

SESSION 1

Full Stack Java Developer

Module 1-: Programming Module (CoreJava, AdvJava, Framework-Spring)

Module 2-: UI Module (Html/CSS/JS/Angular/React)

Module 3-: DataBase Module (Oracle,MySql)

Module 4-: Testing Module (Testing basics: Selenium)

Module 5-: Tools Module (DevSecOps: Development Security and Operations tools)

JAVA

=====

PART 1: Core Java

PART 2: AdvJava

==========

CORE JAVA

==========

Language

========

=> Every language will have their own Alphabets.

=> Every lauguage will have their own Grammer.

=> Every language will have their own Constructions rules.

part 1: CoreJava

==========

1. Java Programming Components(java, Alphabets)

2. Java Programming concepts

3. Object Oriented Programming features.

==========

1. Java Programming Components(java, Alphabets)

(a) Variables

(b)Methods(Functions)

(c) Blocks

(d)Constructor

(e)classes

(f)interfaces

(g) AbstractClasses

2. Java Programming concepts

(a)Object-Oritented progrmming concepts

(b) Exception Handling process

(c) Multi-Threading process

(d) Java Collection Framework(JCF)

(e) File Storage

(f) Networking in java(Socket programming)

3. Object Oriented Programming features.

(a) Class

(b) Object

(c) Abstraction

(d) Encapsulation

(e) PolyMorphism

(f) Inheritance

Define StandAlone Applications :-

=>The application which are installed in one computer and performs actions in the same computer are known as Stand Alone application or DeskTop Application or Window applications.

=> According to developer StandAlone application means,

No Html Input

No Server Envrironment

No Database Storage

==========

AdvJava

==========

=> Advjava provide the following technolgies to construct web application.

(I)JDBC

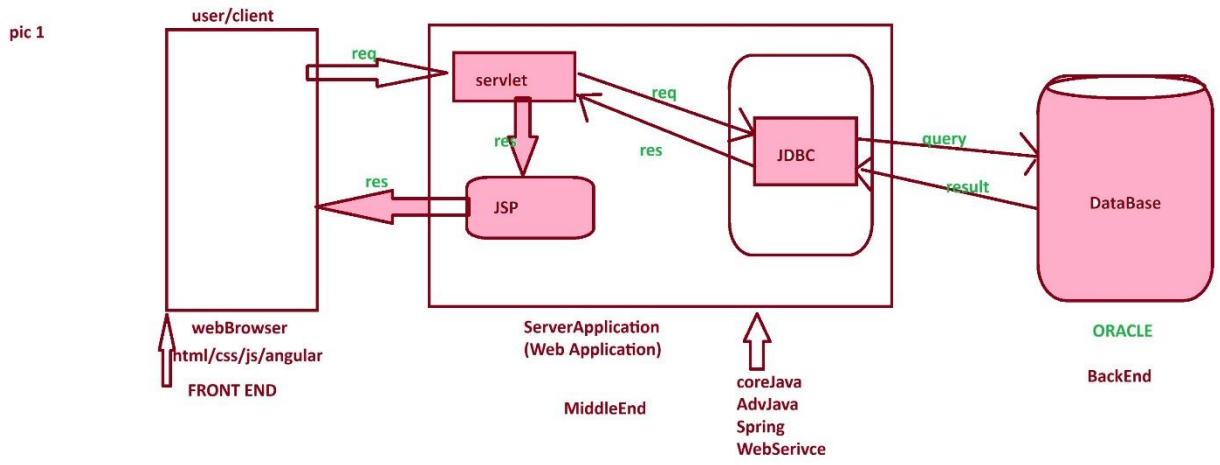
(ii)Servlet

(iii)JSP

Define Web Application

=>The Application which is executing in Web environment or internet environment is known as Web application or internet application.

Web Application Architecture pic 1:



==========

Servlet :- Servlet is a server program which accepts requets from user through WebBrowser.

JSP :- JSP stands for 'java Sever Page' and which is response from webApplication.

JDBC :- JDBC stands for 'java Database connectivity' and which is used to intract with Database.

==========

======

SESSION 2

======

What is the differenct b/w

(i) Language

(ii) Technology

(iii) Framework

(i) Language = Language provide programming components and concepts which are used in contructing program.

eg: java,python,c,c++

(ii) Technology = The process of Converting knowledge into realtime world application development is known as Technology.

eg: Login, Registration, AdvJava.

(iii) Framework = The Stucture which is ready constructed and available for application development is known as Framework.

eg : Spring, webServices

==========

Module 1-: Programming Module (CoreJava, AdvJava, Framework-Spring)

==========

Level -1: core java - standAlone Application

Level -2 : AdvJava - Web Application.

Level -3 : Framework - Enterprise application.

---

what is Enterprise application?

The application which are executing in distributed evironment and depending on the features like "Security", "Load balancing" and "clustring", are known as Enterprise Distributed Application or Enterprise Application.

==========

session 3

========

Define Program?

- Program is a set of instruction.

- After writing program, we have to save the program with language extension.

eg: Test.c

Test.cpp

Test.java

Stages of Program:

- The program will have following two stages:

1. Compilation

2. Execution

Compilation:

- The process of checking the program constructed according to the rule of language or not, is known as Compilation process.

- After Compilation process is Successfull, the sourceCode is converted into Compiled codes.

- C and C++ programs will generate ObjectiveCode and java program generate Byte code.

Execution

- The process of running Compiled codes and checking the required output is generated or not, is known as Execution process.

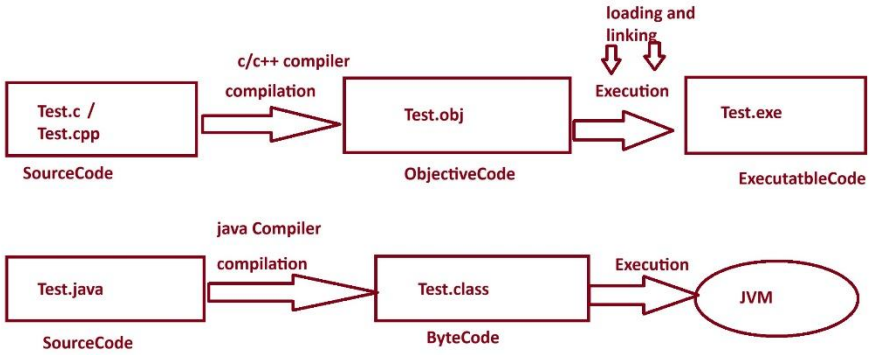
- In C and C++ languages, while executing Objective Code Internally " loading and Linking" process is performed and Object code converted into excutable code and generate result.

- In Java Language, the Byte code is executed directly on JVM (Java Virtual Machine)

==========

stages of program pic 2.

PIC 2



==========

session 4

========

what is the diff b/w

(i) Objective Code

(ii) Byte Code

(i) Objective Code

==========

=> The copiled code genreated from c and c++ program is known as Objective Code.

=> While Objective code generation OperatingSystem is participated, becuase of this reason ObjectiveCode is PlatForm dependent code.

DisAdvantages

=> Objective Code generated from one PlatForm cannot be executed on other PlatForm.

Note:

=> c and c++ language which are generating Objective Code are PlatForm dependent language.

(ii) Byte Code

==========

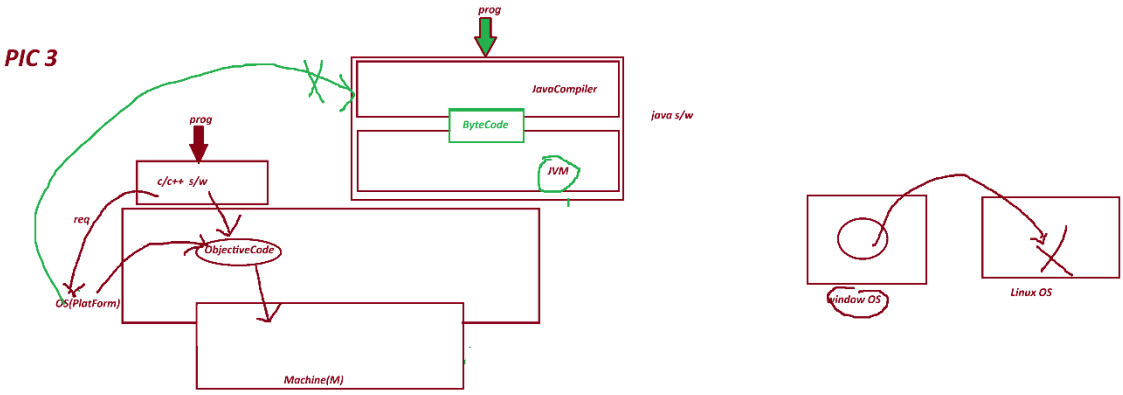
=> The byte code generated from one PlatForm canbe executed on all PlatForm based on JVM.

Note:

=> java language which is generating ByteCode is PlatForm independent language.

PIC 3

PIC 3



======

Define High Level language?

==========

=> The language format whic are understandable by the users are known as High Level Language.

c,c++,java

Define Low Level Language?

==========

=> The language format which are not understandable by the user are known as Low Level Language.

Ex: Machine languages.

Define Translator?

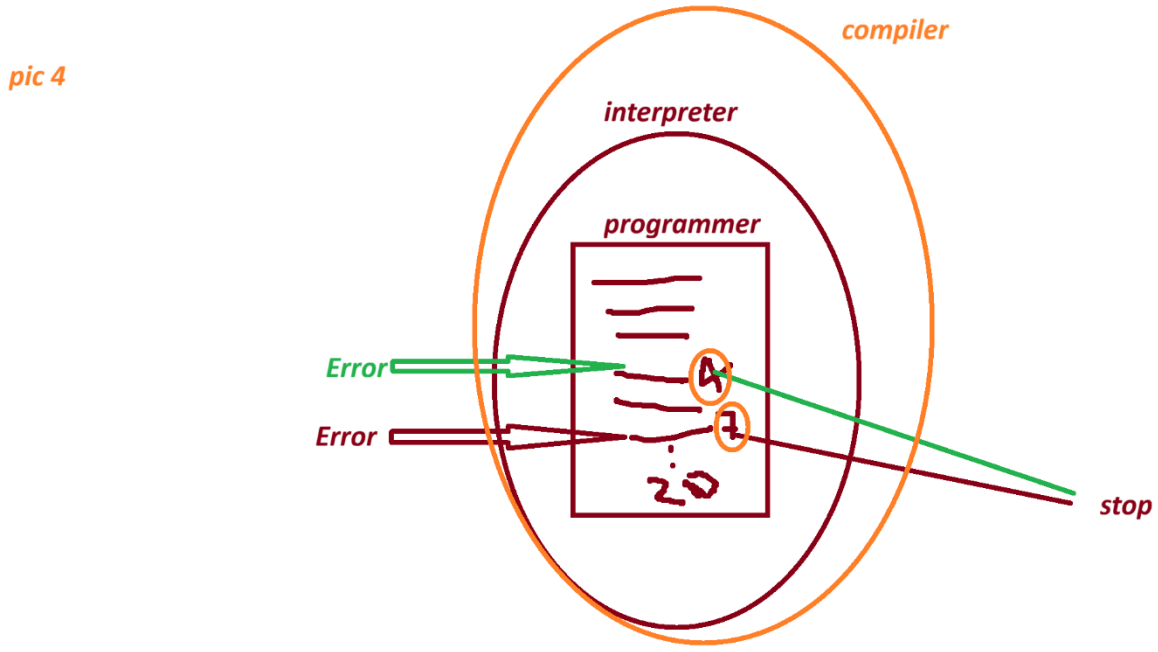
=> Translator are used to convert High Level language formats into Low Level Language formats and Low Level Language formats into High Level Language.

These Translators are categorized into two type.

(i) Interpreters - which translates program line-by-line.

(ii) Compiler - which translates total program at-a-time.

Pic 4



==========

1991 - "James Gosling" Sun Micro System - Code Write(programer)

WORA - Write Once and Run Anywhere.

Test.gt (Green Talk)

OAK

Java

==========

java versions:

1995 - java Alpha&Beta

1996- JDK 1.0

1997- JDK 1.1

1998 - JDK 1.2

2000 - JDK 1.3

2002 - JDK 1.4

2004 - Java5(Tiger)

2006 - java6

2011 - java7

2014- java8

2017 - java9

2018-java10 , java11

2019- java12,java13

..... 2026 -java 26

---

session 5

=======

What is the diff between ?

(i) JDK

(ii) JRE

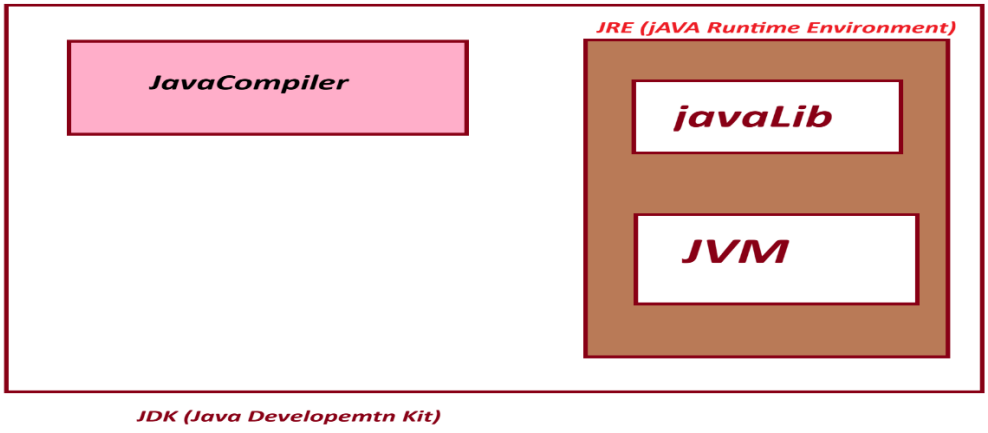
(iii) JVM

(i) JDK : JDK stands for "Java Development Kit" and which is the collection of the following:

(a) JavaCompiler

(b) JavaLib

(c) JVM



(a) JavaCompiler :- JavaCompiler will compile the source code and generate ByteCode when the compilation process is successfull.

(b) JavaLib :- JavaLib will provide pre-defined and ready constructed programming components which are used in construting application.

JavaLib= 'java'

CoreJava Packages:

(i) java.lang - Language package

(ii) java.util - Utility package

(iii) java.io - IO Stream and Files packages

(iv) java.net - Networking package

AdvJava packages:

(i) java.sql - DataBase connection package

(ii) javax.servlet - Servlet progrmming package

(iii) javax.servlet.jsp - JSP programming package

(c) JVM :- JVM Stands for "Java Virtual Machine" and which is used to execute JavaByteCode.

=> JVM internal partition of JDK.

=> The s/w components which intrnally having the behaviour like machine is known as Virtual Machine.

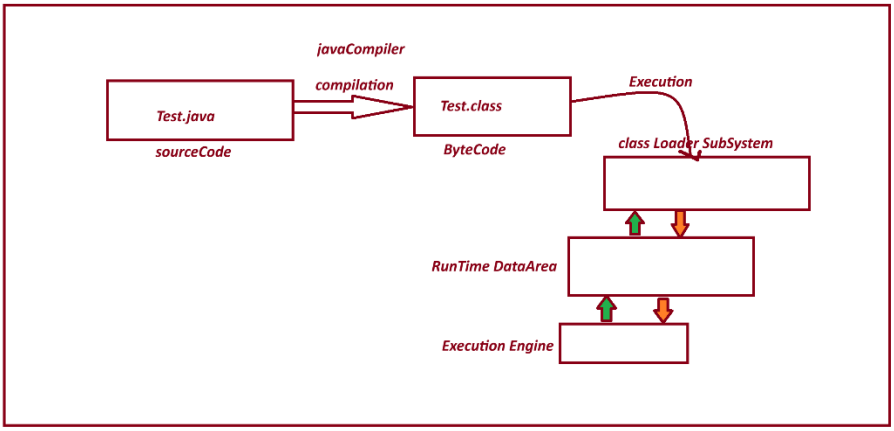
=> The Following are the parts of JVM:

1. Class Loader SubSystem

2. Runtime DataArea

3. Execution Engine

JVM PIC



==========

(ii) JRE :- JRE Stands for 'Java Runtime Environment' and which is collection JavaLib and JVM.

=> JRE is an internal partition of JDK.

========

session 6

========

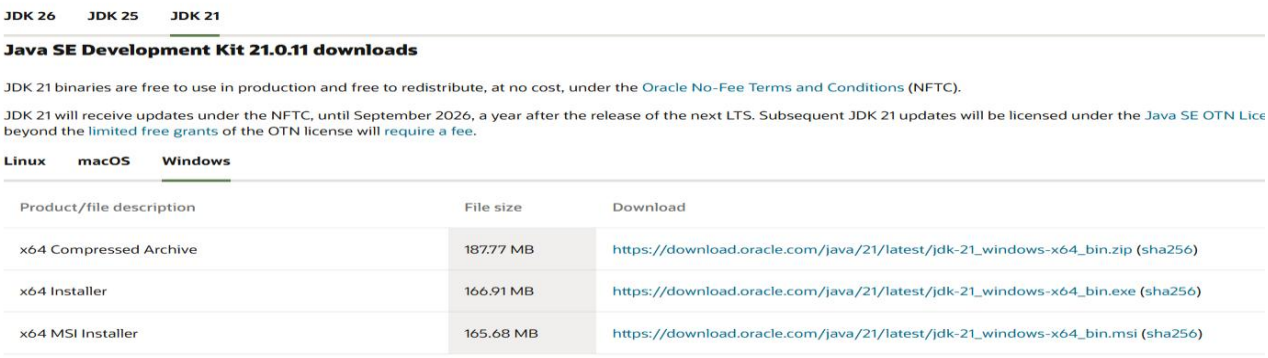
Installing Java s/w and setting path:

==========

step 1 :- Download java s/w(JDK) from Oracle Website.

Link: https://www.oracle.com/java/technologies/downloads/

step-2 : Install JDK-21



JDK 26 JDK 25 JDK 21

Java SE Development Kit 21.0.11 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the Java SE OTN Lice beyond the limited free grants of the OTN license will require a fee.

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	187.77 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	166.91 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	165.68 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

Note: After Installation process we can find one folder with name "java" in program file.

C:\Program Files\Java

step 2 : set java path in "Envrironment variables"

click->new->

variables : path

variable : C:\Program Files\Java\jdk-21.0.11\bin;

Note:

=> Open CommandPrompt and check the follwoing commands are working or not:

javac - compilation commands

java -execution command

C:\Users\satru>java -version

java version "21.0.11" 2026-04-21 LTS

Java(TM) SE Runtime Environment (build 21.0.11+9-LTS-211)

Java HotSpot(TM) 64-Bit Server VM (build 21.0.11+9-LTS-211, mixed mode, sharing)

==========

session 7

========

Environment Variables :- Envrironment Variables are OperatingSystem variable, which hold the information about s/w components installed in Computer System.

These Environment variable are categorized into two types.

(i) User variable

(ii) System Variables

(i) User variable :- The variable whihc are related to individual users of Computer System are known as User Variables.

=>The information in User variable can be used by only individual user.

(ii) System Variables :-The variable which are related to all the users of computer system are known as system variables.

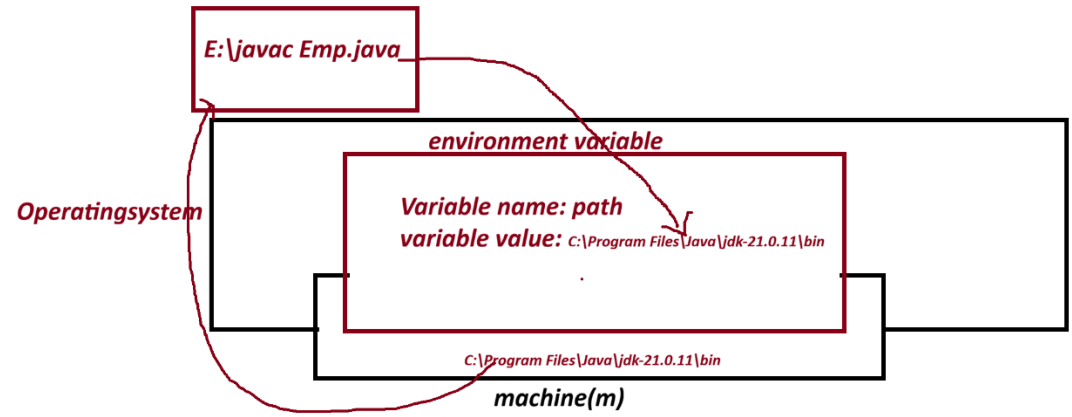
=> The Information in System variable can be used by all users of computer system.

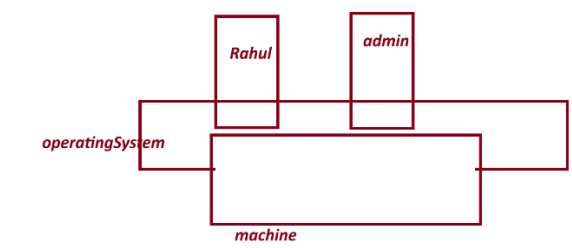
----

what is the advantage of having javaPath in environment variable?

=> When we have javaPath in Environment variable, then we can compile and execute java program from any location of computer System.

diagram:





session 8

define 'class'?

=> "class" is a "Structured Layout" generate "Object".

=> class in java is a collection of Variables, Methods, Blocks and Construtors.

=> The class in java can be added with main() to start the program execution.

=> main() is having the following pre-defined standard systax

ex: public static void main(String[] arg){}

=>we use "class" keyword in java to construct classes.

Structur of class in java:

```
class Class_name{
        //variable
        //methods
        //Blocks
        //Construtors
        //main()
}
```

==========

Ex program-1 : - Wap to display the msg as "Welcome to India Free Internship"?

```
class Display{
public static void main(String[] args){
 System.out.print("India Free Internship");
  }
}
```

step -1 Open Notedpad and type the program.

step -2 Save the program in folder with language-extendion (.java)

Systax :

Class_name.java

ex:

Display.java

step 3- File->Save-> Browser and select folder -> core java prgrams

-> save as type must be "All Files" -> click "save"

=> Open CommandPrompt and perform compilatin and Execution process

To open CommandPrompt , goto,folder-> type "cmd" in Addressbar and press "Enter"

step 4- compile the program as follows:

Syntax:

javac Class_name.java

ex:

javac Display.java

Step -5 Execute the program as follows:

Syntax:

java Class_name

Ex:

java Display

==========

Session 9

==========

Ex program -2

wap to display the sum of two numbers?

```
class Addition{
        public static void main(String[] args){
   int a=10;
```

```
    int b=20;
    int c=a+b;
    System.out.println("a value = "+a);
    System.out.println("b value = "+b);
    System.out.println("result (a+b) ="+c);
  }
}
```

ouput

=====

D:\core java programs>javac Addition.java (compilation)

D:\core java programs>java Addition (execution)

a value = 10

b value = 20

result (a+b) =30

==========

what is the diff b/w print() and println()

(i) print()

(ii)println()

(i)print():- print() method will display message and result, and makes the cursor wait in the same line.

(ii) println():- println() method also display message and result, and moves the cursor to next line or new line.

Note:- "+" symbol in print() method will concatenate(combine) message with result.

==========

Ex - program-3

wap to display Employee details?

(id,name,desg,sal)

Employee.java

```
class Employee
{
    public static void main(String[] args)
    {
      String id="SE121";
      String name="Alex";
      String desg="SE";
      int sal=30000;
      System.out.println("EmpId ="+id);
      System.out.println("EmpName ="+name);
      System.out.println("EmpDesg ="+desg);
      System.out.println("EmpSalary ="+sal);
    }
}
```

ouput

```
javac Employee.java
java Employee

EmpId =SE121
EmpName =Alex
EmpDesg =SE
EmpSalary =30000
```

==========

Assignment -1

wap to display BookDetails?

(code,name,author,price,qty)

Assignment -2

wap to display StudentDetails?

(name,branch,rollNo,6 sub marks, totMarks,per)

ouput

------

name=

branch=

rollNo=

s1=

s2=

s3=

s4=

s5=

s6=

totMarks=s1+s2+s3+s4+s5+s6;

per=

==========

session 10

=========

Naming Conventions in java.

=>The coding rules which are followed by the progrmmer in writing porgrams are known as Naming conventions in Java.

(Realtime Coding Standard).

package:

def => Packages are collection of Classes,interfaces and enum.

rule=> packages must be writing in Lowercase.

ex. com.app

==========

Classs and interfaces

def => Classes and Interface are collection of "variable and Methods"

rule : In Classes and interfaces, the starting letter of every word must be UpperCase.

ex: EmployeeSalary, DataInputStream

==========

Variables and Methods:

def : Variables are data holders and Methods are actions.

rule : In Variables and methods, the first word must be in LowerCase and from second onwords the starting letter must be UpperCase.

Ex: variable : rollNo, panCard,aadharCard

methods: readLine(), getSalary(),calculateMakrs(),.. etc

==========

Keywords:

def : The pre-defined built in words are known as keyword.

Rule : Keywords in java must be LowerCase.

Ex: void, static, public...

session 11

========

DataType:

Data Type specifies what type of data a variable can store in memory.

=> DataType in Java are categorized into two types.

1. Primitive DataType

2. Non-Primitive DataType

1. Primitive DataType:

-The "single valued data formats" are known as Primitive data types.]

-These Primitive data Type are categorized into four types.

(a) Integer DataTypes

(b) Float DataTypes

(c) Character DataType

(d) Boolean DataTypes

(a) Integer DataTypes

-The numrics data which is represented without decimal point are known as Integer DataTypes.

-These Integer dataTypes are categorized into four (4) types:

(i) bytes - 1 byte(8 bits)

(ii) short - 2 bytes

(iii) int - 4 bytes

(iv) long - 8 bytes

=> "byte" and "short" datatypes are used for Stream-data or MultiMedia data

=> "int" datatype is used for normal progrmming.

=> "long" datatype is used for biggest integer values like PhoneNo,AadharCardNo,BankCardNo....

=> In the process of assigning long-value we must use "L" or "l" in the RHS of declaration.

Ex: long phoneNo=99985241213L

==========

(b) Float DataTypes:

=> The numeric data with decimal point represented are known as Float DataTypes.

=> Float DataType are categorized into two types:

(i) float - 4 bytes

(ii) double -8 bytes

=> "float" datatype is used in normal programming.

=> "double" datatype is used to hold biggest scientific calculated values.

=> In the process of assigning float value we must use "F" or 'f' in the RHS of declaration.

Ex:

float f=123.34F

==========

ex : program

VariableDataType.java

```
class VariableDataType
{

  public static void main(String[] args){

    byte b=127;
    short s=1234;
    int i=1234567;
    long no=9898351245L;
    float f=123.5F;
```

```
    double d=23344.56;


    System.out.println("b value = "+b);
    System.out.println("s value = "+s);
    System.out.println("i value = "+i);
    System.out.println("no value = "+no);
    System.out.println("f value ="+f);
    System.out.println("d value = "+d);
  }
}
```

---

output

```
D:\core java programs>javac VariableDataType.java
D:\core java programs>java VariableDataType
b value = 127
s value = 1234
i value = 1234567
no value = 9898351245
f value =123.5
d value = 23344.56
```

==========

(c) Character DataType

- The "single valued Character" which is represented in single quotes is known as Character Datatype.

Ex: 'a', 'b', 'h', 'l', 'r' etc

char = 2 bytes

=========

why Character 2 bytes in java?

=> java language will use UNICODE (Universal Code) representation and which is ready to accept any type of language-codes available in the world.

=> The Folowing are some important language code in the world.

a. ASCII (Americal Standard Code for Information Interchange) for United State

b. ISO 8859-1 for Western European language.

c. KOI-8 for Russian.

d. GB18030 and BIG-5 for chines

==========

a. ASCII (Americal Standard Code for Information Interchange) for United State

=> The unique-numeric number which is genreated for every character entered from the keyboard is known as ASCII code.

==========

(d) Boolean DataTypes

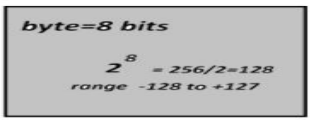
=> The datatype which is avalable in the form of true or false is known as Boolean DataType.

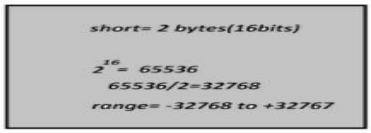
boolean =1 bits

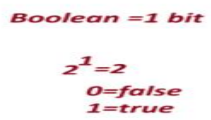
0=false

1=true

Diagram







session 12

==========

2. Non-Primitive DataType :-

The "group valued data formats" are known as Non-Primitive data Types or Referential datatypes.

- These Non-Primitive datatype are categorized into four types:

(a) Class

Ex class Student{

}

(b) Interface

Ex:

interfaces Animal{

}

(c) Array

Ex : int arr[]={10,20,40};

(d) Enum

enum Days{

MONDAY, TUESDAY

}

Note: "String is a Class in java and comes under Non-Primitive Datatype.

Ex String city="patna"

==========

Object Oriented Programming

=> The process of cosntructing programs using Class-Object concept is known as Object-Oritented Programming.

=> In Object oriented programming we work with Non-Primitive datatype or Referential datatypes.

=> The following are the levels in Object Oriented Programming.

1. Object definition
2. Object Creation
3. Object Location
4. Object components
5. Object Type.

6. Object Collection (Grouping object)

7. Object Sorting

8. Object Locking

9. Object Serialization

10. Object Cloning

==========

Define Object

=> Object is a Storage(memory) related to a class holding the member of class.

=> we use "new" keyword in java to create object.

Syntax :

Class_name obj_name=new Class_name();

==========

define 'class'?

=> "class" is a "Structured Layout" generate "Object".

=> class in java is a collection of Variables, Methods, Blocks and Construtors.

=> The class in java can be added with main() to start the program execution.

=> Classes in java are categorized into two types

(i) pre-defined class

(ii) user-defined class

(i) pre-defined class : The classes which are ready constructed and available from JavaLib are known as Pre-define classes or Built-in-classes.

Ex: String, System

(ii) user-defined class : The classes which are defined by the programmer are known as user defined classes.

Display

Addition

Employee

VariableDataType

==========

Variables in java:

=> variable are the data holder in the program

=> Based on Datatype the variable in java are categroiezed into two types:

1. Primitive DataType variables

2. Non-Primitive DataType variables

1. Primitive DataType variables : The variable which are declared with primitve data type like byte, short, int, long, double, char, boolean are known as primitve datatype variable.

=> These primitve data type variables will hold values.

2. Non-Primitive DataType variables : The variable which are declared with Non-primitve DataType like Class, Interface, Array, Enum are known as Non-Primitive DataType variables or referential data type variable.

=> These Non-Primitive Datatype variable will hold Object references.

==========

session 13

==========

define "static" ?

=> "static" keyword in java will decide the location of component memory in Class or Object.

=> bsed on "static" keyword the progrmming components are categorized into 2 types.

a. static programming components.

b. nonStatic programming components.

a. static programming components.

=> The programming components which are declared with "static" keyword are known as static programming components or Class Programming components.

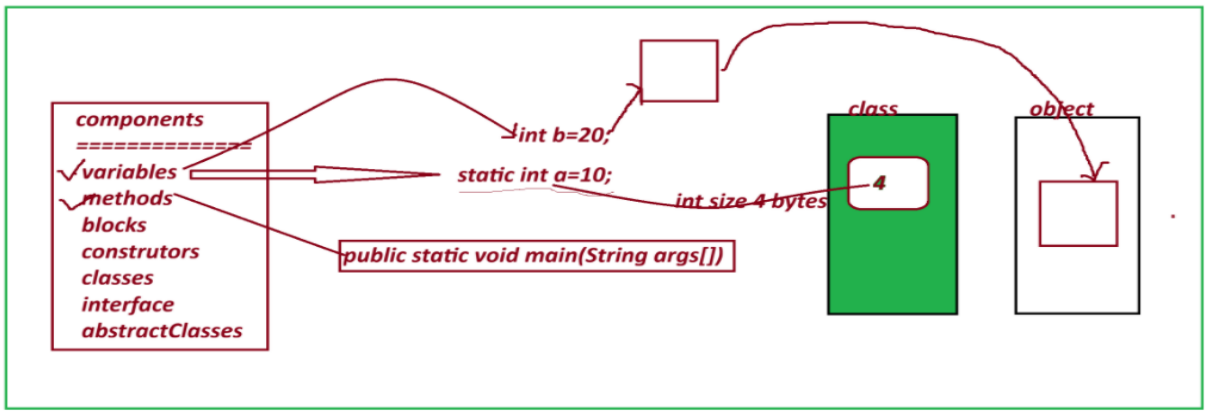
=> These static programming components will get the memory within the class and can be accessed with Class_name.

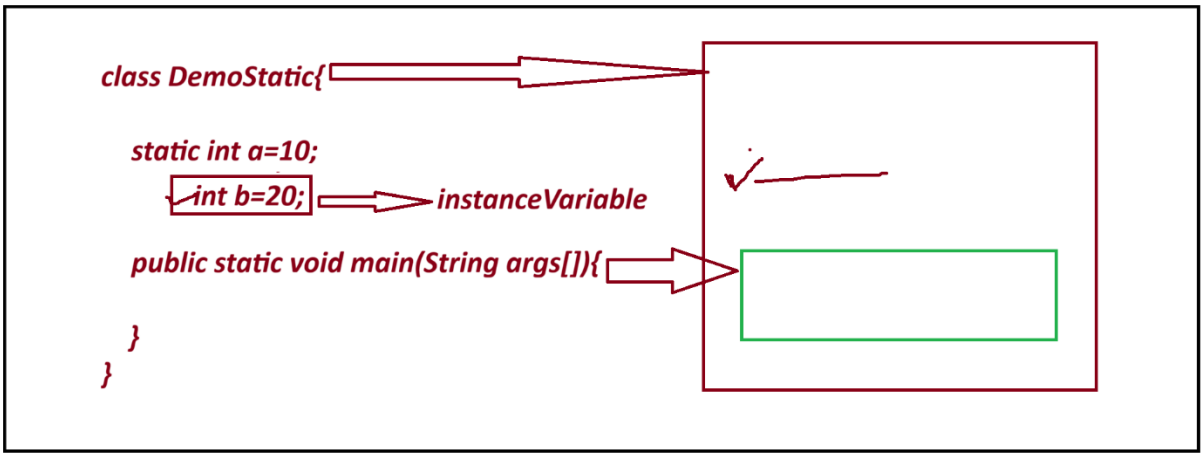
b. nonStatic programming components.

=> The programming components which are delcared without "static" keyword are known as NonStatic programming components.

=> These NonStatic programming components will get the memrory within the Object and can be accessed with Object_name.

==========





=> based on "static" keyword the variables are categorized into two types:

1. static variable

2. nonStatic variable

1. static variable:

=> The variable which are declared with "static" keyword are known as static variable or class variables.

=> These static variable will get the memory within the class while class loading and can be accessed with Class_name.

2. nonStatic variable

=> The variable which are declared without "static" keyword are known as NonStatic variables.

=> NonStatic variable are categorized into two types:

a. Instance Variables

b. Local Variables.

a. Instance Variables

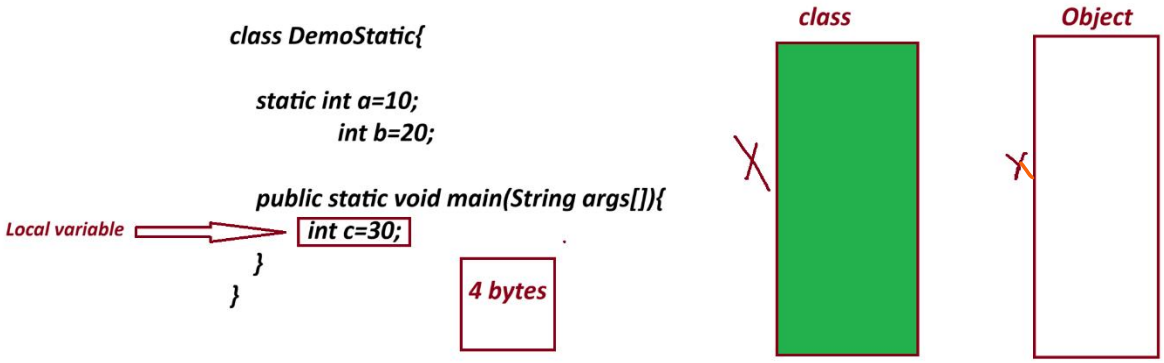
=> The NonStatic variable which are declared outside the method and inside class are known as instance Variables or Object_variable.

=> The instance variables will the get the memory within the object while Object creation process and can be accessed with Object_name.

b. Local Variables.

=> The NonStatic variable which are declared inside the methods are known as Local Variables or Method Variables.

=> These local variables will get the memory within the method while method exection and can be accessed directly inside the method.



==========

program :

DemoStatic.java

```
class DemoStatic{
        static int a=10;
          int b=20;
        public static void main(String args[]){
       int c=30;
     System.out.println("a value "+DemoStatic.a);
     DemoStatic ob=new DemoStatic();
     System.out.println("b value "+ob.b);
     System.out.println("c value "+c);
   }
}
```

output:

D:\core java programs>javac DemoStatic.java

D:\core java programs>java DemoStatic

a value 10

b value 20

c value 30

==========

session 14

==========

Method In Java

=> Methods are the actions which are performed to generate results.

=> based on "static" keyword the methods are categorized into two types:

1. static methods

2. NonStatic Methods (Instance methods)

1. static methods

=> The methods which are declared with "static" keyword are known as static methods or Class Method.

=> These static methods will get the memory within the class while class loading and can be accessed with Class_name.

structure of static methods:

```
static  return_type method_name(para_list)
{
        // method_body
}
```

=> These static methods are categorized into two types:

1. Pre-defined static methods

2. User defined static methods

1. Pre-defined static methods

=> The static methods which are constructed and available from javaLib are as known as Pre-defined static methods.

2. User defined static methods:

=> The static methods which are defined by the programmer are known as user defined static methods.

coding Rule:

=> static methods can access static variable of same class directly, but cannot access instance variable directly.

==========

ex program:

Method.java

```
class Method{
                    static int x=20;
              int y =30;
static  void m1()
{
          System.out.println("x value "+x);
      //System.out.println("y value "+y);
 }
public static void main(String[] arg){
   Method ob= new Method();
          ob.m1();
  }
}
```

output

```
D:\core java programs>javac Method.java
D:\core java programs>java Method
x value 20
```

==========

2. NonStatic Methods (Instance methods)

=> The methods which are declared without static keyword are known as NonStatic or Instance Methods.

=> These instance methods will get the memroy within the object while object creation process and can be accessed with Object_name.

Structure of Instance Method(NonStatic Methods)

```
return_type method_name(para_list)
{
        //method body
}
```

=> These Instance methods are categorized into two types:

1. Pre-defined Instance methods.
2. User defined Instance methods.

1. Pre-defined Instance methods.

=> The Instance methods which are constructed and available from javaLib are known as Pre-defined Instance methods.

2. User defined Instance methods.

=> The Instance methods which are defined by the programmer are known as user defined Instance methods.

coding Rule:

=> Instance methods can access both variables "static and Instance variable" directly, becuase the Object generated from classes and belongs to classes.

ex : program

Method.java

```
class Method{
        static int x;
          int y;

static  void m1()
```

```
{
        System.out.println("========static method m1()==========");
        System.out.println("x value "+x);

}
void m2()
 {
        System.out.println("======= non staic method m2()======");
        System.out.println("x value "+x);
        System.out.println("y value "+y);
 }
public static void main(String[] arg){
   Method ob=new Method();
   Method.m1();
  ob.m2();
  }
}
----------
output
D:\core java programs>java Method
========static method m1()==========
x value 0
======= non staic method m2()======
x value 0
y value 0
==============================
```

session 15

=========

JVM Architecture.

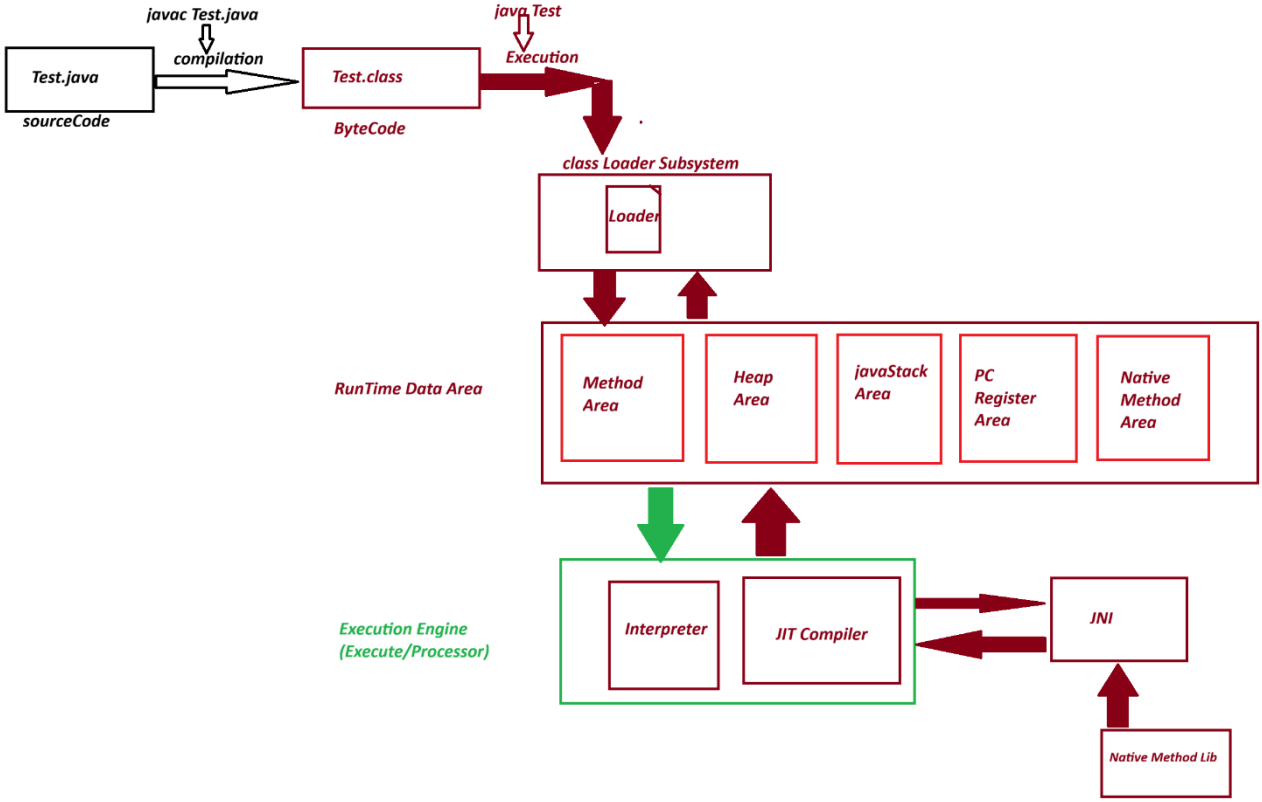
=> JVM Stands for Java Virtural Machine and which is used to execute javaByteCode.

=> This JVM Internally divided into the following three partition

1. Class Loader Subsystem

2. RunTime Data Area

3. Exectution Engine



---

1. Class Loader Subsystem

=> Class Loader Subsystem will load the class-file (byte-code) on to RunTime Data Area.

=> Class loadder Subsystem internally having loaders, linkers,verifiers and decoders.

2. RunTime Data Area

=> This Runtime data area internally divided into the following partition

1. Method Area

2. Heap Area

3. JavaStack Area

4. PC Register Area

5. Native Method Area

1. Method Area

=> This partition of Runtime data area where classes are loaded is known as Method Area.

=> while class loading static programming components will get the memory within the class.

=> In this process, main() method will get the memory within the class, and automatically copied onto JavaStackArea to start the execution-process

2. Heap Area

=> The partition of Runtime data Area where object are created is known as Heap Area.

=> while Object creation the Instance members will get the memory within the object

3. JavaStack Area

=> The partition of Runtime data Area where methods are executed is known as java Stack Area.

=> main() is the first method copied onto javaStackArea to start the execution process and main() will call remaining methods for execution.

4. PC Register Area

=> program Counter(PC) register will hold the status of method execution in java Stack Area.

=> Every method which is executing in javaStackArea will have its own program couner Register.

=> All these program counter registers are opened in a separated memory block known as PC register Area.

Advantage

======

=> Using the information in PC-Register the execution-engine will resume the execution process.

5. Native Method Area

=> The methods which are declred with "native" keyword in JavaLib are known as Native Methods.

=> These native methods internally having c/c++ codes.

=>when these native methods are used in application, then class loader subsystem will identify these methods and loads onto a separate memory block known as Native Method Area.

=>Execution-Engine will execute native methods using JNI(Java Native method Interface)

=> JNI while executing native methods used native method libraries.

Session 16

Define paramters?

=> Parameters are the variables which are used to transfer the data from one method to another method.

=> based on parameters the methods are categorized into two types:

(i) Methods without parameter

(ii) Methods with parameter.

(i) Methods without parameter

=> The methods which are declared without parameters are known as 0-Parameter methods without parameters.

(ii) Methods with parameter.

=> The methods which are declared with parameters are known as parameterized methods or Method parameters

example program

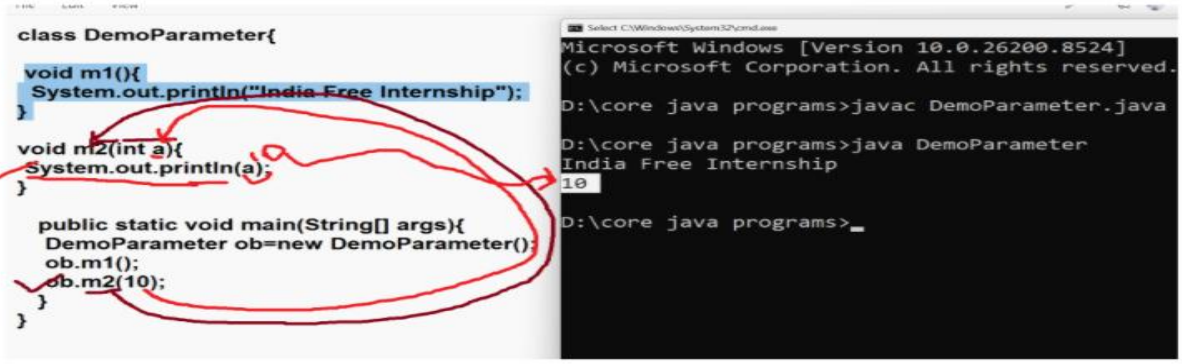
DemoParameter.java

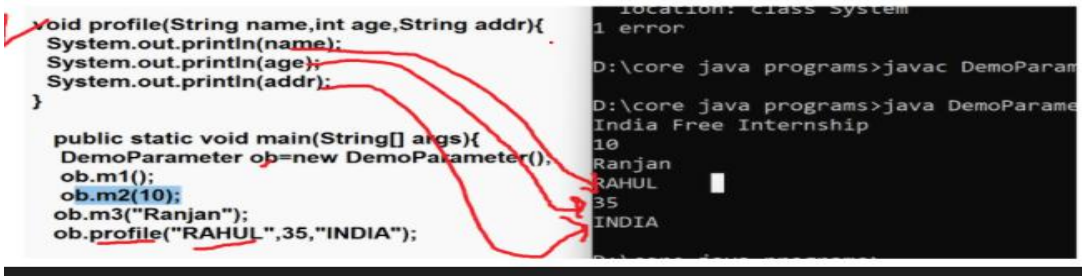
```
class DemoParameter{

 void m1(){
  System.out.println("India Free Internship");
}

void m2(int a){
 System.out.println(a);
}

void m3(String name){
        System.out.println(name);
}

void profile(String name,int age,String addr){
  System.out.println(name);
  System.out.println(age);
  System.out.println(addr);
}

   public static void main(String[] args){
    DemoParameter ob=new DemoParameter();
    ob.m1();
    ob.m2(10);
   ob.m3("Ranjan");
```

```
  ob.profile("RAHUL",35,"INDIA");


  }
}
```

output

```
--------
D:\core java programs>javac DemoParameter.java
D:\core java programs>java DemoParameter
India Free Internship
10
Ranjan
RAHUL
35
INDIA
```





Session 17

editPlus

download editPlus software

link : https://www.editplus.com/download.html

1. Return_Type

2. Scanner class

1. Return_Type

==========

=> "return_type" specify the methods will return the value after execution or not.

=> based on return_type the methods are categorized into two types:

1. Non return_type methods (void)

2. return_type methods (int, byte , short , long, class..)

1. Non return_type methods (void)

=> The methods which will not return, any value after method execution are known as Non return_type methods.

=> The methods which are declared with "void" are known as "Non return_type methods".

2. return_type methods

=> The methods which return the value after method exectution are known as return_type methos.

=> The methods which are declared without "void" are known as return_type methods.

example program

Demo.java

```
class Demo
{

        static int sum(int a,int b)
  {
        int c=a+b;
        return c;
  }

   static void mul(int x, int y){
                int z=x*y;

   }

        public static void main(String[] args){
                System.out.println("Session 17");
                int result=Demo.sum(10,20);
                System.out.println(result);

                //int mulResult=Demo.mul(2,3); error
                //System.out.println(mulResult);
```

```
    }
}
```

==========

output

Session 17

30

==========

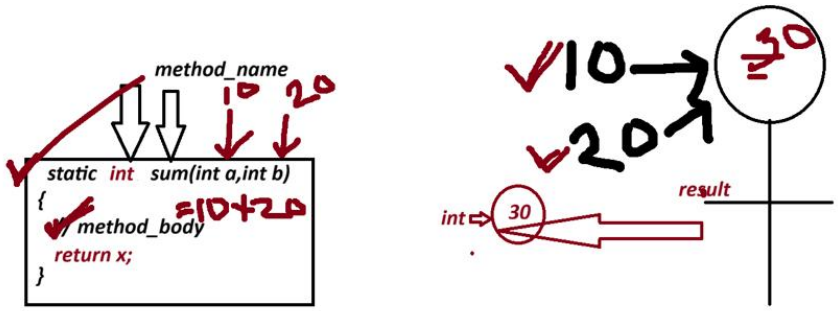
Scanner class

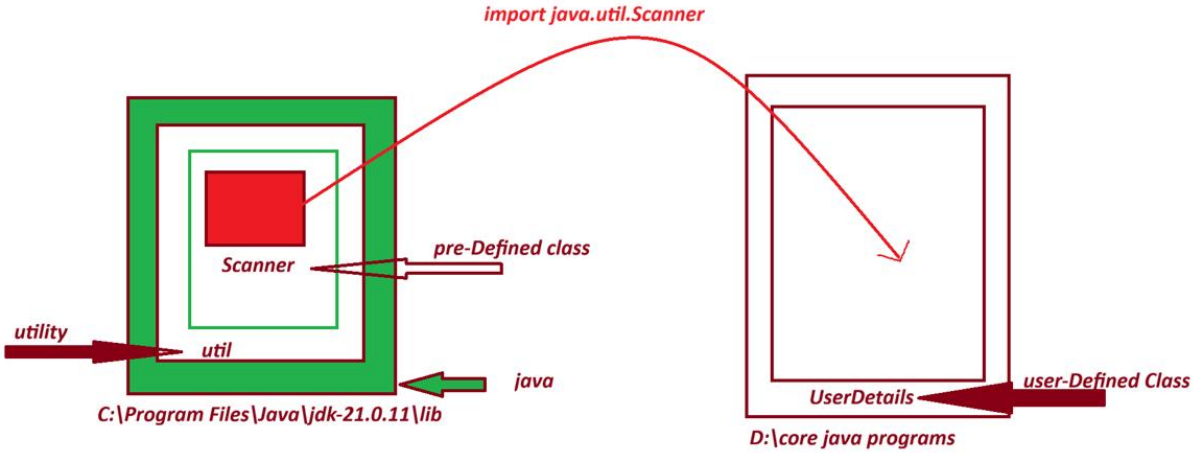
=> "Scanner" is a pre-defined class from "util"(Utility) package and which provide the following some important intance to read data into java program.

=>

1. **nextByte()**
2. **nextShort()**
3. **nextInt()**
4. **nextLong()**
5. **nextFloat()**
6. **nextDouble()**
7. **nextBoolean()**
8. **nextLine()**

==========





1. nextByte()=> used to read byte data

syntax:

byte val=s.nextByte();

2. nextShort() => used to read short data

syntax:

short var=s.nextShort();

3. nextInt() => used to read int data

syntax:

int var=s.nextInt();

4. nextLong() => used to read long data

syntax:

Long val=s.nextLong();

5. nextFloat()=> used to read float data

syntax:

float var=s.nextFloat();

6. nextDouble() => used to read double data

syntax:

double var=s.nextDouble();

7. nextBoolean()=> used to read boolean data

syntax:

boolean var=s.nextBoolean();

8. nextLine()=> used to read String data

syntax

String variableName=s.nextLine();

wap to read and display UserDetails?

(userName,mailId,phNo)

UserdDeatails.java

```
import java.util.Scanner;
class UserDetails
```

```
{


        public static void main(String[] args){
                Scanner s=new Scanner(System.in); // object Scanner
                System.out.println("Enter Username");
                String userName=s.nextLine();

                System.out.println("Enter mailId");
                String mailId=s.nextLine();

                System.out.println("Enter phone Number");
                Long phNo=s.nextLong();

                System.out.println("=======UserDetails==========");
                System.out.println("username :"+userName);
                System.out.println("mailId "+mailId);
                System.out.println("phone Number "+phNo);
        }
}
----------
output
D:\core java programs>javac UserDetails.java
D:\core java programs>java UserDetails
Enter Username
Rahul kumar
Enter mailId
```

indiafree123@gmail.com

Enter phone Number

7878123456

=======UserDetails==========

username :Rahul kumar

mailId indiafree123@gmail.com

phone Number 7878123456

Assignment 1

wap to read and display Book Details?

(name,author, price, qty)

Assignment 2:

wap to read and display Customer Details?

(custName,custCity,custMailid,custPhNo)

==========

session 18

==========

Assignment 1 (Solution)

---

wap to read and display Book Details?

(name,author, price, qty)

code

```
import java.util.Scanner;
class BookDetails
{
```

```
        public static void main(String[] args)
        {
                Scanner s=new Scanner(System.in);
                System.out.println("Enter the BookName: ");
                String name=s.nextLine();
                System.out.println("Enter the BookAuthor: ");
                String author=s.nextLine();
                System.out.println("Enter the BookPrice: ");
                int price=s.nextInt();
                System.out.println("Enter the bookQty");
                int qty=s.nextInt();
                System.out.println("==========BOOK DETAILS===========");
                System.out.println("BookName "+name);
                System.out.println("BookAuthor "+author);
                System.out.println("BookPrice "+price);
                System.out.println("BookQty "+qty);

        }
}
```

output

```
-------
Enter the BookName:
java-Language
Enter the BookAuthor:
B-Swamy
Enter the BookPrice:
400
```

Enter the bookQty

2

==========BOOK DETAILS==========

BookName java-Language

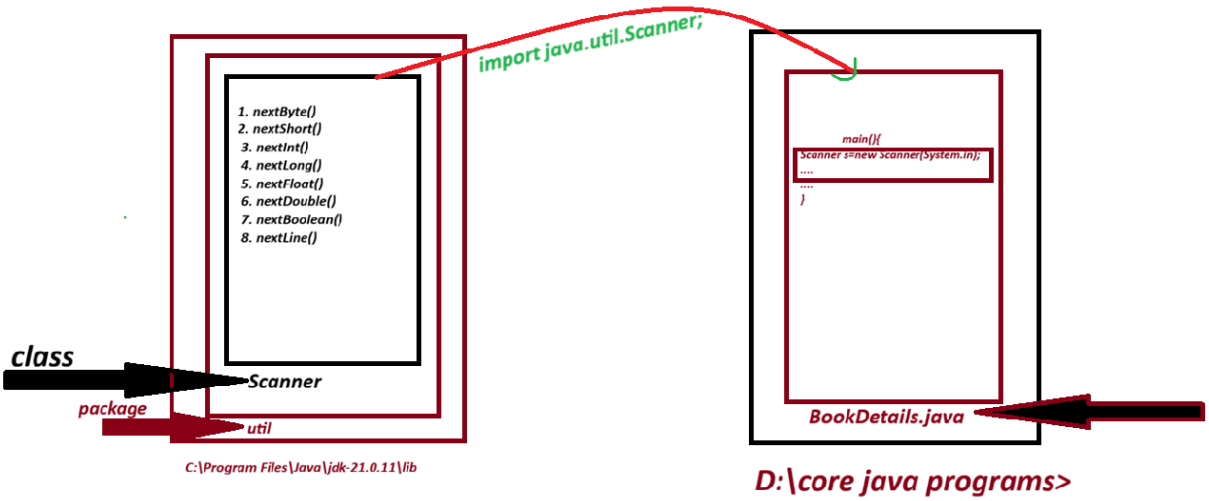
BookAuthor B-Swamy

BookPrice 400

BookQty 2

==========

Diagram



Operators In Java

==========

=> Operators are the special symbols which perform operations.

=> The following are some important operators used in java.

1. **Arithmetic Operators**
2. **Relational Operators**
3. **Logical Operators**
4. **Increment/Decrement Operators**

1. Arithmetic Operators

=> The Operators which perform basic operations or fundamental operations are known as Arithmetic Opertors.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	ModDivision

==========

2. Relational Operators

=> The Operators which are used to perform comaparions are known as Relational Operators.

Operator	Meaning
>	Greater Than
>=	Greater Than or equal
<	Less Than
<=	Less Than or equal
==	is equal
!=	Not equal to

==========

3. Logical Operators

=> The Operators which are used to compare two comparisions are known as Logical Operators.

Operator	Meaning
&&	Logical AND

|| Logical OR

! Logical NOT

==========

4. Increment/Decrement Operators

=> Increatment Operation will increament the value by-1 and decrement operatos will descrment the value by 1;

Operator	Meaning
++	Increment
--	Decrement

==========

Ex: wap to read 6 sub marks and display totMarks and Percentage?

```
import java.util.Scanner;
class StudentResult
{
	public static void main(String[] args)
	{
		Scanner s=new Scanner(System.in);
		System.out.println("Enter the marks of sub-1");
		int sub1=s.nextInt();
		System.out.println("Enter the marks of sub-2");
		int sub2=s.nextInt();
		System.out.println("Enter the marks of sub-3");
		int sub3=s.nextInt();
		System.out.println("Enter the marks of sub-4");
		int sub4=s.nextInt();
```

```
            System.out.println("Enter the marks of sub-5");
            int sub5=s.nextInt();
            System.out.println("Enter the marks of sub-6");
            int sub6=s.nextInt();

            int totMarks=sub1+sub2+sub3+sub4+sub5+sub6;
            float per=((float)totMarks/600)*100;

            System.out.println("==========StudentResult=========");
            System.out.println("Total Marks :"+totMarks);
            System.out.println("Percentage = "+per);
        }
}
```

output

```
=========
Enter the marks of sub-1
89
Enter the marks of sub-2
78
Enter the marks of sub-3
67
Enter the marks of sub-4
80
Enter the marks of sub-5
91
Enter the marks of sub-6
67
```

==========StudentResult==========

Total Marks :472

Percentage = 78.66667

========================

wap to read employee basicSal,hra,da and calculate totSal?

hra=91% of bSal

da=63% of bSal

totSal=bSal +hra+da

==========================

Control Structure in java.

====================

=> The Structures whicha are used by the programmer to control execution flow are known as Control Structures.

=> Control Structure in java are categorized into three types

1. Selection Statements

2. Iterative Statements

3. Branching Statements

1. Selection Statements

=> The Statement which are used to select one part of the program for execution are known as Selection Statements or Conditional Statement.

=> Types:

a. simple if

b. if-else

c. Nested if

d. Ladder if-else

e. switch-case

a. simple if:

=> simple if represents only true-block.

syntax:

```
if(condition){
//body
}
```

b. if-else:

=> if-else represents both true and false block.

syntax:

```
if(condition){
//body
}else{
    //body
}
```

c. Nested if :

=> The process of declaring "if" inside "if" is known as Nested-if or Inner-if.

syntax:

```
if(condition){
 if(condition){
  }
}
```

d. Ladder if-else

=> The process of interlinking more number of if-else blocks is known as ladder if-else

syntax:

```
if(condition){
        //body
}else if(condition){
        //body
}else if(condition){
        //body
}
.
.
.
```

e. switch-case:

=> switch-case statement is used to select one from multiple available options or cases.

syntax:

```
switch(value){
case 1: statements;
break;
case 2: statements;
break;
case 3: statements;
break;
...
..
..
case n: statements;
```

```
            break;
            default: statemetns;
            }
==============
session 19
==========
Ex-Program
----------
wap to read two integer values and display greater values?
(using subClass concept)
import java.util.Scanner;
class GreaterValue
{
    int greater(int x,int y){
        if(x>y){
            return x;
        }else{
            return y;
        }
    }
}

class DemoMethods1
{
    public static void main(String[] args)
    {
        Scanner s=new Scanner(System.in);
```

```
            System.out.println("Enter the int value-1:");
            int v1=s.nextInt();
            System.out.println("Enter the int value-2:");
            int v2=s.nextInt();
            GreaterValue gv=new GreaterValue();
            int res=gv.greater(v1,v2);
            System.out.println("Greater Value : "+res);
        }
}
```

---

output

Enter the int value-1:

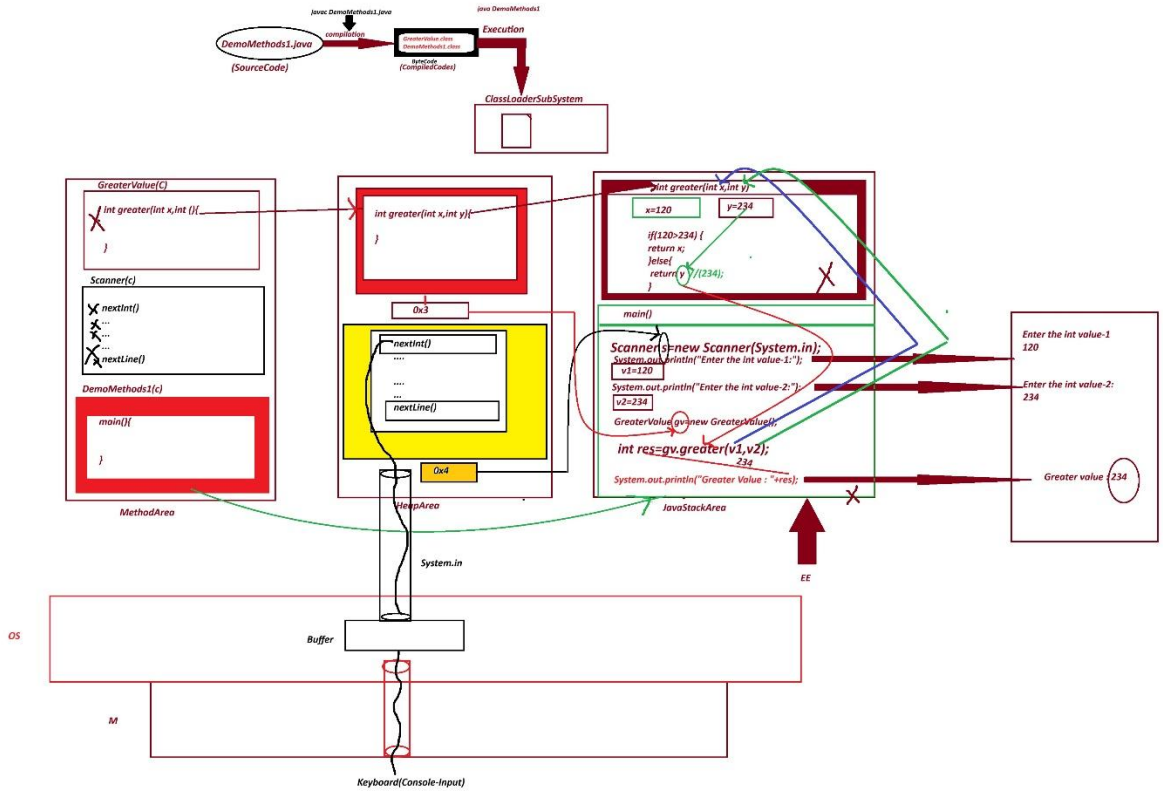
120

Enter the int value-2:

234

Greater Value : 234

Execution flow diagram



Assigment 1:

wap to read three integer values and display the greater value?

(using SubClass Concept)

Assignment 2:

Wap to read a value and display the value is Even or Not?

(using SubClass concept)

Even -display "true"

Odd- dispaly "false"

==========

session 20

=========

Variable from DemoMethods1.java

==========

variable : (s,v1,v2,gv,res,x,y)

primitive DataType variable : v1,v2, res,x,y

non-primitive DataType variable : s, gv

Actual Parameter (Argument) :

An actual parameter (argument) is the value or variable passed to a method when the method is invoked.

ex :

gv.greater(v1,v2);

Formal Parameter:

A formal parameter is a variable declared in a method definition that receives a value when the method is called.

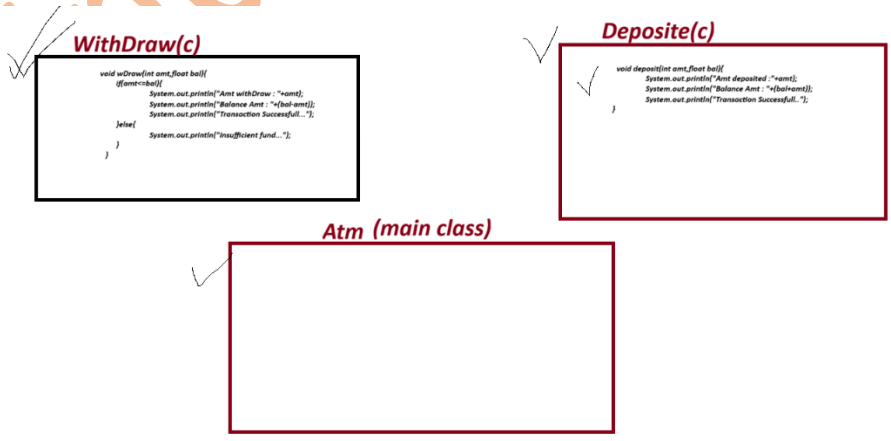
ex : int greater(int x,int y)

==========

Atm Machine Application

==========

wap to construct the application from the following layout:



ex Program

==========

```
import java.util.Scanner;
class WithDraw
{
	void wDraw(int amt,float bal){
		if(amt<=bal){
			System.out.println("Amt withDraw : "+amt);
			System.out.println("Balance Amt : "+(bal-amt));
			System.out.println("Transaction Successfull...");
		}else{
			System.out.println("Insufficient fund...");
		}
	}
}

class Deposit
{
	void deposit(int amt,float bal){
		System.out.println("Amt deposited :"+amt);
		System.out.println("Balance Amt : "+(bal+amt));
		System.out.println("Transaction Successfull..");
	}
}

class Atm
{
```

```
public static void main(String[] args)
{
        Scanner s=new Scanner(System.in);
        float bal=3000.0F;
        System.out.println("====choice=======");
        System.out.println("1. WithDraw \n2. Deposit");
        System.out.println("Enter the choice");
        int choice=s.nextInt();

        switch(choice){
            case 1:
                System.out.println("Enter the amount");
              int a1=s.nextInt();
                if(a1>0 && a1%100==0){
                    WithDraw wd=new WithDraw();
                    wd.wDraw(a1,bal);

                }else{
                    System.out.println("invalid amt..");
                }
                break;
            case 2:
                System.out.println("Enter the amt");
              int a2=s.nextInt();
                if(a2>0 && a2%100==0 ){

                Deposit dp=new Deposit();
```

```
                    dp.deposit(a2,bal);


                    }else{
                        System.out.println("invalid amt...");
                    }

                    break;
                default:
                    System.out.println("Invalid choice...");

            }

        }
}
```

==========

output

====choice=======

1. WithDraw

2. Deposit

Enter the choice

2

Enter the amt

1500

Amt deposited :1500

Balance Amt : 4500.0

Transaction Successfull..

==========

Assignment :

wap to read two integer values and perform Arithmetic operation based on user Choice

==========

User choice

=======

========choice==========

1. add

2. sub

3. mul

4. div

5. modDiv

condition : The two integer values must be greater than zero

Note :

=> In this Application, use 5 subClasses and one mainclass.

==========

session 21

========

Assigment 1:

wap to read three integer values and display the greater value?

(using SubClass Concept)

solution

----

program : GreatestValue.java

==========

```
import java.util.Scanner;
class Comparision
{
	int compare(int x,int y, int z){

		if(x>y && x>z){
			return x;
		}else if(y>x && y>z){
			return y;
		}else{
			return z;
		}
	}
}



class GreatestValue
{
	public static void main(String[] args)
	{
		Scanner s=new Scanner(System.in);
		System.out.println("Enter the value 1 ");
		int v1=s.nextInt();
		System.out.println("Enter the value 2 ");
		int v2=s.nextInt();
```

```
            System.out.println("Enter the value 3 ");
            int v3=s.nextInt();

            Comparision ob=new Comparision();
            int res=ob.compare(v1, v2, v3);
            System.out.println("Greatest value "+res);

        }
}
```

output

```
---------
Enter the value 1
10
Enter the value 2
20
Enter the value 3
45
Greatest value 45
========================
```

Assignment 2:

Wap to read a value and display the value is Even or Not?

(using SubClass concept)

Even -display "true"

Odd- dispaly "false"

solution-

```
import java.util.Scanner;
class Test
{

        boolean check(int n){
                if(n%2==0){
                        return true;
                }else{
                        return false;
                }
        }
}



class CheckEven
{
        public static void main(String[] args)
        {
                Scanner s=new Scanner(System.in);
                System.out.println("Enter number");
                int v1=s.nextInt();
                Test ob=new Test();
                boolean res=ob.check(v1);
                System.out.println(res);
```

```
        }
}
```

---

output

Enter number

88

true

==========

2. Iterative Statements

=> The statement which are used to execute some lines of program repeadtedly are known as Iterative statement or Repeatitive statement or Looping Structures.

=> The following are some important interative statement:

(a) while loop

(b) do while loop

(c) for loop

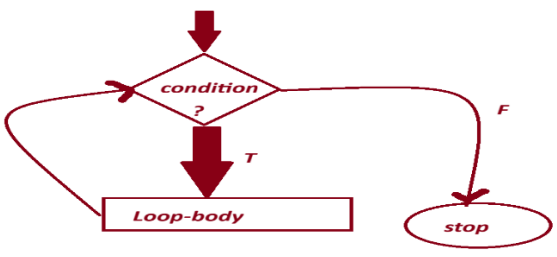
(a) while loop:

In while looping structure the condition is checked first and then the loop-body is executed, this process is repeated unitl the condition is false.

syntax:

```
while(condition){
        //loop body
}
```

diagram flow chart



==========

ex- wap to read a number and display the sum of digits in the given number?

input 123

res=1+2+3=6

program

```
import java.util.Scanner;
class AddOfDigits
{
    int s=0;
    int sum(int n){

        while(n>0){
            int k=n%10;
            s=s+k;
            n=n/10;
        }
        return s;
    }
}

class SumOfDigits
{
    public static void main(String[] args)
    {
        Scanner s=new Scanner(System.in);
        System.out.println("Enter number");
        int v=s.nextInt();
```

```
            AddOfDigits ob=new AddOfDigits();
            int res=ob.sum(v);
            System.out.println("sum of digits "+res);
        }
}
```

---

output

Enter number

456

sum of digits 15

Assignment :

Wap to read a number and display the reverse of given number?

input 123

output 321

==========

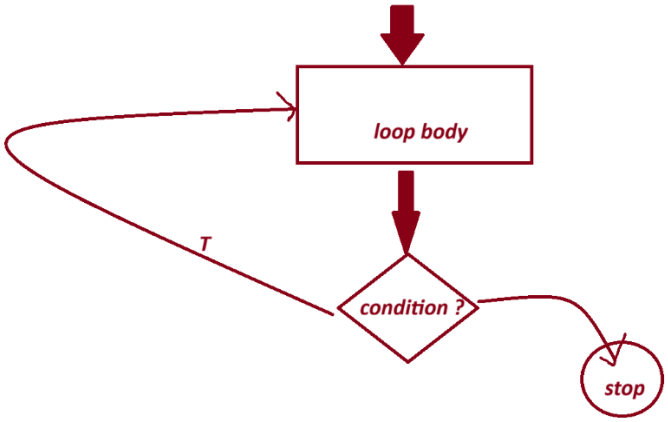
(b) do while loop

=> In do-while loop the loop-body is executed first and then Condition is checked, this process is repeated until condition is false.

syntax:

```
    do{
    //loop body
    }while(condition);
```

Flowchart



==========

disadvantage

=> In do-while loop, the loop body is exectured once for false condition and, which waste the exectution time and degrades the performances of an application.

Note:

=> In realtime do-while loop is less used when compared to while loop.

==========

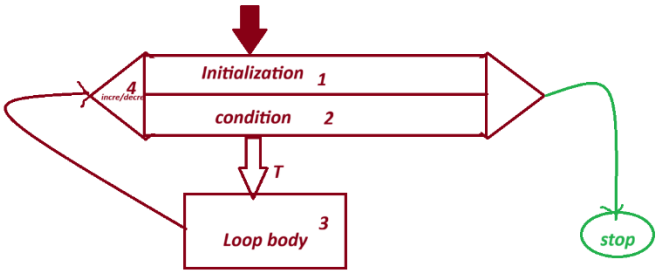
(c) for loop

=> for loop is more simple in representation when compared to while and do-while loops, because in for-loop the initialization, condition and increment/decrement are declared in the same line.

syntax:

```
for(Initialization; condition;incre/decre){
        //loop body
}
```

Flowchart



==========

session 22

==========

Ex

Wap to add and display n number?

input n=5

output 1+2+3+4+5=15

starting value 1:

ending value 5:

input n=6

ouput 1+2+3+4+5+6=21

ex program

```
import java.util.Scanner;
class SumOfDigits
{
        //1+2+3+4+5+6=21
```

```
        int sm=0;
        int sum(int n){

                for(int i=1;i<=n;i++){
                        sm=sm+i;
                }
                return sm;
        }
}


class SumOfNumberMain
{
        public static void main(String[] args)
        {
                Scanner s=new Scanner(System.in);
                System.out.println("Enter the value of n: ");
                int n=s.nextInt();
                SumOfDigits ob=new SumOfDigits();
                int res=ob.sum(n);
                System.out.println("Reslult " +res);
        }
}
```

---

starting value : 1

ending value : 6

```
 for(int i=1; i<=6;i++){
        System.out.println("india free internship");
 }
```

Note :

=> If the starting value is less than ending value then we must use Increment

=> If the starting value is greater than ending value then we must use Decrement

---

Ex:

Wap to read a number and display the factorial of given n number?

input n=4

4*3*2*1=24

```
import java.util.Scanner;
class Factorial
{
        int f=1;
        int fact(int n){
                for(int i=n;i>=1;i--){
                        f=f*i;
                }
```

```
            return f;
        }
}

class FactorialMain
{
        public static void main(String[] args)
        {
            Scanner s=new Scanner(System.in);
            System.out.println("Enter the value of n:");
            int n=s.nextInt();
            Factorial ob=new Factorial();
            int res=ob.fact(n);
            System.out.println("Result " +res);
        }
}
```

ouput

Enter the value of n:

5

Result 120

---

i-- // i=i-1;

start value : 4

ending value : 1

```
for(int i=4; i>=1;i--){
                        //loop body
 }
```

==========

Assignment 1:

wap to add and display multiples of 3 from given n numbers?

input n=10

1 2 3 4 5 6 7 8 9 10

output : 3+6+9=18

---

define prime number?

=> The number which is having only two factors is known as Prime Number.

Assignment 2:

wap to add and display prime number from given n numbers?

input n=10

1 2 3 4 5 6 7 8 9 10

output : 2+3+5+7=17

==========

session 23

==========

Assignment : solution

Wap to read a number and display the reverse of given number?

input 123

output 321

---

program

```
import java.util.Scanner;
class Reverse
{
	int r=0;

	int rev(int n){

		while(n>0){
			int k=n%10;
			r=(r*10)+k;
			n=n/10;
		}
		return r;
	}
}

class ReverseMain
{
	public static void main(String[] args)
	{
		Scanner s=new Scanner(System.in);
		System.out.println("Enter the value");
		int v1=s.nextInt();
		Reverse ob=new Reverse();
```

```
            int res=ob.rev(v1);
            System.out.println("Reverse number "+res);
        }
}
```

---

output

Enter the value

123

Reverse number 321

D:\core java programs>java ReverseMain

Enter the value

45879

Reverse number 97854

----- ----------

Assignment:

wap to check the given number is palindrome number or not?

palindrome=> The reverse of number is equal to given number is known as palindrome number.

ex 121 => 121

141 => 141

333=> 333

Assignment 1: (Solution)

wap to add and display multiples of 3 from given n numbers?

input n=10

1 2 3 4 5 6 7 8 9 10

output : 3+6+9=18

program

```
import java.util.Scanner;
class MultipleOfThree
{
        int sm=0;
        int sum(int n){

                for(int i=1;i<=n;i++){
                        if(i%3==0){
                                sm=sm+i;
                        }
                }
                return sm;
        }
}

class MultipleOfThreeMain
{
        public static void main(String[] args)
        {
                Scanner s=new Scanner(System.in);
                System.out.println("Enter the value of n");
                int n=s.nextInt();
                MultipleOfThree ob=new MultipleOfThree();
```

```
            int res=ob.sum(n);
            System.out.println(res);
        }
}
```

output

Enter the value of n

10

18

D:\core java programs>java MultipleOfThreeMain

Enter the value of n

15

45

==========

Assignment 2:

wap to display the reverse of all numbers within the range n?

int n=20

1 2 3 4 5 6 7 8 9 01 11 21 31 41 51 61 71 81 91 02

Assignment 3:

wap to add and display prime numbers from given n numbers?

input n =10;

1 2 3 4 5 6 7 8 9 10

ouput 2 +3+ 5+7=17

practice from this playlist link

https://www.youtube.com/watch?v=dhl1_Xln6kw&list=PLfGnHbLGc2pmsS7wCK0jNvncldXMSO5AD

==========

session 24

=========

3. Branching Statements

=>The statement which are used to transfer the execution -control from one location to another location are known as Branching Statement or Transfer Statement.

=> The following are some important Branching Statements:

(a) break

(b) continue

(c) return

(d) exit

(a) break

=> "break" is used to stop switch-case execution and also used to stop iterative-statement.

(b) continue

=> "continue" is used to skip the lines from the iteration based on condition

(c) return

=> "return" is used to transfer the value from the method after method execution.

("return" is used to return the value after method exectution)

(d) exit

=> "exit" is used to stop the program execution.

Note:

=> "goto" statement not available in java.

==========

Assignment :

wap to read two integer values and perform Arithmetic operation based on user Choice

==========

User choice

=======

========choice==========

1. add

2. sub

3. mul

4. div

5. modDiv

condition : The two integer values must be greater than zero

Note :

=> In this Application, use 5 subClasses and one mainclass.

Solution

```
import java.util.Scanner;

class Add

{
```

```
        int add(int x,int y){
                return x+y;
        }
}
class Sub
{
        int sub(int x,int y){
                return x-y;
        }
}
class Mul
{
        int mul(int x,int y){
                return x*y;
        }
}
class Div
{
        float div(int x,int y){
                return (float)x/y;
        }
}
class ModDiv
{
        int modDiv(int x,int y){
                return x%y;
        }
```

```
}

class CalculatorMain
{
        public static void main(String[] args)
        {
                Scanner s=new Scanner(System.in);
                System.out.println("Enter the int value -1:");
           int v1=s.nextInt();
                System.out.println("Enter the inv value -2:");
                int v2=s.nextInt();

                if(v1>0 && v2>0){
                        System.out.println("==========CHOICE=======");
                        System.out.println("1.add\n2.sub\n3.mul\n4.div\n5.modDiv");
                        System.out.println("Enter the choice:");
                        int choice=s.nextInt();

                        switch(choice){
                                case 1:
                                        Add ad=new Add();
                                  int r1=ad.add(v1,v2);
                                  System.out.println("Addition "+r1);
                                        break;
                                case 2:
                                        Sub sb=new Sub();
                                        int r2=sb.sub(v1,v2);
```

```
                System.out.println("Subtraction "+r2);
                break;
            case 3:
                Mul ml=new Mul();
              int r3=ml.mul(v1,v2);
                System.out.println("Multiplication "+r3);
                break;
            case 4:
                Div dv=new Div();
              float r4=dv.div(v1,v2);
                break;
            case 5:
                ModDiv md=new ModDiv();
              int r5=md.modDiv(v1,v2);
                System.out.println("Mod Division "+r5);
                break;
            default:
                System.out.println("Invalid choice...");
            }

            }// end of if
            else{
                System.out.println("Invalid values...");
            }
        }
}
```

---

ouput

Enter the int value -1:

40

Enter the inv value -2:

15

==========CHOICE=======

1.add

2.sub

3.mul

4.div

5.modDiv

Enter the choice:

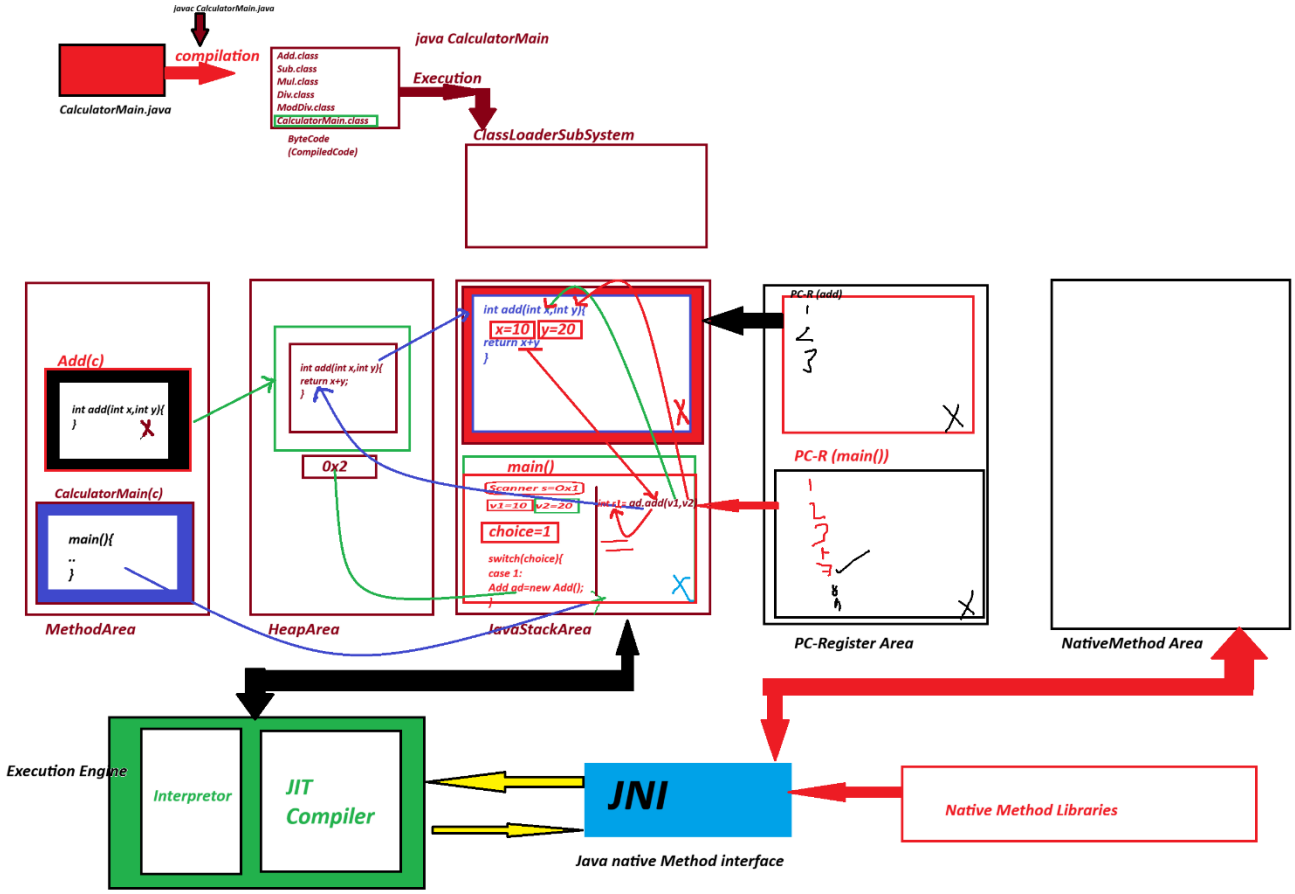
1

Addition 55

execution flow of above program

==========

Diagram



Session 25

==========

Native method Area

==========

=> The methods from the library which are delcared with 'native' keyword are known as native methods.

=> These native methods from the library will have c or c++ codes.

=> when these methods are used in the program, the classLoader subsystem will identify these methods and load on to a separate memory location known as Native method Area.

Diagram

Native Method Area

-Native Method Area is a memory area in JVM.
-It Stores information realated to native methods.
-Native mthods are written in languages like c/c++ instead of java
-When a native method is called, its related information is stored in this area.

Example:

```
public class Test{
 public native void display();
}
```

Here 'native' keyword means the method implementation is not in java

---

Execution Engine:

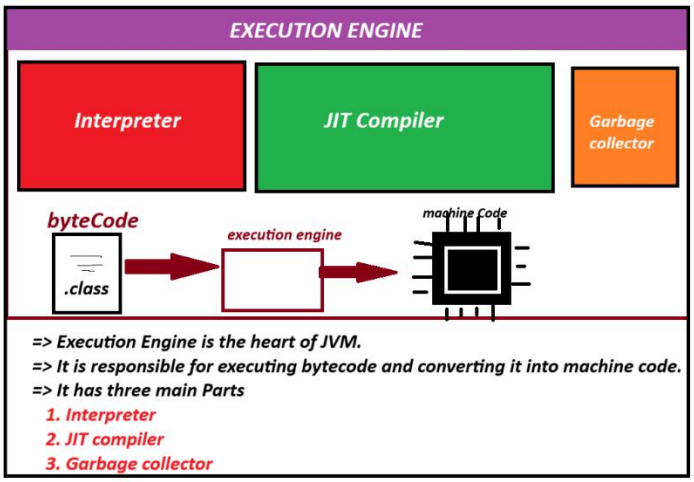
=>Execution Engine is an executor of JVM and which is also known as Virtual process of JVM.

=> This Execution Engine will start the execution process with main() available in JavaStackArea.

=> This ExecutionEngine internally having the following three translators.

1. **Interpreter**
2. **JIT(Just-In-Time) compiler**
3. **Garbage collector.**

Diagram



---

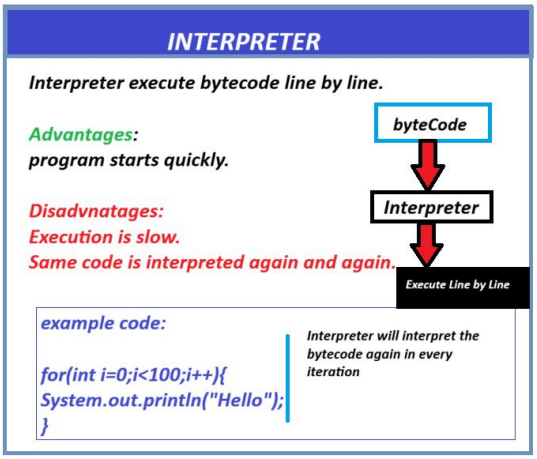
1. Interpreter

=> Interpreter will start the execution process and executes normal Instructions.

(Normal Instructions mean Non-Multimedia Instructions)

=> when interpreter finds the MultiMedia(Stream) Instruction then the execution control is transferred to JIT-Compiler.

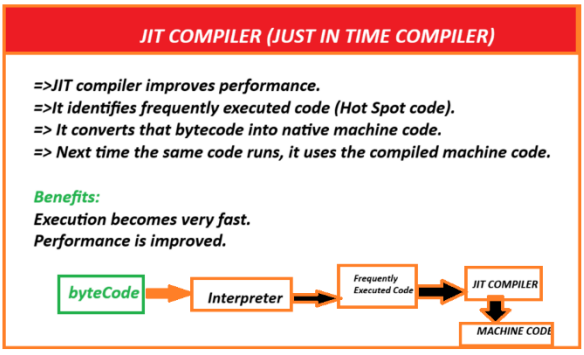
Diagram



JIT(Just-In-Time) compiler

=> JIT-Compiler will execute MultiMedia instructions or Stream Instruction, which means it executes Bulk-Instructions.

Diagram



==========

session 26

Constructor in Java:

=>Constructor is a method having the same name of the class and executed while object creation process, because the constructor call is available in Object creation process attached with 'new' keyword.

=> while declaring constructors we must not use return_type because the Constructor will have Class_return_type.

Structure of Constructor:

```
Class_name(para_list){
// method_body
}
```

Type of constructor:

=> based on prameters the constructor are categorized into two types:

1. **constructor without parameters**
2. **constructor with parameter**

1. constructor without parameters:

=> Constructors which are declared without parameters are known as 0-parameter constructor or 1. constructor without parameters

==========

example program

==========

```
class CTest1
{
    int a=10;
```

```
        CTest1(){
   System.out.println("**********Constructor CTest1() **********");
        System.out.println("The value a: "+a);
    }

    void dis(){
        System.out.println("**********isntance -dis()*********");
        System.out.println("The value a:"+ a);
    }

}

class DemoCon1
{
        public static void main(String[] args)
        {
                CTest1 ob=new CTest1(); // Con_Call
                ob.dis();

        }
}
```

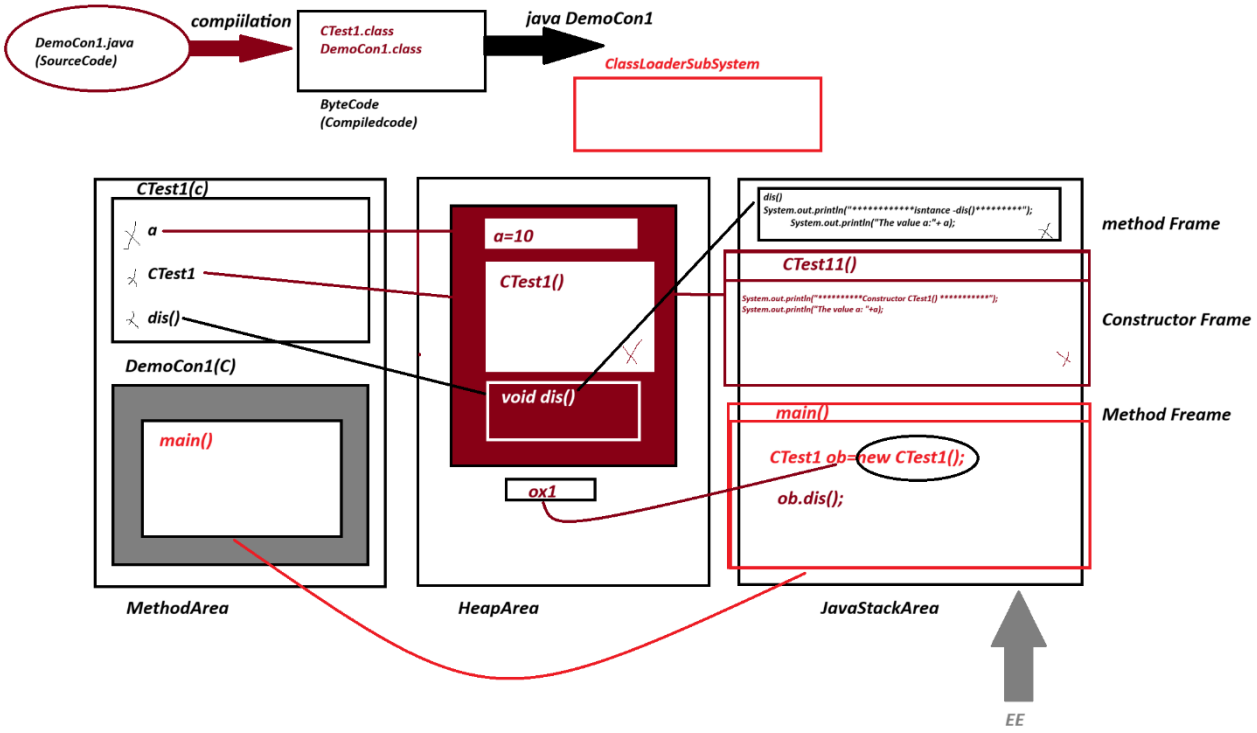
---

output

```
**********Constructor CTest1() **********
The value a: 10
**********isntance -dis()*********
```

The value a:10

Diagram



---

what is the diff b/w

(1) construtor

(2) Instance method

=> contructor is executed while object creation process, but instance method is executed after object creation.

=> constrcutor is executed only once while Object creation process, but intance method can be executed any number of times after object creation process.

==========

Define static Constructor in java?

There is no concept of static constructors in java, because contructor means exectued while object creation and cannot be a class level component. (CompileTime Error);

==========

Define Default Constructor?

The Constructor without parameters added by the compiler at compilation stage is known as default construtor.

In what situtation default construtor is added?

=> compiler at compilation stage finds any class declared without any constructor then default constructor is added.

Session 27

==========

Constructor with parameter

=> The Constructor which are declared with parameters are known as parameterized constructor or Constructor with parameter.

=> Parameterized Constructor is used to load the data to object while object creation process.

example program

```
import java.util.Scanner;
class Customer
{

        String cId,cName,mId;
        long phNo;

        Customer(String a,String b,String c,long d){
```

```
            cId=a;
            cName=b;
            mId=c;
            phNo=d;
        }

        void getCustomer(){
            System.out.println("===========Customer Details==========");
            System.out.println("CustomerId "+cId);
            System.out.println("CustomerName "+cName);
            System.out.println("CustomerMailId "+mId);
            System.out.println("CustomerPhNo "+phNo);
        }

}


class Constructor2
{
        public static void main(String[] args)
        {
            Scanner s=new Scanner(System.in);
            System.out.println("Enter the CustomerId");
            String id=s.nextLine();
            System.out.println("Enter the CustomerName");
            String name=s.nextLine();
            System.out.println("Enter the CustomerMailId");
```

```
            String mid=s.nextLine();
            System.out.println("Enter the CustomerPhoneNumber");
            long phNo=s.nextLong();

            Customer ob=new Customer(id,name,mid,phNo);//Con_Call
            ob.getCustomer();//method_Call

        }
}
```

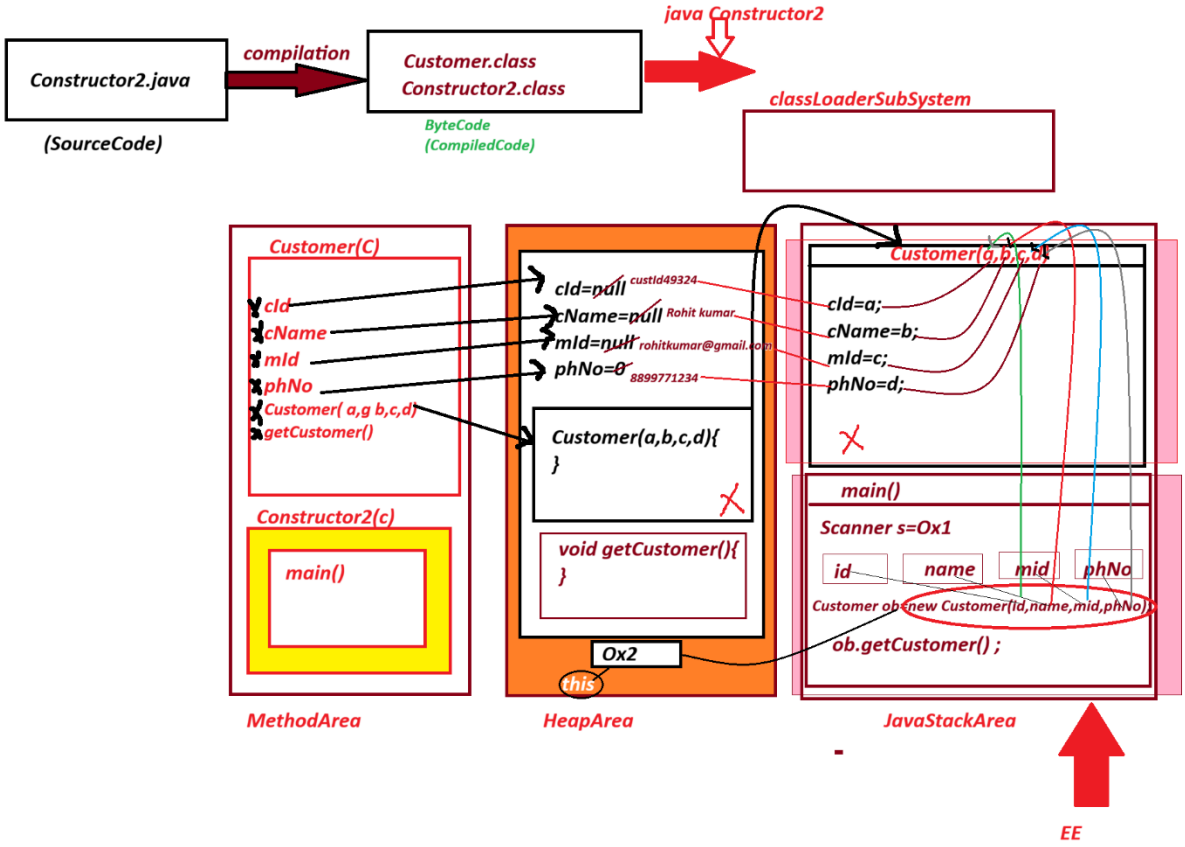
==========

output

```
Enter the CustomerId
CUSTID123456987
Enter the CustomerName
ROHIT KUMAR
Enter the CustomerMailId
rohitkumar@gmail.com
Enter the CustomerPhoneNumber
8899771234
==========Customer Details==========
CustomerId CUSTID123456987
CustomerName ROHIT KUMAR
CustomerMailId rohitkumar@gmail.com
CustomerPhNo 8899771234
==========
```

Execution flow of above program

Diagram



Advantage of Cosntructor:

=> Constructor are used to initialize instance variable while object creation process, and which saves the execution time and generate high performance of an application.

==========

define "this" keyword?

=> "this" is a pre-defined reference variable, which will hold reference of Object from where the Construtor or method exectuting.

=> "this" keyword is used when we want to load data from Local Variable to Instance Variable having the same names.

========

example program

import java.util.Scanner;

class Customer

```
{

        String cId,cName,mId;
        long phNo;

        Customer(String cId,String cName,String mId,long phNo){
                this.cId=cId;
                this.cName=cName;
                this.mId=mId;
                this.phNo=phNo;
        }

        void getCustomer(){
                System.out.println("==========Customer Details==========");
                System.out.println("CustomerId "+cId);
                System.out.println("CustomerName "+cName);
                System.out.println("CustomerMailId "+mId);
                System.out.println("CustomerPhNo "+phNo);
        }

}


class Constructor2
{
        public static void main(String[] args)
        {
```

```
            Scanner s=new Scanner(System.in);
            System.out.println("Enter the CustomerId");
            String id=s.nextLine();
            System.out.println("Enter the CustomerName");
            String name=s.nextLine();
            System.out.println("Enter the CustomerMailId");
            String mid=s.nextLine();
            System.out.println("Enter the CustomerPhoneNumber");
            long phNo=s.nextLong();

            Customer ob=new Customer(id,name,mid,phNo);//Con_Call
            ob.getCustomer();//method_Call

        }
}
```

==========

output

```
Enter the CustomerId
CUSTID123456987
Enter the CustomerName
ROHIT KUMAR
Enter the CustomerMailId
rohitkumar@gmail.com
Enter the CustomerPhoneNumber
8899771234
==========Customer Details==========
CustomerId CUSTID123456987
```

CustomerName ROHIT KUMAR

CustomerMailId rohitkumar@gmail.com

CustomerPhNo 8899771234

Assigment:

wap to read and Display BookDetails?

subclass : BookDetails(code,name,author,price)

MainClass: MainClassName.java

Note:

=> use Constructor to load the BookDetails.

==========

Assigment:

wap to read and display AddressDetails?

subclass : AddressDetails(hNo,sName,city,state,pinCode)

MainClass: MainClassName.java

Note:

=> use Constructor to load the AddressDetails.

==========

Note:

=> Draw Exectution flow of above programs.

==========

Session 28

session 28

=========

Loading data to Objects:

1. Using Construtors

2. Using Object reference variable

3. Using Setter methods

1. Using Construtors=>

we can load the data to Object using Constructor while object creation process.

Assigment:

wap to read and Display BookDetails(using construtor)?

subclass : BookDetails(code,name,author,price)

MainClass: MainClassName.java

Note:

=> use Constructor to load the BookDetails.

solution

BookDetails.java==> .class

Constructor3.java==> .class

```
import java.util.Scanner;
class BookDetails
{
        String code;
        String name;
```

```
        String author;
        float price;

        BookDetails(String code,String name,String author,float price){
                this.code=code;
                this.name=name;
                this.author=author;
                this.price=price;

        }

        //getter method
        void getBookDetails(){
                System.out.println("================BookDetails============");
                System.out.println("BookCode :"+code);
                System.out.println("BookName "+name);
                System.out.println("BookAuthor :"+author);
                System.out.println("BookPrice :"+price);

        }
}

class Constructor3
{
        public static void main(String[] args)
        {
```

```
        Scanner s=new Scanner(System.in);
        System.out.println("Enter the BookCode");
        String bCode=s.nextLine();
        System.out.println("Enter the BookName");
        String bName=s.nextLine();
        System.out.println("Enter the BookAuthor");
        String bAuthor=s.nextLine();
        System.out.println("Enter the BookPrice");
        float bPrice=s.nextFloat();

        BookDetails ob=new BookDetails(bCode,bName,bAuthor,bPrice); //
Con_Call
        ob.getBookDetails();



    }
}

----------
output
Enter the BookCode
J0001
Enter the BookName
java
Enter the BookAuthor
bala guru swamy
```

Enter the BookPrice

400

==========BookDetails==========

BookCode :J0001

BookName java

BookAuthor :bala guru swamy

BookPrice :400.0

==========

2. Using Object reference variable

=> we can also load the data to objects using Object reference variable or Object name;

example pgogram

```
import java.util.Scanner;
class Student
{
        String rollNo,name,branch;

        void getStudent(){
                System.out.println("==========Student Details==========");
                System.out.println("Student rollNO "+rollNo);
                System.out.println("Student name "+name);
                System.out.println("Student branch "+branch);
        }

}
```

```
class Constructor4
{
	public static void main(String[] args)
	{
		Scanner s=new Scanner(System.in);
		Student ob=new Student();// Con_Call
		System.out.println("Enter the RollNo");
		/*String rNo=s.nextLine();
		ob.rollNo=rNo;*/

		ob.rollNo=s.nextLine();
		System.out.println("Enter the Student Name");
		ob.name=s.nextLine();
		System.out.println("Enter the Student branch");
		ob.branch=s.nextLine();
		ob.getStudent();

	}
}

----------
output
Enter the RollNo
CS15243
Enter the Student Name
Chandan kumar
Enter the Student branch
```

CSE

==========Student Details==========

Student rollNO CS15243

Student name Chandan kumar

Student branch CSE

==

3. Using Setter methods

=> we load the data to object using "Setter methods".

Define Setter methods?

=> The method which are used to set the data to object are known as "Setter method".

define Getter methods?

=> The methods which are used to get the data from object are known as "Getter Methods".

Coding Rule for Setter and Getter methods:

=> Every variable in the class mut have its own Setter and Getter method.

Example program

```
import java.util.Scanner;
class CourseDetails
{
        String id;
        String name;
        int duration;
```

```
float fee;

void setId(String id){
        this.id=id;
}

String getId(){
        return id;
}


void setName(String name){
        this.name=name;
}

String getName(){
        return name;
}

void setDuration(int duration){
        this.duration=duration;
}

int getDuration(){
        return duration;
}
```

```
        void setFee(float fee){
                this.fee=fee;
        }
        float getFee(){
                return fee;
        }



}

class Constructor5
{
        public static void main(String[] args)
        {
                Scanner s=new Scanner(System.in);
                CourseDetails ob=new CourseDetails();
                System.out.println("Enter the CourseId");
                ob.setId(s.nextLine());
                System.out.println("Enter the CousreName");
                ob.setName(s.nextLine());
                System.out.println("Enter the CourseDuration");
                ob.setDuration(s.nextInt());
                System.out.println("Enter the CourseFee");
                ob.setFee(s.nextFloat());
                System.out.println("========CousreDetails=========");
                System.out.println("CourseID "+ob.getId());
```

```
            System.out.println("CourseName "+ob.getName());
            System.out.println("CousreDuration "+ob.getDuration());
            System.out.println("CourseFee "+ob.getFee());

        }
}
```

output-

```
----------
Enter the CourseId
SE3697
Enter the CousreName
Soft Eng
Enter the CourseDuration
6
Enter the CourseFee
3000
========CousreDetails=========
CourseID SE3697
CourseName Soft Eng
CousreDuration 6
CourseFee 3000.0
==================
```

Note:

=> using "object reference variable" and "Setter Method" we can load the data to object after object creation process.

=> using Constructor we can load the data while object creation process and which generate high performance of an applications.

==========

Session 29

==========

Blocks in java:

=> The set-of -statement which are declared within the flower-brackets("{}") and executed automatically is known as blocks.

=>Blocks in java are categorized into two types:

1. static blocks
2. Instance blocks(NotStatic blocks)

1. static blocks:

=>The blocks which are declared with "static" keyword are known as Static blocks:

syntax:

```
static
{
    //statement
}
```

Execution behaviour:

=> static block is executed only once with highest priority when the class is used for the first time.

=> static block also use static variables directly, but cannot use Instance variables.

program : Block1.java

```
class Block1
{
```

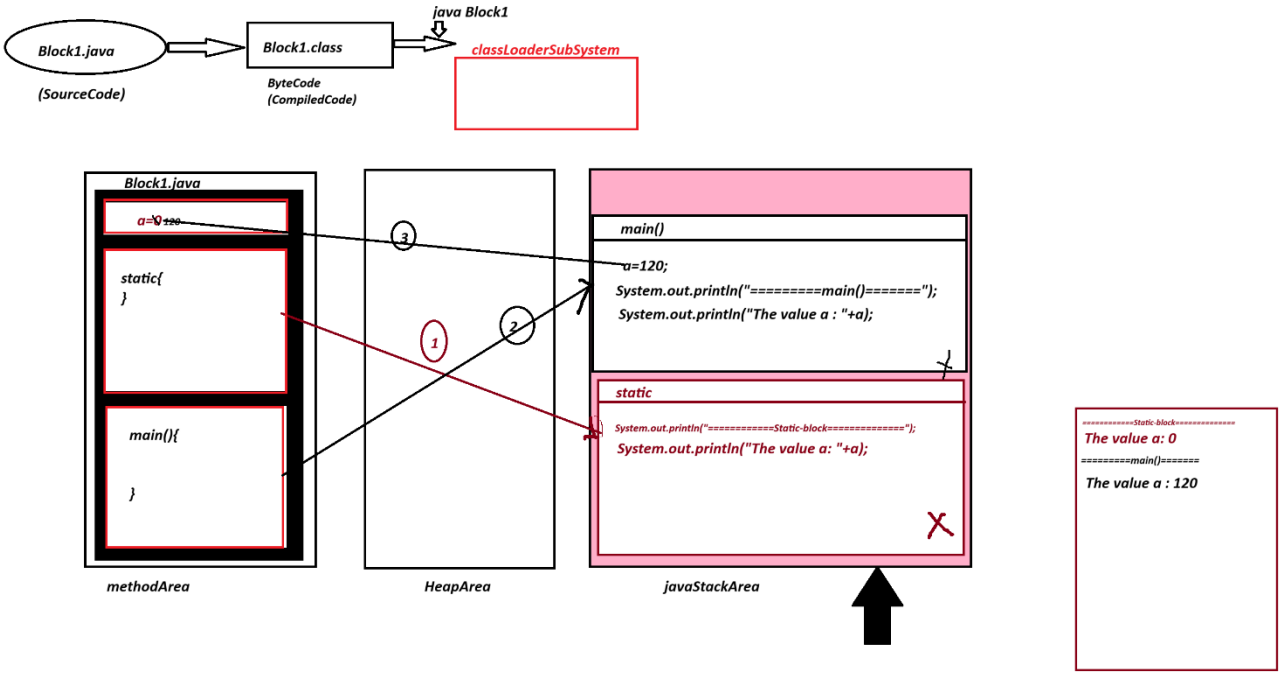
```
        static int a;

        static{
                System.out.println("============Static-block==============");
                System.out.println("The value a: "+a);
        }

        public static void main(String[] args)
        {
                a=120;
                System.out.println("=========main()=======");
                System.out.println("The value a : "+a);
        }
}
```

---

output

```
============Static-block==============
The value a: 0
=========main()=======
The value a : 120
```

Execution flow of above proram (Block1.java)



==========

Note:

=> static block which are declared within the MainClass willhave highest priority in exectution the main() method.

program : Block2.java(using subclass concept static-block)

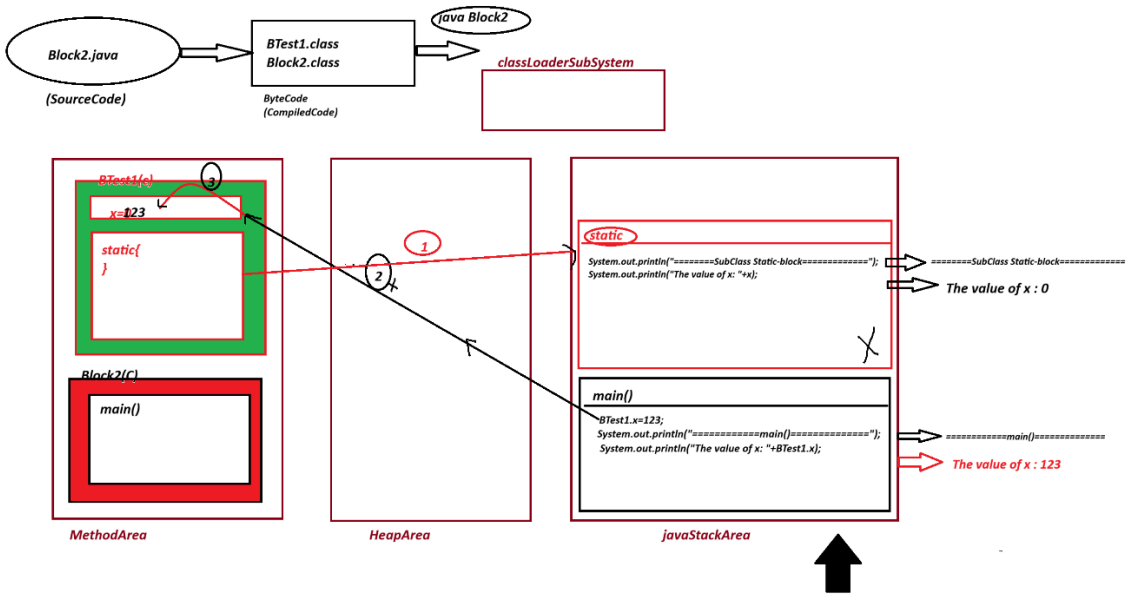
```
class BTest1
{
        static int x;

        static{
                System.out.println("========SubClass Static-block============");
                System.out.println("The value of x: "+x);
        }
}
```

```
class Block2
{
	public static void main(String[] args)
	{
		BTest1.x=123;
		System.out.println("============main()==============");
		System.out.println("The value of x: "+BTest1.x);
	}
}
```

output

```
----------
========SubClass Static-block============
The value of x: 0
============main()==============
The value of x: 123
```

Diagram



Note:

=> In realtime static-block is used to hold Database connection code part of DAO(Data Access Object) layer in MVC(Model View Controller)

==========

2. Instance blocks(NotStatic blocks):

=> The blocks which are declared without "static" keyword are known as NonStatic blocks or Instance blocks.

syntax:

```
{
        // statement
 }
```

Execution behaviour:

=> Instance blocks are executed while object creation process.

=> Instance blocks are executed for all multiple Object creations.

=> Instance blocks can access both variable, which mean can access Instance variable and Static variable.

==========

program Block3.java

```
class BTest2
{
        int x;
        static int y;
        {
         x++;
```

```
        y++;
                System.out.println("===========Subclass Instance-block==========");
                System.out.println("The value of x: "+x);
                System.out.println("The value of y: "+y);
        }

}
class Block3
{
        public static void main(String[] args)
        {
                System.out.println("----------Object1---------");
                BTest2 ob1=new BTest2();
                System.out.println("------Object2-------------");
                BTest2 ob2=new BTest2();
        }
}
---------------
output
----------Object1---------
===========Subclass Instance-block==========
The value of x: 1
The value of y: 1
------Object2-------------
===========Subclass Instance-block==========
The value of x: 1
The value of y: 2
```

==========

session 30

==========

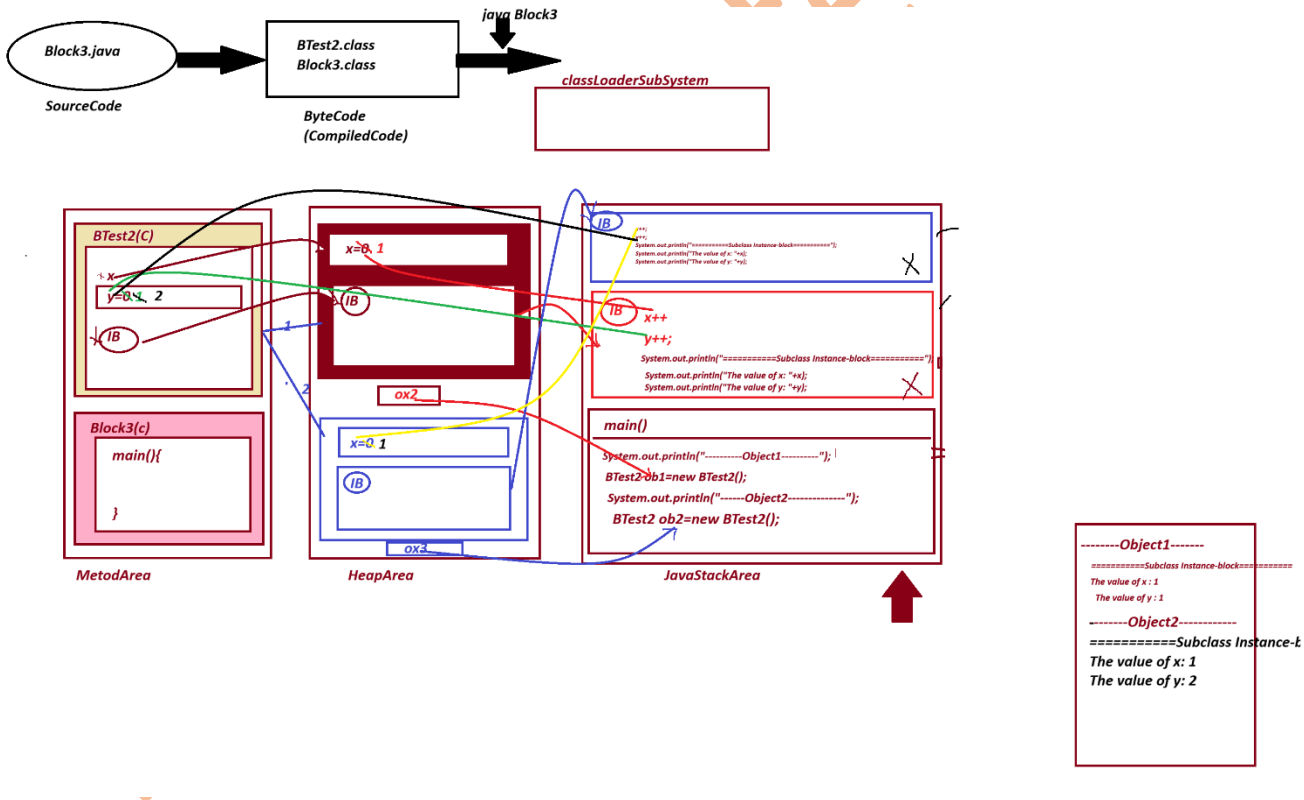
Execution Flow of above program

ClassFile

BTest2.class

Block3.class(Main class)

Diagram



---

class Generating Multiple Object:

=> class in java can generate multilple Objects and the multiple Object are independed by their memory location on HeapArea of JVM.

=> The class loads only once onto method Arae to generate multiple objects.

==========

Note:

=> when we want to share any programming component among multiple Objects then the programming components must be in Class_level, which means the programming components must be static.

==========

What is the difference b/w

(i) method Frame

(ii) Constructor Frame

(iii) Block Frame

(i) method Frame

=> The partition of java Stack Area where method is copied for execution is known as method Frame.

(ii) Constructor Frame

=> The partition of java Stack Area where Construtor is copied for execution is known as construtor Frame.

(iii) Block Frame

=> The partition of java Stack ARea where block is copied for execution is known as Block Frame.

==========

what is the diff b/w

(i) method

(ii) Block

(i) method=> Methods are executed on method call, but blocks are executed automatically.

(ii) Block=> Blocks willhave highest priority in execution than methods.

Note:

=> Static block will have highest priority in execution than static methods.

=> Instance blocks will have highest priority in execution than Instance methods.

==========

What is the diff b/w

(i) Instance block

(ii) Constructor

=> Both component are executed while Object creation process, but instance block will have highest priority in execution than construtor becuase constructor is a method.

========

what is the diff b/w

(i) static block

(ii) Constructor

=> Both Component are executed only once, but Static block is executed with highest priority when teh class is used for first time and Constructor is executed while Object creation

==========

example program

```
class BTest4
{
```

```
        BTest4(){
                System.out.println("=========Constructor=======");
        }

        {
                System.out.println("=========Instance-Block====");
        }

        static{

                System.out.println("========Static block=======");
        }
}

class Block4
{
        public static void main(String[] args)
        {
                System.out.println("=====Object1========");
                BTest4 ob1=new BTest4();
                System.out.println("=====Object2=========");
                BTest4 ob2=new BTest4();
        }
}
```

---

output

```
=====Object1========
```

========Static block=======

=========Instance-Block====

=========Constructor=======

=====Object2=========

=========Instance-Block====

=========Constructor=======

---

session 31

==========

Naming Convention in java

=>The coding rules followed by thr programmer in writing programs are known as Naming Conventions in java.

1. package

def : package are collection of "Classess and interfaces"

rule: package must be in LowerCase.

2. Classess and Interface

def : Classess and Interface are collection of "variable and methods"

rule: In "Classess and Interfaces" the starting letter of every word must be UpperCase letter.

ex:

SumOfPrime

AdditionOfTwoNumber

DataInputStream

3. Variable and Methods

def: Variable are the data holder and the methods are the action.

rule: In "variable and Methods" the first word must be in LowerCase and from second word onwards the starting letter must be UpperCase.

ex: Variable:

rollNo,panCard,aadharCard....etc

Method:

nextLine(), nextInt(), getEmployee()

4. Keywords:

def: The pre-defined words are known as Keywords or Built-in words.

rule: The keywords must be in LowerCase.

==========

Eclipse IDE Install

link https://www.eclipse.org/downloads/download.php?file=/oomph/epp/2026-06/R/eclipse-inst-jre-win64.exe

==========

package in java

==========

=> packages are collection of "Classes and Interfaces".

=> package in java are categorized into two types.

1. Pre-defined package
2. User-defined package

1. Pre-defined package:

=> The package which are defined and avaible from JavaLib are known as pre-defined packages or Built-in Package.

=>The following are some important packages related to CoreJava.

java.lang - Language package(default package)

java.util - Utility package

java.io - IO Stream and Files package

java.net - Networing package

2. User-defined package

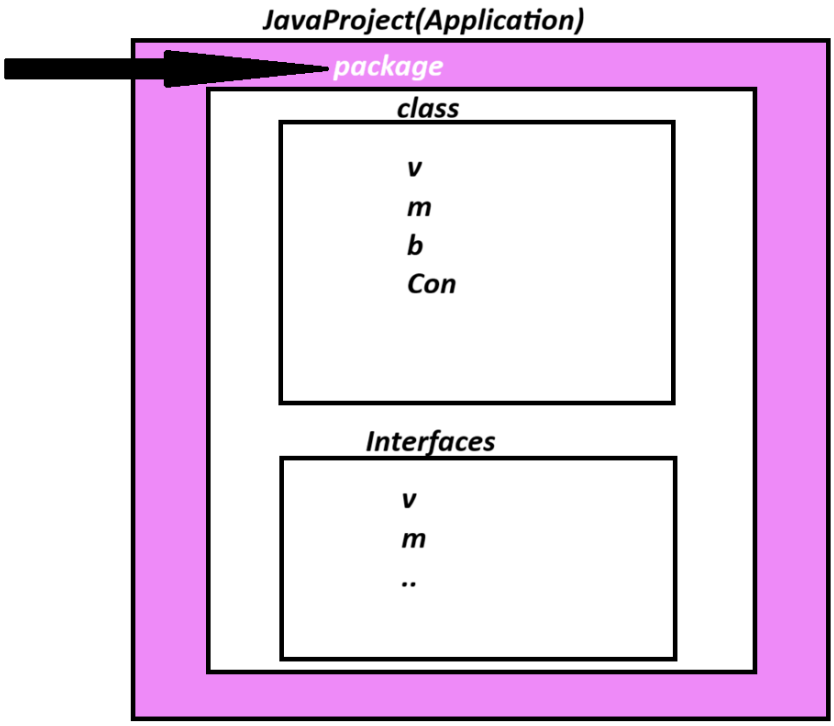
=> The package which are defined by the programmer are known as User defined package or Custom packages

=> we use "package" keyword to define user defined packages:

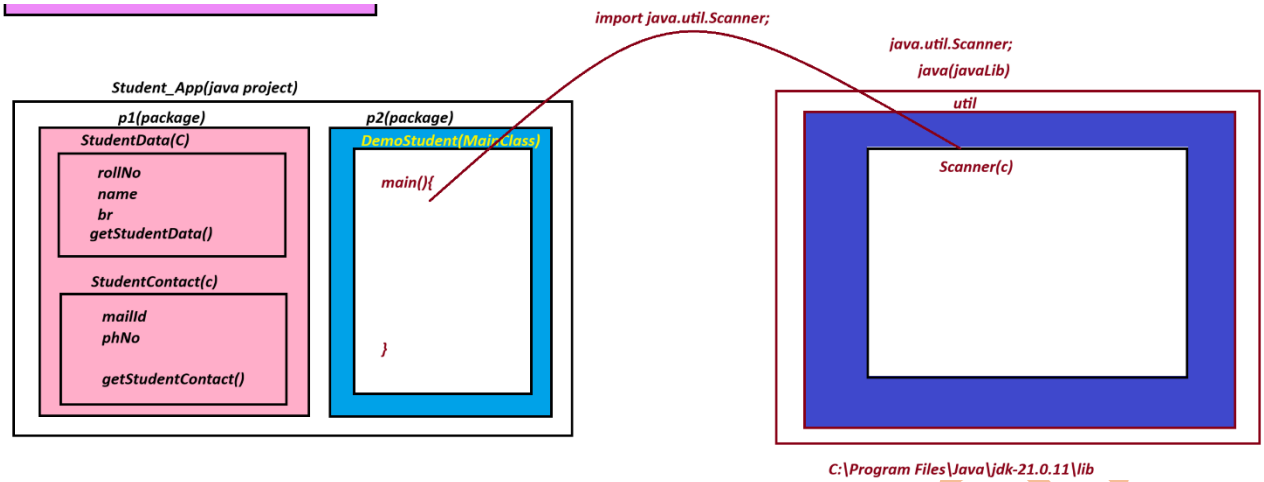
syntax:

package package_name;

diagram



Diagram



---

projectname : Student_App

package:

p1: StudentData.java

```
package p1;


public class StudentData {


    public String rollNo;
    public String name;
    public String br;


    public void getStudentData() {
        System.out.println("=======StudentData==========");
        System.out.println("RollNo : "+rollNo);
        System.out.println("Name : "+name);
        System.out.println("Branch : "+br);
    }
```

```
}
-------------
p1 StudentContact.java
package p1;

public class StudentContact {

	public String mailId;
	public  long phNo;

	public void getStudentContact() {
		System.out.println("===========StudentContact=======");
		System.out.println("MailId :"+mailId);
		System.out.println("phoneNo : "+phNo);

	}

}
--------------------
p2 DemoStudent.java (MainClass)
package p2;
import java.util.Scanner;
import p1.StudentData;
import p1.StudentContact;
public class DemoStudent {

	public static void main(String[] args) {
```

```
        Scanner s=new Scanner(System.in);
        StudentData sd=new StudentData();
        System.out.println("Enter the RollNo");
        sd.rollNo=s.nextLine();
        System.out.println("Enter the name");
        sd.name=s.nextLine();
        System.out.println("Enter the Branch");
        sd.br=s.nextLine();

        StudentContact sc=new StudentContact();
        System.out.println("Enter the mailId");
        sc.mailId=s.nextLine();
        System.out.println("Enter the PhoneNo");
        sc.phNo=s.nextLong();

        sd.getStudentData();
        sc.getStudentContact();
    }

}
```

---

output

Enter the RollNo

215578

Enter the name

Priya

Enter the Branch

CSE

Enter the mailId

priya@gmail.com

Enter the PhoneNo

9977123456

=======StudentData==========

RollNo : 215578

Name : Priya

Branch : CSE

==========StudentContact=======

MailId :priya@gmail.com

phoneNo : 9977123456

==========

session 32

==========

Access Modifiers

=> Access Modifiers will specify the scope and visibility of programming component within the project.

=> The following are some important access modifiers in java.

1. **public**
2. **private**
3. **protected**
4. **default**

1. public :

=> public is an access modifier that makes a class, method, variable, or constructor accessible from anywhere in the application.

2. private :

=> private programming components are accessed only within the class.

3. protected:

=> protected programming components are accessed within the same package.

Note:

=> protected programming components can also be accessed by the ChildClass outside the package.(Inheritance process)

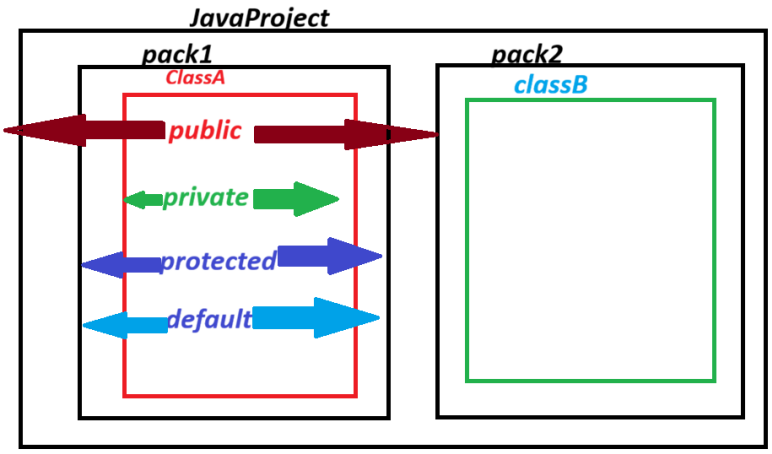
4. default:

=> The programming components whicha are declared without any access modifiers are considered as 'default'.

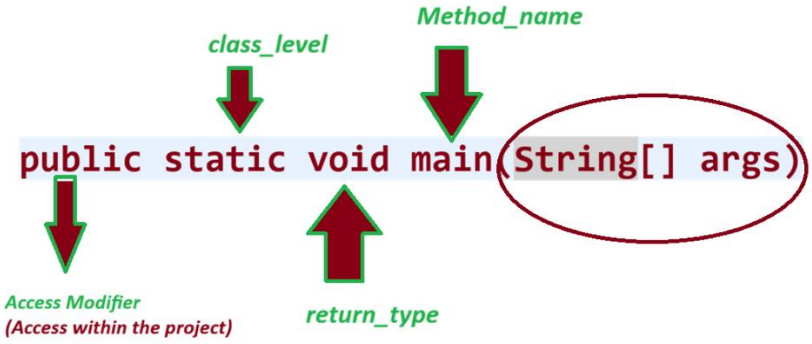
=> default programming components are accessed only inside the packages.

Note:

=> we cannot use 'default' keyword in classess. (Compiletion Error)



Access Modifier	Same Class	same package	Subclass(Other package)	other Package
private	yes	No	No	No
default	yes	yes	No	No
protected	yes	yes	yes	No
public	yes	yes	yes	yes



==========

define import statement?

=> "import" statement will make class or interface available from one package to another package.

=> This importing process can be done in three ways.

(i) using 'import pakcage_name.ClassName/InterfaceName';

(ii) Using 'import package_name.*'

(iii) Using Fully 'Qualified Names'

(i) using 'import pakcage_name.ClassName/InterfaceName';

=> In this importing process the required class or interface from the package are available to current running program.

=> This importing process is also known as 'Explicit importing process'

Ex:

```
import java.util.Scanner;
```

```
import p1.StudentData;
import p1.StudentContact;
----------
```

ii) Using 'import package_name.*'

=> In this importing process all the classess and interfaces from the package are available to current running program.

=> This importing process is also known as "implicit importing process"

Ex :

```
import java.util.*;
import p1.*;
```

(iii) Using Fully 'Qualified Names'

=> The process of declaring classess and interfaces with package_name part of programming code are known as "fully Qualified Name".

ex:

```
java.util.Scanner s=new java.util.Scanner(System.in);
        p1.StudentData sd=new p1.StudentData();
        p1.StudentContact sc=new p1.StudentContact();
```

==========

```
import java.util.Scanner;
class Add
{
    int add(int x,int y){
```

```
            return x+y;
        }
}
class Sub
{
        int sub(int x,int y){
            return x-y;
        }
}
class Mul
{
        int mul(int x,int y){
            return x*y;
        }
}
class Div
{
        float div(int x,int y){
            return (float)x/y;
        }
}
class ModDiv
{
        int modDiv(int x,int y){
            return x%y;
        }
}
```

```
class CalculatorMain
{
        public static void main(String[] args)
        {
                Scanner s=new Scanner(System.in);
                System.out.println("Enter the int value -1:");
           int v1=s.nextInt();
                System.out.println("Enter the inv value -2:");
                int v2=s.nextInt();

                if(v1>0 && v2>0){
                        System.out.println("==========CHOICE=======");
                        System.out.println("1.add\n2.sub\n3.mul\n4.div\n5.modDiv");
                        System.out.println("Enter the choice:");
                        int choice=s.nextInt();

                        switch(choice){
                                case 1:
                                        Add ad=new Add();
                                  int r1=ad.add(v1,v2);
                                  System.out.println("Addition "+r1);
                                        break;
                                case 2:
                                        Sub sb=new Sub();
                                        int r2=sb.sub(v1,v2);
                                        System.out.println("Subtraction "+r2);
```

```
                    break;
                case 3:
                    Mul ml=new Mul();
                  int r3=ml.mul(v1,v2);
                    System.out.println("Multiplication "+r3);
                    break;
                case 4:
                    Div dv=new Div();
                  float r4=dv.div(v1,v2);
                    break;
                case 5:
                    ModDiv md=new ModDiv();
                  int r5=md.modDiv(v1,v2);
                    System.out.println("Mod Division "+r5);
                    break;
                default:
                    System.out.println("Invalid choice...");
                }

            }// end of if
            else{
                System.out.println("Invalid values...");
            }
        }
}
```

Assignemnt 1

Convert CalculatorMain programm into package-concept

project_name : Calculator_App

packages,

p1 : Add.java

p1 : Sub.java

p1 : Mul.java

p1: Div.java

p1: ModDiv.java

p2: CalculatorApp.java

==========

Assignment 2:

==========

convert Atm program into package concept

project_name : Bank_App

packages

p1. WithDraw.java

p1. Deposite.java

p2. Bank_App.java(mainClass)

==========

session 33

==========

Assignemnt 1(Solution)

Convert CalculatorMain programm into package-concept

project_name : Calculator_App

packages,

p1 : Add.java

```
package p1;

public class Add {

  public int add(int x, int y) {
                return x+y;
        }
}
```

---

p1 : Sub.java

```
package p1;

public class Sub {

        public int sub(int x,int y) {
        return x-y;
        }
}
```

---

p1 : Mul.java

```
package p1;

public class Mul {
```

```
	public int mul(int x,int y) {
		return x*y;
	}
}
```

---

p1: Div.java

```
package p1;

public class Div {

	public float div(int x,int y) {
		return (float)x/y;
	}
}
```

---

p1: ModDiv.java

```
package p1;

public class ModDiv {

	public int modDiv(int x,int y){
		return x%y;
	}
}
```

---

p2: CalculatorApp.java

```
package p2;
import java.util.Scanner;
import p1.Add;
import p1.Sub;
import p1.Mul;
import p1.Div;
import p1.ModDiv;

public class CalculatorApp {

	public static void main(String[] args) {
		Scanner s=new Scanner(System.in);
		System.out.println("Enter the int value -1:");
	   int v1=s.nextInt();
		System.out.println("Enter the inv value -2:");
		int v2=s.nextInt();

		if(v1>0 && v2>0){
			System.out.println("==========CHOICE=======");
			System.out.println("1.add\n2.sub\n3.mul\n4.div\n5.modDiv");
			System.out.println("Enter the choice:");
			int choice=s.nextInt();

			switch(choice){
				case 1:
					Add ad=new Add();
				  int r1=ad.add(v1,v2);
```

```
            System.out.println("Addition "+r1);
                break;
        case 2:
                Sub sb=new Sub();
                int r2=sb.sub(v1,v2);
                System.out.println("Subtraction "+r2);
                break;
        case 3:
                Mul ml=new Mul();
           int r3=ml.mul(v1,v2);
                System.out.println("Multiplication "+r3);
                break;
        case 4:
                Div dv=new Div();
           float r4=dv.div(v1,v2);
                break;
        case 5:
                ModDiv md=new ModDiv();
            int r5=md.modDiv(v1,v2);
                System.out.println("Mod Division "+r5);
                break;
        default:
                System.out.println("Invalid choice...");
    }
```

```
                }// end of if
                else{
                        System.out.println("Invalid values...");
                }

        }

}
```

---

output

```
Enter the int value -1:
20
Enter the inv value -2:
30
==========CHOICE=======
1.add
2.sub
3.mul
4.div
5.modDiv
Enter the choice:
3
Multiplication 600
```

==========

define Static import?

=>The process of declaring import statement with "static" keyword in known as "static import" and which is introduced by java5 version(2004).

syntax:

import static package_name.Class_name/Interface_name.*;

=> In static import all the static memebers of class/interface available directly and can be accessed without Class_Name or Interface_name.

=> In static-import the class/interface not available to current running program.

========

project name : - Static_Import_App

p1 : STest.java

```
package p1;


public class STest {


	public  int a;
	public static int b;


	public static void dis1() {
		System.out.println("=========static dis1()==========");
		System.out.println("The value of b : "+b);
	}


	public void dis2() {
		System.out.println("==========Instance dis2()=========");
		System.out.println("The value of a : "+ a);
		System.out.println("The value of b : "+b);
	}
```

```
}
----------
p2 : DeoStaticImport.java(MainClass)
package p2;
import java.util.Scanner;
import static p1.STest.*;
import p1.STest;
public class DemoStaticImport {


        public static void main(String[] args) {
        Scanner s=new Scanner(System.in);
        STest ob=new STest();

        System.out.println("Enter the vaue of a: ");
        ob.a = s.nextInt();
        System.out.println("Enter the value of b:");
        //STest.b=s.nextInt();
        b=s.nextInt();

        //STest.dis1();
        dis1();
        ob.dis2();
        }
```

}

----

ouput

Enter the vaue of a:

12

Enter the value of b:

13

=========static dis1()==========

The value of b : 13

==========Instance dis2()=========

The value of a : 12

The value of b : 13

==========

Relations in java

==========

=> The process of establishing communication between components classes or Object is known as "relations"

=> Relations in java are categorized into three types:

1. **References**
2. **Inheritance**
3. **InnerClasses**

1. References:

=> The process of interlinking objects is known as references concept.

=> References concept means one object holding the reference of another object.

=> References concept is categorized into two types:

(a) Tightly Couple references

(b) Loosly Couple references

(a) Tightly Couple references:

=>The references concept, in which the reffered object is created while creating outer-object is known as Tightly coupled references.

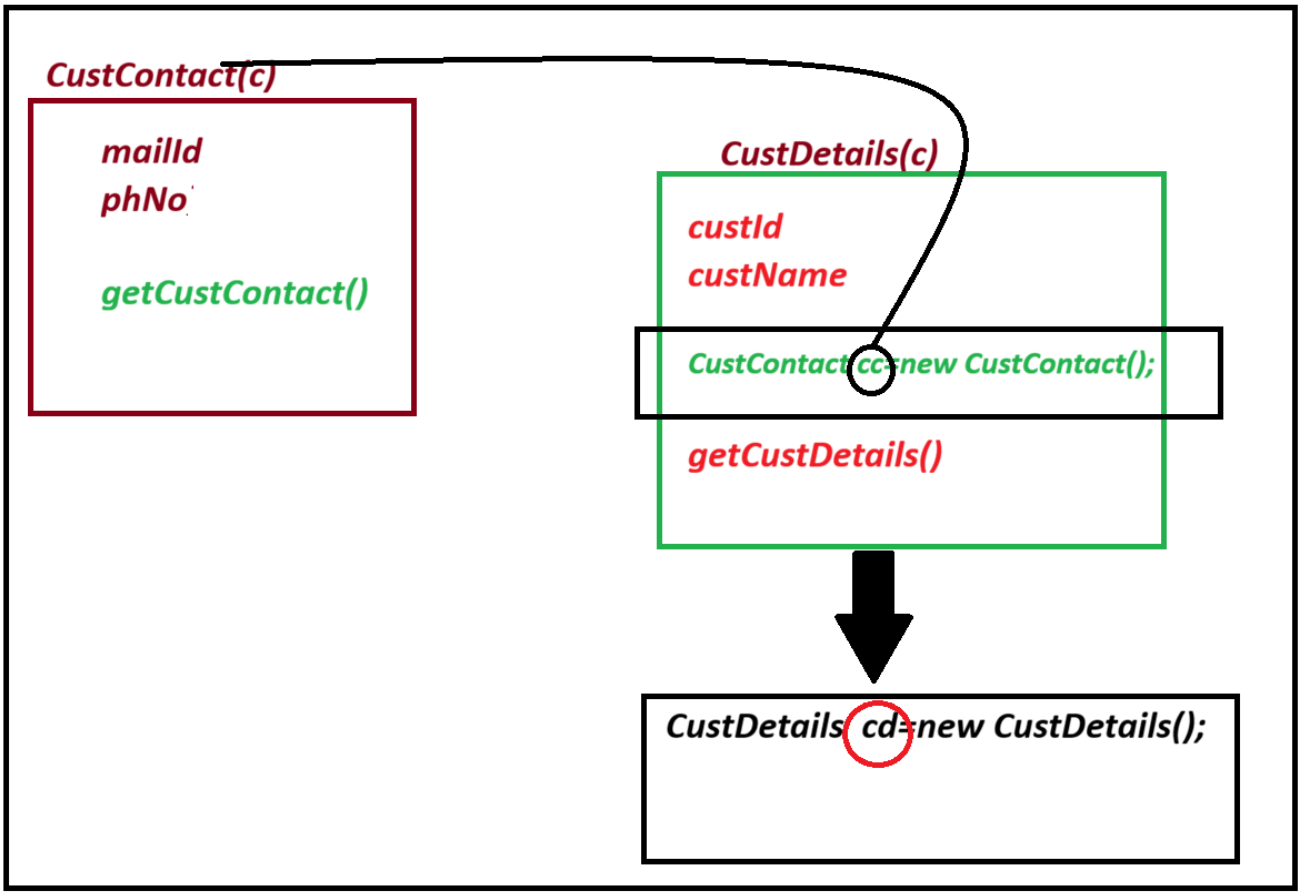
Oute-Object : CustDetails object

reffered object : CustContact object

(reffered object is also known Interlinked Object)

=> In Tightly coupled references the reffered object is available only one Outer-object.

Layout



diagram

Project : References_App1

p1: CustContact.java

```
package p1;

public class CustContact {

	public String mailId;
	public long phNo;

	public void getCustContact() {
		System.out.println("========CustContact=======");
		System.out.println("MailId : "+mailId);
		System.out.println("PhoneNo : "+phNo);
	}

}
```

---

p1: CustDetails.java

```
package p1;

public class CustDetails {

	public String custId;
	public String custName;
```

```
	public CustContact cc=new CustContact();

	public void getCustDetails() {
		System.out.println("======CustDetails=======");
		System.out.println("CustId : "+custId);
		System.out.println("custName :"+custName);
	}

}
```

---

p2: DemoRef1.java

```
package p2;
import java.util.Scanner;
import p1.CustDetails;
public class DemoRef1 {

	public static void main(String[] args) {
		Scanner s=new Scanner(System.in);
		CustDetails cd=new CustDetails();
	System.out.println("Enter custId :");
	cd.custId=s.nextLine();
	System.out.println("Enter custName :");
	cd.custName=s.nextLine();
	System.out.println("Enter the custMailId");
	cd.cc.mailId=s.nextLine();
	System.out.println("Enter the PhoneNo :");
	cd.cc.phNo=s.nextLong();
```

```
        cd.getCustDetails();
        cd.cc.getCustContact();
       s.close();

          }

}
=========
 ----------
 output
 ----------
 Enter custId :
BOB995578645
Enter custName :
RANI KUMARI
Enter the custMailId
ranikumari@gmail.com
Enter the PhoneNo :
9988771234
======CustDetails=======
CustId : BOB995578645
custName :RANI KUMARI
========CustContact=======
MailId : ranikumari@gmail.com
PhoneNo : 9988771234
==================================
```

session 34

==========

(b) Loosly Couple references:

=> The references Concept in which referred Object is not created while Outer-Object creation is known as Loosly Couple references.

=>In Loosly Coupled referencs the refeered objects are available to more than one Out objects.

=> we use the following two processes to perform Loosely Coupled refernces.

(i) Constructor Injection process

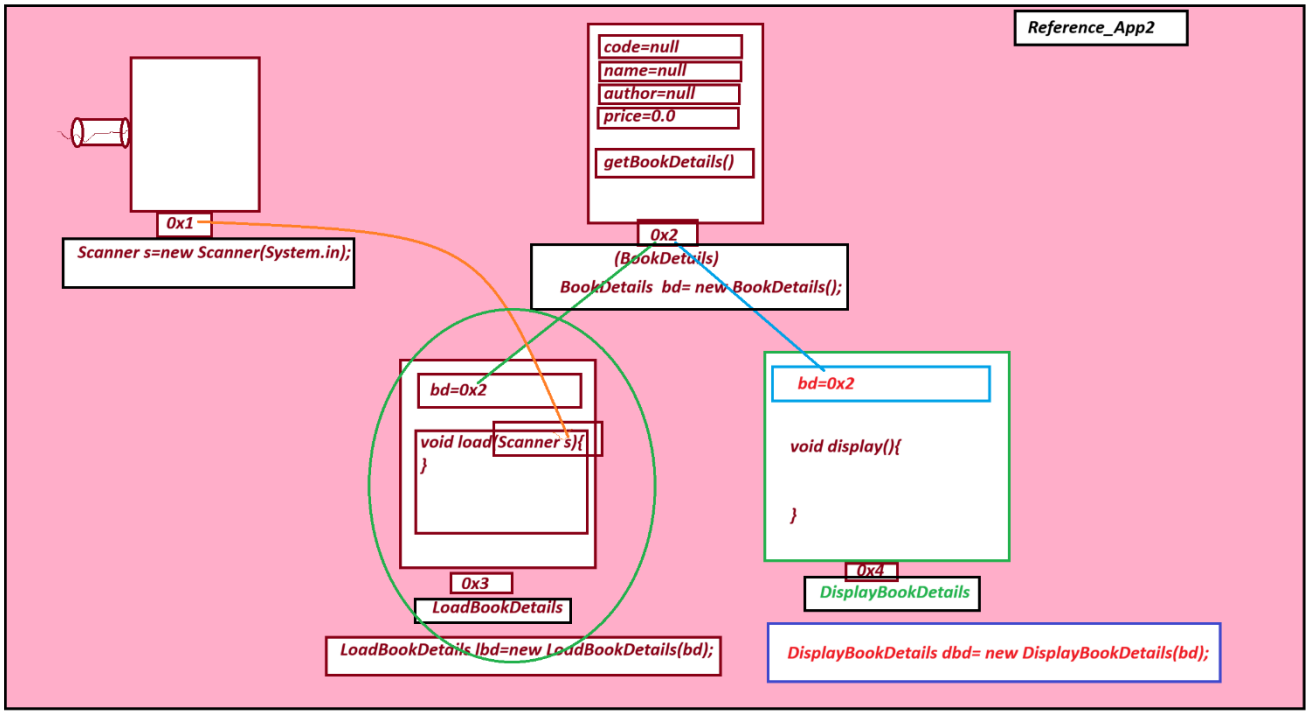
(ii) Setter method Injection process

(i) Constructor Injection process:

=> In Constructor Injection process the reference is binded to Object using Constructor.

==========

Diagram



Example project Reference_App2

packages

p1:

BookDetails.java

```
package p1;

public class BookDetails {

        public String code;
        public String name;
        public String author;
        public float price;


        public void getBookDetails(){
                System.out.println("=========BookDetails=======");
                System.out.println("BookCode: "+code);  //ctrl+alt+down Arrow
                System.out.println("BookName: "+name);
                System.out.println("BookAuthor: "+author);
                System.out.println("BookPrice : "+price);
        }

}
```

---

p1: LoadBookDetails.java

```
package p1;
import java.util.Scanner;
public class LoadBookDetails {



	public BookDetails bd=null;


	public LoadBookDetails(BookDetails bd){
		this.bd=bd;
	}


	public void load(Scanner s){
		System.out.println("Enter the BookCode");
		bd.code=s.nextLine();
		System.out.println("Enter the BookName");
		bd.name= s.nextLine();
		System.out.println("Enter the BookAuthor");
		bd.author=s.nextLine();
		System.out.println("Enter the BookPrice");
		bd.price=s.nextFloat();


	}
}
```

---

p1: DisplayBookDetails.java

```
package p1;
```

```
public class DisplayBookDetails {

	public BookDetails bd=null;

	public DisplayBookDetails(BookDetails bd) {
		this.bd=bd;

	}

	public void display(){
		bd.getBookDetails();
	}
}
```

---

p2:

DemoRef2.java

```
package p2;

import java.util.Scanner;

import p1.BookDetails;
import p1.DisplayBookDetails;
import p1.LoadBookDetails;

public class DemoRef2 {

	@SuppressWarnings("resource")
```

```
        public static void main(String[] args) {
                Scanner s = new Scanner(System.in);
                BookDetails bd=new BookDetails();//Con_call

                LoadBookDetails lbd=new LoadBookDetails(bd);// Con_Call
                 DisplayBookDetails dbd=new DisplayBookDetails(bd);//Con_Call

                 lbd.load(s);
                 dbd.display();
        }

}
------------------------------
===============
output
-----------
Enter the BookCode
JV3526
Enter the BookName
JAVA
Enter the BookAuthor
B-SWAMY
Enter the BookPrice
400.0
=========BookDetails=======
BookCode: JV3526
BookName: JAVA
```

BookAuthor: B-SWAMY

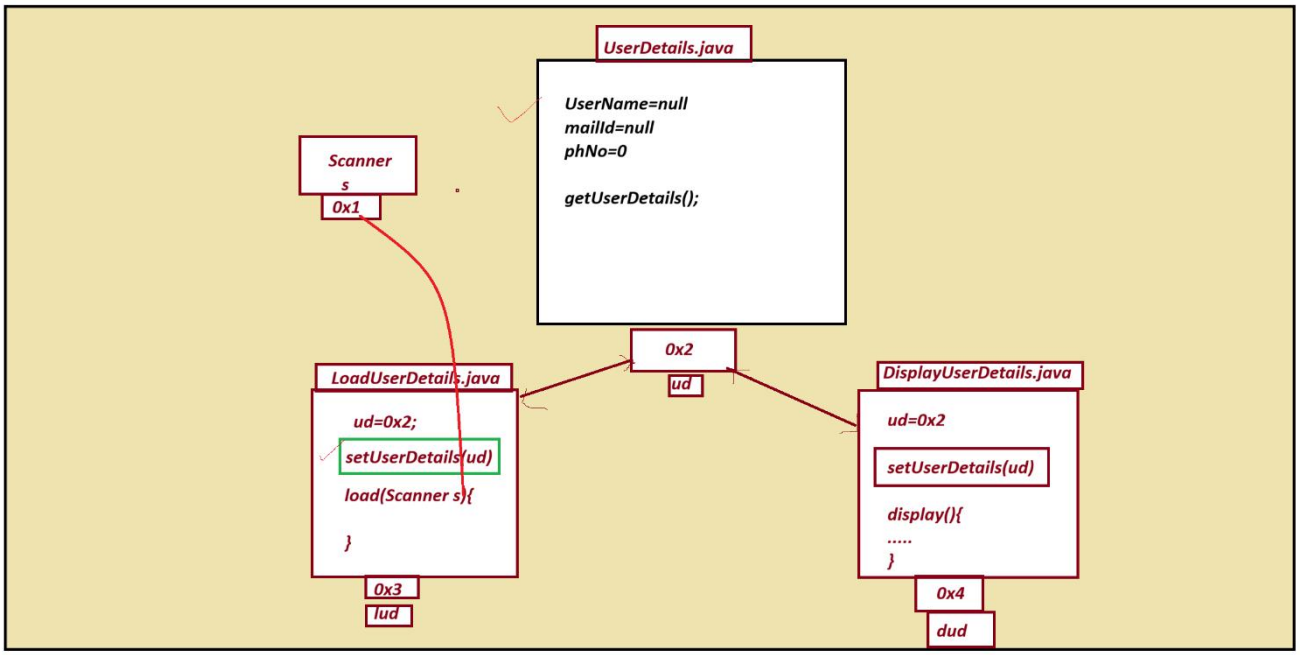
BookPrice : 400.0

==========

(ii) Setter method Injection process:

=> In setter method injection process the reference is binded to Object using Setter method.

diagram



Example

project Name : Reference_App3

p1: UserDetails.java

package p1;

public class UserDetails {

public String userName;

```
	public String mailId;
	public long phNo;

	public void getUserDetails() {
		System.out.println("========UserDetails========");
		System.out.println("UserName : "+userName);
		System.out.println("MailId :"+mailId);
		System.out.println("phoneNo : "+phNo);
	}

}
```

---

p1: LoadUserDetails.java

```
package p1;

import java.util.Scanner;

public class LoadUserDetails {


	public UserDetails ud=null;

  public void setUserDetails(UserDetails ud){
		this.ud=ud;
	}

  public void load(Scanner s) {
```

```
            System.out.println("Enter the UserName ");
            ud.userName=s.nextLine();
            System.out.println("Enter the MailId");
            ud.mailId=s.nextLine();
            System.out.println("Enter the phoneNo");
            ud.phNo=s.nextLong();
    }
}
```

---

p1: DisplayUserDetails.java

```
package p1;

public class DisplayUserDetails {



        public UserDetails ud=null;

        public void setUserDetails(UserDetails ud) {
                this.ud=ud;
        }

        public void display() {
                ud.getUserDetails();
        }
}
```

==========

p2 DemoRef3.java

```
package p2;

import java.util.Scanner;

import p1.DisplayUserDetails;
import p1.LoadUserDetails;
import p1.UserDetails;

public class DemoRef3 {

	@SuppressWarnings("resource")
	public static void main(String[] args) {
		Scanner s = new Scanner(System.in);
		UserDetails ud = new UserDetails();
		LoadUserDetails lud = new LoadUserDetails();
		DisplayUserDetails dud = new DisplayUserDetails();

		lud.setUserDetails(ud);
		dud.setUserDetails(ud);
		lud.load(s);
		dud.display();
		s.close();

	}

}
```

==========

output

Enter the UserName

kumar@123

Enter the MailId

kumar123@gmail.com

Enter the phoneNo

8844337712

========UserDetails========

UserName : kumar@123

MailId :kumar123@gmail.com

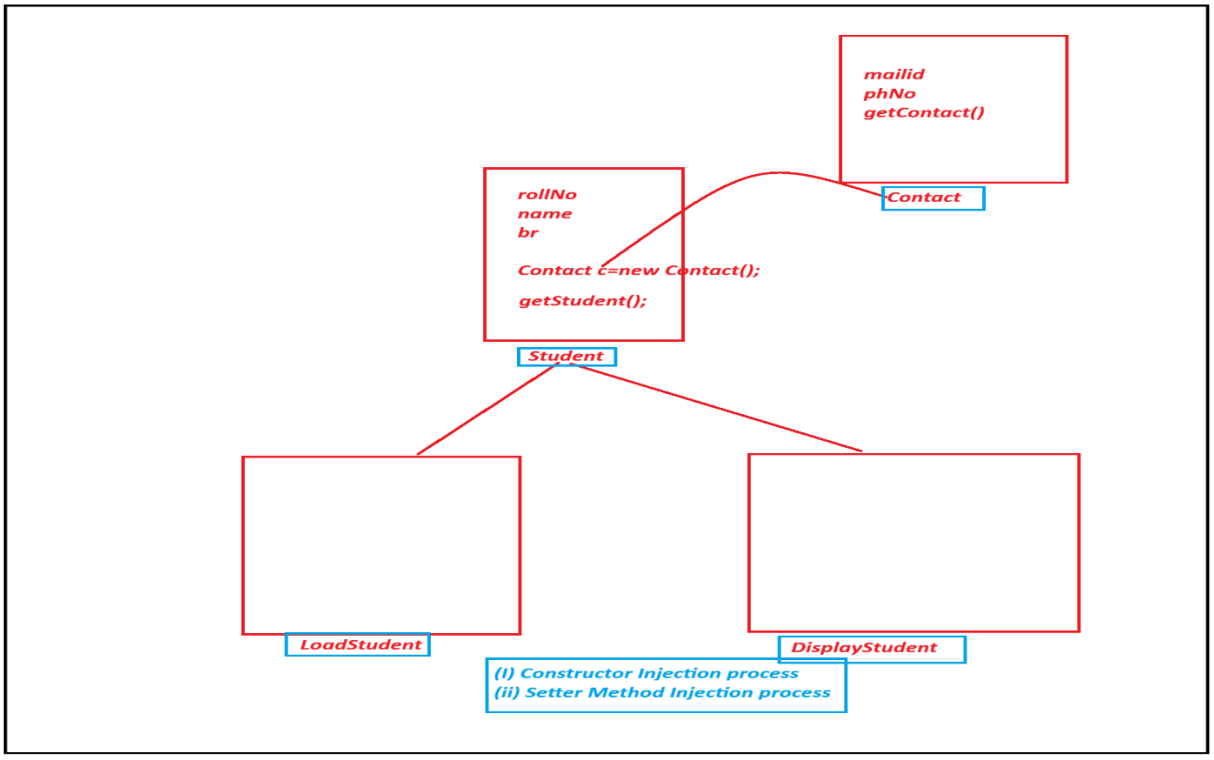
phoneNo : 8844337712

========================

Assignment:

Construct the Application using the following layout:

Diagram



=======

session 35

Assignment Solution)

Construct the Application using the following layout:

p1: Contact.java

```
package p1;

public class Contact {

	public String mailId;
	public long phNo;

	public void getContact() {
		System.out.println("============StudentContact==========");
		System.out.println("MailId : "+mailId);
		System.out.println("PhoneNo : "+phNo);
	}

}
```

==========

p1: Student.java

```
package p1;

public class Student {
```

```
	public String rollNo;
	public String name;
	public String br;

	public Contact ct=new Contact();

	public void getStudent() {
		System.out.println("========StudentDetails==========");
		System.out.println("Student RollNO : "+rollNo);
		System.out.println("Student Name : "+ name);
		System.out.println("Student branch : "+br);
	}

}
```

==========

p1: LoadStudent.java

```
package p1;

import java.util.Scanner;

public class LoadStudent {

	public Student stu=null;

	public LoadStudent(Student stu) {
		this.stu=stu;
	}
```

```
	public void load(Scanner s) {
		System.out.println("Enter the student RollNo :");
		stu.rollNo=s.nextLine();
		System.out.println("Enter the Student Name : ");
		stu.name=s.nextLine();
		System.out.println("Enter the student branch");
		stu.br=s.nextLine();
		System.out.println("Enter the student mailId:");
		stu.ct.mailId=s.nextLine();
		System.out.println("Enter the student PhoneNo ");
		stu.ct.phNo=s.nextLong();

	}

}
```

P1: DisplayStudent.java

```
package p1;

public class DisplayStudent {

	public Student stu=null;

	public DisplayStudent(Student stu) {
		this.stu=stu;
```

```
	}

	public void display() {
		stu.getStudent();
		stu.ct.getContact();
	}
}
```

==========

p2: DemoRef4(Main Class)

```
package p2;

import java.util.Scanner;

import p1.DisplayStudent;
import p1.LoadStudent;
import p1.Student;

public class DemoRef4 {

	@SuppressWarnings("resource")
	public static void main(String[] args) {
		Scanner s = new Scanner(System.in);
		Student stu = new Student();
		LoadStudent ls = new LoadStudent(stu);
		DisplayStudent ds = new DisplayStudent(stu);

		ls.load(s);
```

```
                ds.display();

                s.close();

        }

}
```

---

output

```
========
Enter the student RollNo :
CSE15236
Enter the Student Name :
Rahul Kumar
Enter the student branch
CSE
Enter the student mailId:
rahul@gmail.com
Enter the student PhoneNo
9977258369
========StudentDetails==========
Student RollNO : CSE15236
Student Name : Rahul Kumar
Student branch : CSE
============StudentContact===========
MailId : rahul@gmail.com
```

PhoneNo : 9977258369

==========

session 36

==========

Note:

=> This references concept is also known HAS-A relation, which means one Object HAS-A reference of another Object.

Advantages of HAS-A relation(Reference Concept):

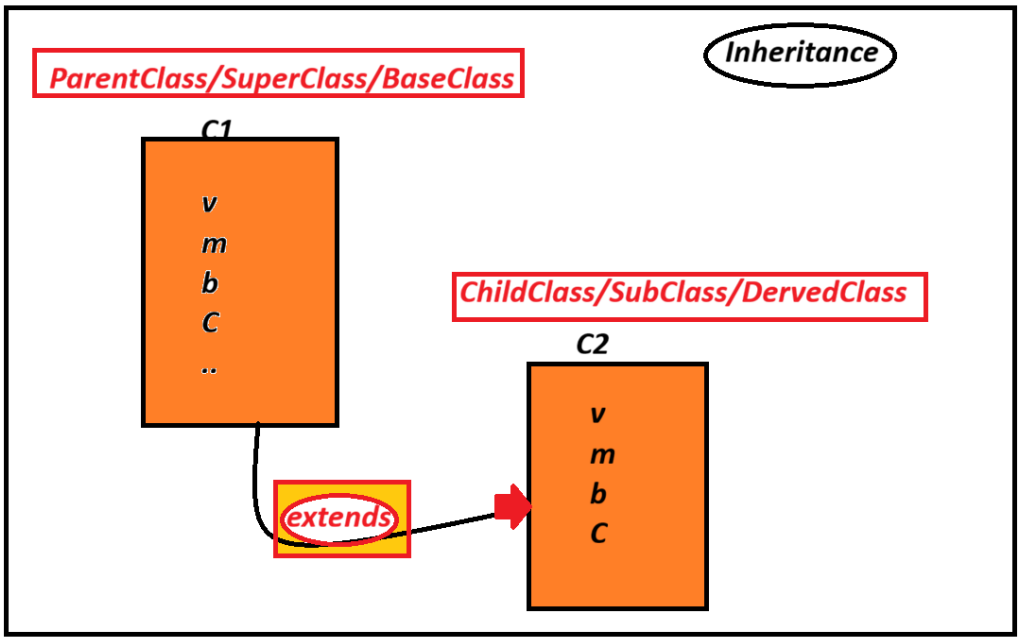
=> Through One object we can access the members of another Object, which means members of one Object can access the members of another Objects.

==========

2. Inheritance in Java:

=> The process of Interlinking two classess with "extends" keyword is known as Inheritance process.

Diagram



---

syntax:

class C1{

```
        //ParentClass_body
    }

    class C2 extends C1{
        // Childclass_boyd
    }
```

=> In Inheritance process we always create object for ChildClass, because the ChildClass object will hold all the members of ParentClass and ChildClass.

syntax:

```
C2 ob=ne C2();
```

==========

define inheritance:

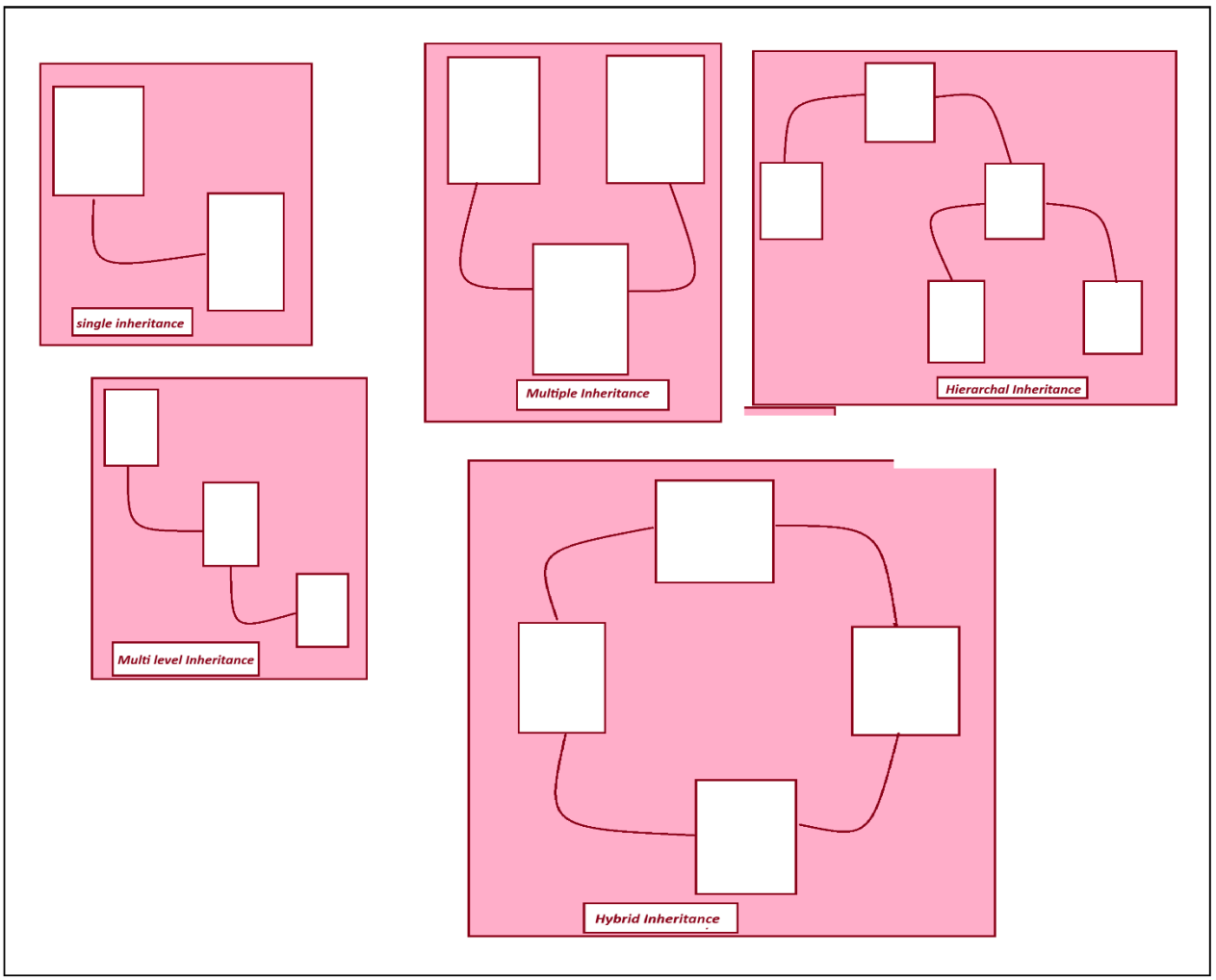
=> The process of extracting features from one class into another class is known as Inheritance process.

Types of Inheritance

=> The following are the types of Inheritances:

1. **Single Inheritance**
2. **Multiple Inheritance**
3. **Multi - Level Inheritance**
4. **Hierarichal Inheritance**
5. **Hybrid Inheritance**

Diagram



---

define Single Inheritance:

=>The process of extracting the features from one class at-a-time is known as Single Inheritance process.

Case 1: Variables and Methods from the ParentClass/SuperClass/BaseClass.

=> In Inheritance process all the Variable and methods of ParentClass are available to childClass.

=> Instance variable and methods of ParentClass are accessed using ChildClass object_name.

=> static variables and methods of ParentClass are accessed using ChildClass_Name.

---

project name :Inheritance_App1

package

p1:

C1.java

```
package p1;

public class C1 {

        public int a;
        public static int b;

 public void m1(){
                System.out.println("==========PClass Instance-m1()==========");
                System.out.println("The value a :"+a);
                System.out.println("The value b : "+b);

 }

  public static void m2(){
        System.out.println("========PClass Static-m2()==========");
        //System.out.println("The value a :"+a);
        System.out.println("The value b: "+b);
    }

}
```

---

p1: C2.java

```
package p1;

public class C2 extends C1{

	public int x;
	public static int y;

	public void m3() {
		System.out.println("=======CClass Instance-m3()========");
		System.out.println("The value of x "+x);
		System.out.println("The value of y "+y);
	}

	public static void m4() {
		System.out.println("=======CClass Instance-m4()========");
		//System.out.println("The value of x "+x);
		System.out.println("The value of y "+y);
	}


}
```

---

p2: DemoInheritance.java

```
package p2;
```

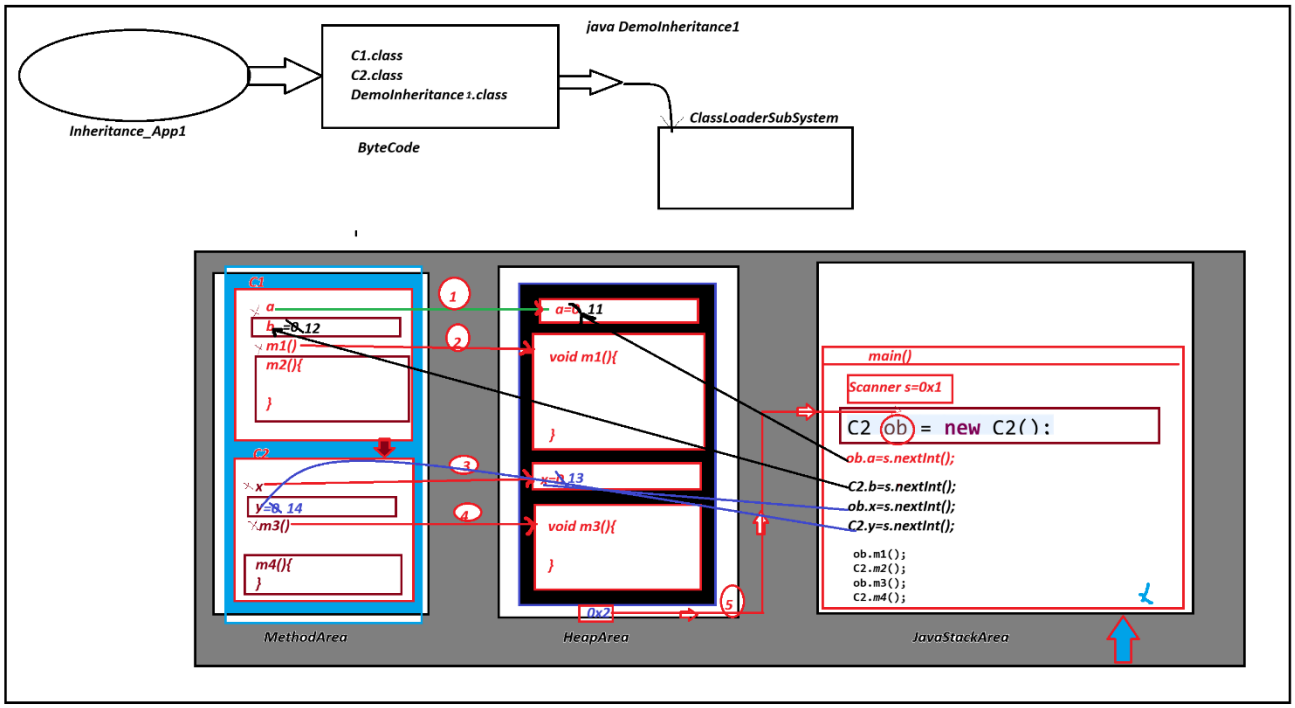
```
import java.util.Scanner;

import p1.C2;

public class DemoInheritance1 {


	@SuppressWarnings("resource")
	public static void main(String[] args) {
		Scanner s = new Scanner(System.in);
		C2 ob = new C2();// child Class Object
		System.out.println("Enter the value for a :");
		ob.a=s.nextInt();
		System.out.println("Enter the value for b :");
		C2.b=s.nextInt();
		System.out.println("Enter the value for x :");
		ob.x=s.nextInt();
		System.out.println("Enter the value for y :");
		C2.y=s.nextInt();

		ob.m1();
		C2.m2();
		ob.m3();
		C2.m4();
		s.close();
```

```
        }
}
```

```
=======
output
Enter the value for a :
11
Enter the value for b :
12
Enter the value for x :
13
Enter the value for y :
14
==========PClass Instance-m1()==========
The value a :11
The value b : 12
========PClass Static-m2()==========
The value b: 12
=======CClass Instance-m3()========
The value of x 13
The value of y 14
=======CClass Instance-m4()========
The value of y 14
============================
```

Execution flow of above application

diagram



=======

session 37

=======

Note:

=> In Inheritance process PClass is loaded on to Method Area of JVM first and then CClass is loaded.

=> In Inheritance process, while object creation PClass Instance memebers will get the memory first and then CClass Instance members will get the memory.

==========

define Method Overriding process?

=> The method with same method signature in PClass and CClass, then PClass method is replaced by CClass method while object creation process is known as Method Overriding process or Method Replacement Process.

=> Same method signature means,

same return_type

same method_name

same para_list

same para_type

(i) Instance method Overriding process:

=> we can perform Instance Method Overriding process.

ProjectName : Inheritance_App2

package p1

C1.java

```
package p1;

public class C1 {

    public double calculate(double x)//Overrided_Method
    {
        System.out.println("====PClass calculate(x) ==========");
        return Math.sqrt(x);
    }

}
```

---

package p1 C2.java

```
package p1;

public class C2 extends C1{
```

```
	public double calculate(double x) //Overriding method
	{
		System.out.println("======CClass calculate(x)====");
		return Math.pow(x, 3);

	}

}
```

---

p2 DemoInheritanceApp2.java

```
package p2;

import java.util.Scanner;

import p1.C2;

public class DemoInheritance2 {

	@SuppressWarnings("resource")
	public static void main(String[] args) {
		Scanner s = new Scanner(System.in);
		C2 ob = new C2();

		System.out.println("Enter the value : ");
		double val = s.nextDouble();

		double res = ob.calculate(val);
```

```
            System.out.println("Result : "+res);

            s.close();

        }

}
```

---

output

Enter the value :

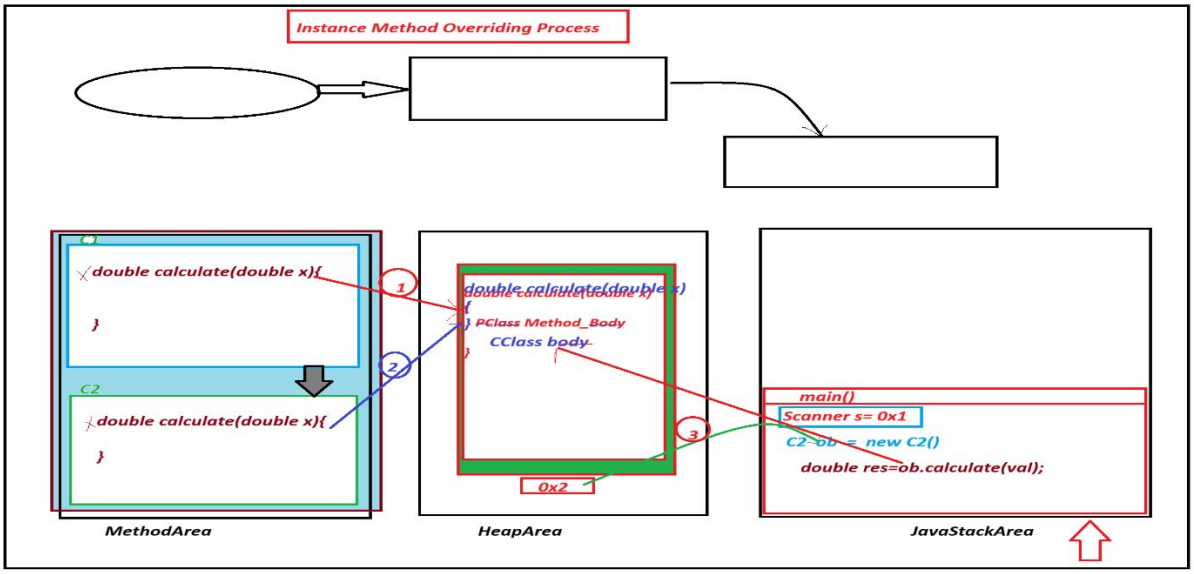
2

======CClass calculate(x)====

Result : 8.0

==========

Execution flow of above program.



==========

(ii) static method Overriding process:

=> There is no concept of static method Overrding process, because static methods will get the memory within the class and available within the class.

projectName : Inheritance_App3

p1 C1.java

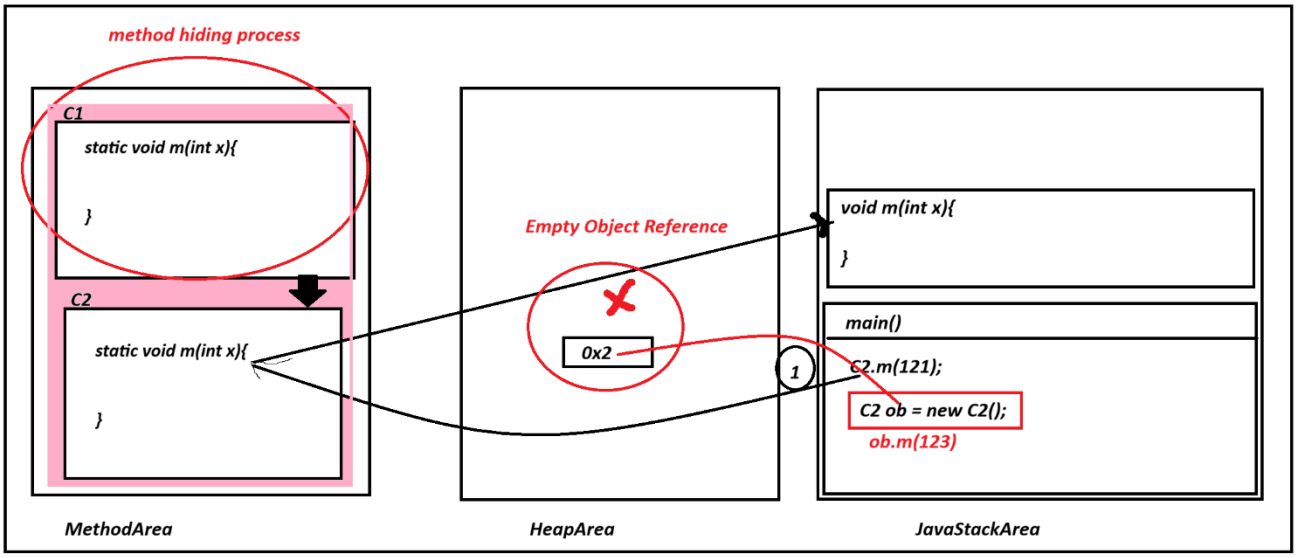
```
package p1;


public class C1 {


	public static void m(int x) {
		System.out.println("======PClass m(x)=======");
		System.out.println("The value of x :"+x);
	}


}
```

---

p1 C2.java

```
package p1;


public class C2 extends C1{


	public static void m(int x) {
		System.out.println("====CClass m(x)===========");
		System.out.println("The value of x :"+x);
	}
```

```
}
---------
p2. DemoInheritance3.java(mainClass)
package p2;

import p1.C2;

public class DemoInheritance3 {
	public static void main(String[] args) {
		System.out.println("=====Using  Class Name=======");
		C2.m(121);

		System.out.println("========Using Object Name ===========");
		C2 ob = new C2();
		ob.m(123);
	}

}
-------------------------
output
=====Using  Class Name=======
====CClass m(x)===========
The value of x :121
========Using Object Name ===========
====CClass m(x)===========
The value of x :123
```



==========

Method Hiding process:

=>When we have same static method signature in PClass and CClass , then PClass method is hided by CClass method while execution process is known as method Hiding process.

Note:

=> when Execution control cannot reach PClass method then PClass method is hided method.

Empty Object reference

=> when we create object for a class holding olnly static memebers, then 'Empty Object reference' is created.

(Question) : Can we acess static member using Object Neme?

Yes, we can acess static member of Class using Object Name, because the Object belongs to Class, which means genereted from the class.

==========

session 38

==========

Summary:

(i) Same Instance method signature in PClass and CClass is known as Method Overriding process.

(ii) Same Static method signature in PClass and CClass is known as Method Hiding process.

==========

Case 2: Constructor from the ParentClass/SuperClass

(i) 0- Parameterized Constructor from the PClass/SuperClass

=> when we have 0-parameter Constructor in the PClass, then Compiler at compilation stage will add "super()" to the Class Constructor and which is PClass Con_Call.

ProjectName : Inheritance_App4

p1: C1.java

```
package p1;


public class C1 {


	public C1() {
		System.out.println("====PClass Constructor=====");
	}

}
```

P1: C2.java

```
package p1;


public class C2 extends C1 {
```

```
        public C2(){
                super();// added by the compiler at compilation stage
                System.out.println("========CClass Constructor=========");

        }

}
```

---

p2: DemoInheritance4.java

```
package p2;
import p1.C2;
public class DemoInheritance4 {
   public static void main(String[] args) {
                C2 ob=new C2(); //CClass Constructor Call
        }
}
```

======

output

```
====PClass Constructor=====
=====CClass Constructor====
```

(ii)Parameterized Constructor from the PClass/SuperClass

=> When we have parameterized Constructor from the PClass, then we must add "super()" to the CClass constructor to call PClass Constructor.

=> we pass parameter to PClass Constructor through CClass Constructor.

ProjectName: Inheritance_App5

p1: C1.java

```
package p1;

public class C1 {

        public C1(int x){
                System.out.println("======PClass Constructor=======");
                System.out.println("The value of x : "+x);
        }
}
```

---

p1: C2.java

```
package p1;

public class C2 extends C1 {

        public C2(int v) {
                super(v); //PClass_Con_Call
                System.out.println("--CClass Constructor--");

        }

}
```

---

p2: DemoInheritance5.java

```
package p2;

import p1.C2;

public class DemoInheritance5 {

	public static void main(String[] args) {
		C2 ob=new C2(123); //CClass_Con_Call
	}

}
```

output

```
-------
======PClass Constructor=======
The value of x : 123
--CClass Constructor--
----------------------------
```

Case 3: Block from ParentClass/SuperClass

projectName : Inheritance_App6

p1: C1.java

```
package p1;
```

```
public class C1 {

	public static int a;

	static {
		System.out.println("=====PClass static Block====== ");
		System.out.println("The value a :"+a);
	}

}
```

---

p1: C2.java

```
package p1;

public class C2 extends C1{

	static {
		System.out.println("====CClass static Block=========");
		System.out.println("The value a :"+a);
	}

}
```

---

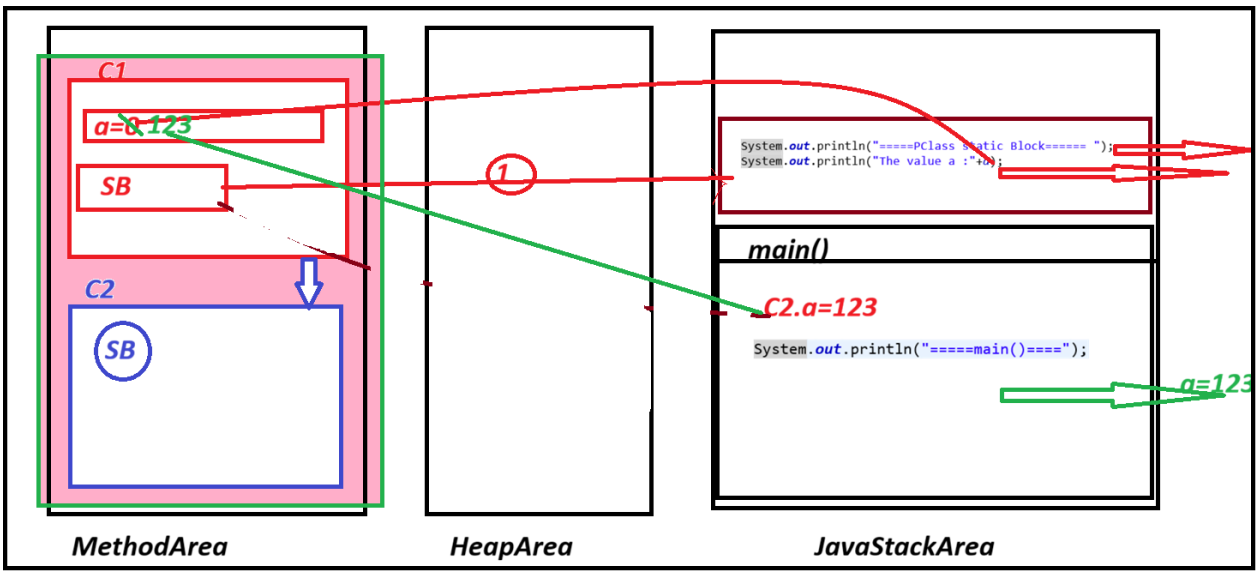
p2: DemoInheritance6

```
package p2;

import p1.C2;
```

```
public class DemoInheritance6 {
	public static void main(String[] args) {
		C2.a=123;
		System.out.println("=====main()====");
		System.out.println("The value a : "+C2.a);
	}

}
```

output

```
=======
=====PClass static Block======
The value a :0
=====main()====
The value a : 123
```



---

project Name : Inheritance_App7

p1: C1.java

```
package p1;

public class C1 {

	public static int a;

	static {
		System.out.println("======PClass Static block=========");
		System.out.println("The value of a : "+a);
	}
}
```

---

p1: C2.java

```
package p1;

public class C2 extends C1 {

	public static int b;

	static {
		System.out.println("==========CClass Static Block=====");
		System.out.println("The value a : "+a);
		System.out.println("The value b: "+b);
```

```
	}

}
```

---

p2: DemoInheritance7

```
package p2;

import p1.C2;

public class DemoInheritance7 {

	public static void main(String[] args) {
		C2.b=123;
		System.out.println("=======main()======");
		System.out.println("The value a : "+C2.a);
		System.out.println("The value b :"+C2.b);

	}
}
```

---

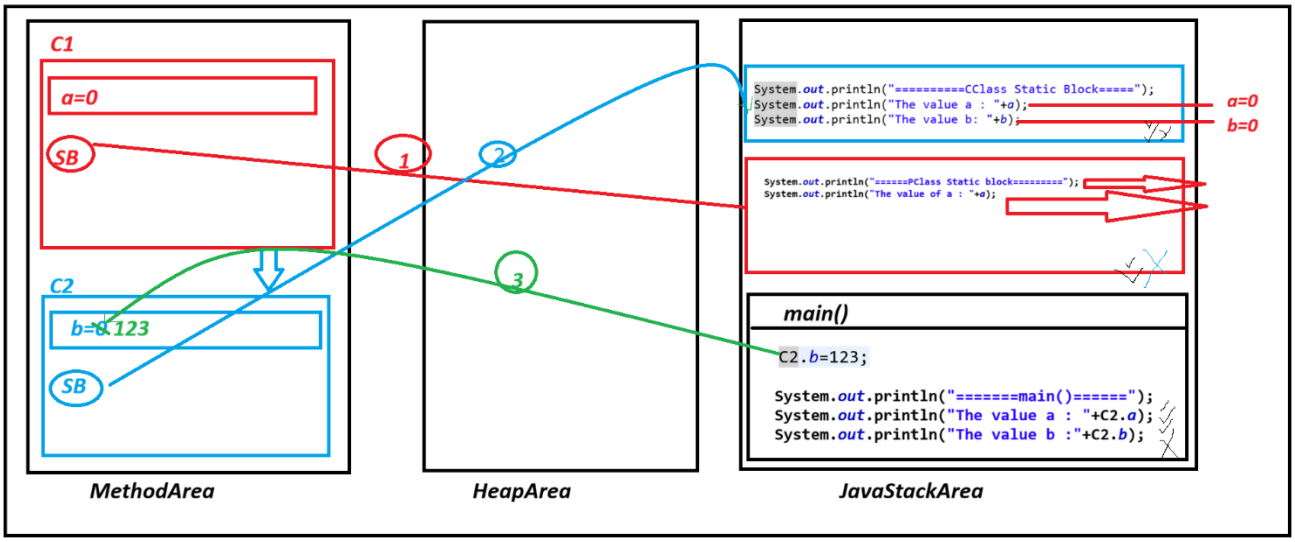
output

```
======PClass Static block=========
The value of a : 0
==========CClass Static Block=====
The value a : 0
The value b: 0
```

=======main()======

The value a : 0

The value b :123



========

ProjectName : Inheritance_App8

p1: C1.java

```
package p1;


public class C1 {

	public static int a;
	static {
		System.out.println("=========PClass block=======");
		System.out.println("The value a : "+a);
	}


}
```

---

p1: C2.java

```
package p1;

public class C2 extends C1{

	public static int b;

	static {
		System.out.println("=======CClass static blokc========");
		System.out.println("The value a : "+a);
		System.out.println("The value b : "+b);
	}
}
```

P2: DemoInheritance8.java

```
package p2;

import p1.C2;

public class DemoInheritance8 {

	public static void main(String[] args) {

		C2.a=123;
		C2.b=125;
		System.out.println("===main()=======");
		System.out.println("The value of a :"+C2.a);
```

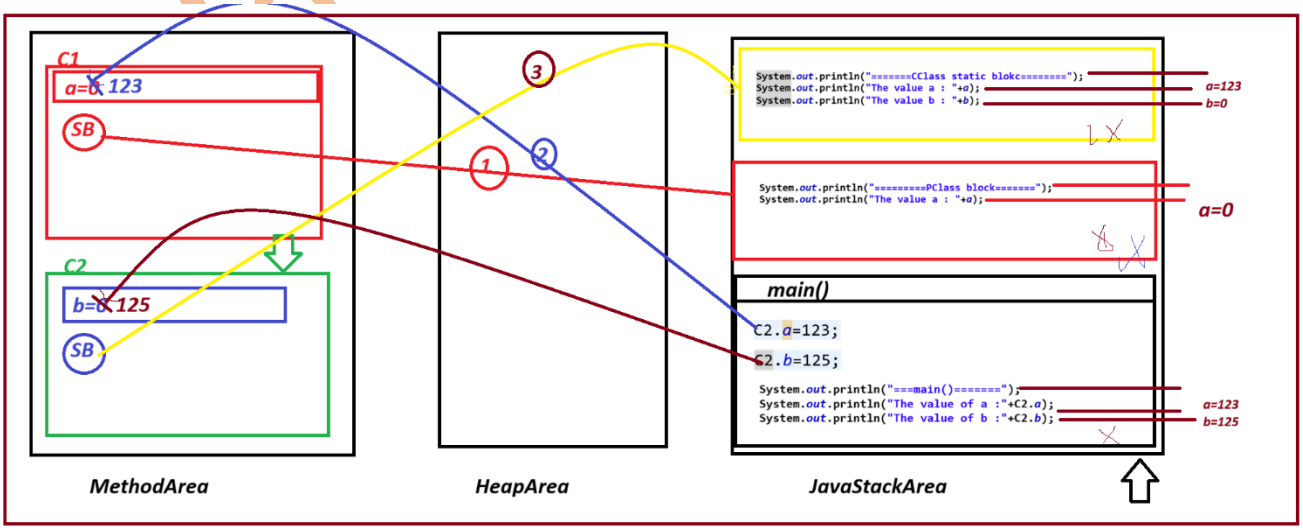
```
        System.out.println("The value of b :"+C2.b);

    }

}
```

---

output

```
=========PClass block=======
The value a : 0
=======CClass static blokc========
The value a : 123
The value b : 0
===main()=======
The value of a :123
The value of b :125
```

Diagram



==========

session 39:

Define Method Overloading process?

=>More than one method with same_method_name, but differentiated by their para_list or para_type is known as Method Overloading process.

=>In Method Overloading process, the return_type of method are not considered.

Note:

=> To perform Method Overriding process, Inheritance process is mandatory

=> To perform Method overloading process, Inheritance process is not needed.

case 1: Instance Method Overloading process

case 2: Static Method Overloading process

case 3: Constructor Overloading process

----

case 1: Instance Method Overloading process

ProjectName : Inheritance_App9

packages,

p1 : Addition.java

```
package p1;

public class Addition {

    public void add(int x,int y){
```

```
            System.out.println("======add(x,y)==========");
            System.out.println("sum: "+(x+y));
        }


        public int add(int x,int y, int z){
            System.out.println("========add(x,y,z)=======");
            return x+y+z;
        }

        public void add(int x,float y){
         System.out.println("========add(x,y)----------");
         System.out.println("sum : "+(x+y));
        }


}
```

---

p2. DemoInheritance9.java]

```
package p2;

import java.util.Scanner;

import p1.Addition;

public class DemoInheritance9 {
```

```
public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        Addition ad = new Addition();

        xyz: while(true) {
            System.out.println("========choice========");
            System.out.println("\t1.add(int,int)" + "\n\t2.add(int,int,int)" +
"\n\t3.add(int,float)" + "\n\t4.exit(stop)");

            int choice = s.nextInt();

            switch (choice) {
            case 1:
                System.out.println("Enter int value 1 :");
                int v1 = s.nextInt();
                System.out.println("Enter int value 2 : ");
                int v2 = s.nextInt();
                ad.add(v1, v2);
                break;

            case 2:
                System.out.println("Enter int value 1 :");
                int v11 = s.nextInt();
                System.out.println("Enter int value 2 : ");
                int v22 = s.nextInt();
```

```
                System.out.println("Enter int value 3 :");
                int v33 = s.nextInt();

                int res = ad.add(v11, v22, v33);
                System.out.println("Sum : " + res);
                break;

            case 3:
                System.out.println("Enter int value 1 :");
                int v111 = s.nextInt();
                System.out.println("Enter float value 2 : ");
                float v222 = s.nextFloat();
                ad.add(v111, v222);

                break;
            case 4:
                System.out.println("Program stopped....");
                System.exit(0);
                break xyz;

            default:
                System.out.println("Invalid Choice...");
            }//end of switch

        }//end of while loop
        s.close();
    }
```

```
}

------------------
output
=======
========choice========
        1.add(int,int)
        2.add(int,int,int)
        3.add(int,float)
        4.exit(stop)
1
Enter int value 1 :
10
Enter int value 2 :
2
======add(x,y)===========
sum: 12
========choice========
        1.add(int,int)
        2.add(int,int,int)
        3.add(int,float)
        4.exit(stop)
2
Enter int value 1 :
23
Enter int value 2 :
```

```
32
Enter int value 3 :
25
========add(x,y,z)=======
Sum : 80
========choice========
        1.add(int,int)
        2.add(int,int,int)
        3.add(int,float)
        4.exit(stop)
3
Enter int value 1 :
20
Enter float value 2 :
3.5
========add(x,y)----------
sum : 23.5
========choice========
        1.add(int,int)
        2.add(int,int,int)
        3.add(int,float)
        4.exit(stop)
45
Invalid Choice...
========choice========
        1.add(int,int)
        2.add(int,int,int)
```

3.add(int,float)

4.exit(stop)

4

Program stopped....

---

what is the diff b/w

(i) super

(ii) this

(i) super:

=> 'super' keyword is used to access variables and methods from the PClass or SuperClass.

(ii) this:

=> 'this' keyword is used to access variable and method from the same class.

Note :

=> Using 'super' and 'this' keyword we can perform Instance method Interlinking process, which means instance Chaining process.

==========

ProjectName : Inheritance_App10

pacakges:

p1: C1.java

package p1;

```
public class C1 {

	public void m(int a, int b) {
		this.m(a);
		System.out.println("=======PClass m(a,b)=======");
		System.out.println("The value of b: "+b);
	}

	public void m(int a) {
		System.out.println("======PClass m(a)=======");
		System.out.println("Tha value of a :"+a);
	}

}
```

---

p1: C2.java

```
package p1;

public class C2 extends C1{

	public void m(int a, int b, int c, int d) {
		this.m(a,b,c);
		System.out.println("======CClass m(a,b,c,d)=========");
		System.out.println("The value d: "+d);
	}
```

```
	public void m(int a,int b,int c) {
		super.m(a, b);
		System.out.println("=======CClass m(a,b,c)=====");
		System.out.println("The value c :"+c);
	}

}
```

---

p2: DemoInheritance10.java

```
package p2;

import p1.C2;

public class DemoInheritance10 {

	public static void main(String[] args) {
		C2 ob = new C2();

		ob.m(1, 2, 3, 4);
	}

}
```

---

output

======PClass m(a)=======

Tha value of a :1

=======PClass m(a,b)=======

The value of b: 2

=======CClass m(a,b,c)=====

The value c :3

======CClass m(a,b,c,d)=========

The value d: 4

============================

case 2: Static Method Overloading process

=> we can perform static method Overloading process in java.

faq:

can we user 'super' and 'this' keyword to access static method?

=> Yes, we can use 'suepr' and 'this' keywords to access static memebers, but 'super' and 'this' keywords must be used with NonStatic Methods.

Note:

=> we cannot perform Static method Interlinking process using 'super' and 'this' keyword.

======================

ProjectName : Inheritance_App11

p1: C1.java

```
package p1;


public class C1 {


        public static void m(int a) {
```

```
        System.out.println("======PClass m(a)=======");
        System.out.println("Tha value of a :"+a);
    }

    public static void m(int a, int b) {
    //      this.m(a);
        System.out.println("=======PClass m(a,b)=======");
        System.out.println("The value of a: "+a);
        System.out.println("The value of b: "+b);
    }


}

---------------
p1: C2.java
package p1;

public class C2 extends C1{


    public static  void m(int a,int b,int c) {
        //super.m(a, b);
        System.out.println("=======CClass m(a,b,c)=====");
        System.out.println("The value a: "+a);
        System.out.println("The value b: "+b);
```

```
            System.out.println("The value c :"+c);
        }

        public static void m(int a, int b, int c, int d) {
            //this.m(a,b,c);
            System.out.println("======CClass m(a,b,c,d)=========");
            System.out.println("The value a: "+a);
            System.out.println("The value b: "+b);
            System.out.println("The value c: "+c);
            System.out.println("The value d: "+d);
        }

        public void access(int a,int b,int c,int d) {
            super.m(a);
            super.m(a, b);
            this.m(a, b, c);
            this.m(a, b, c, d);
        }

}
```

---

p2: DemoInheritance11.java

```
package p2;

import p1.C2;
```

```
public class DemoInheritance11 {

	public static void main(String[] args) {
		C2 ob = new C2();

		ob.access(11, 22, 33, 44);

	}

}
```

output

```
------------------------
======PClass m(a)=======
Tha value of a :11
=======PClass m(a,b)=======
The value of a: 11
The value of b: 22
=======CClass m(a,b,c)=====
The value a: 11
The value b: 22
The value c :33
======CClass m(a,b,c,d)=========
The value a: 11
The value b: 22
The value c: 33
The value d: 44
```

---

faq:

can we perform standard main() method Overloading process?

=> Yes, we can perform Overloading process for standard main() method, because statndar main() method is static method.

==========

case 3: Constructor Overloading process

=> we can also perform Constructor Overloading process in java.

faq:

what is the diff b/w

(i) super()

(ii) this()

(i) super() :

=> 'super()' is used to execute constructor from the PClass or SuperClass

(ii) this():

=> 'this()' is used to execute constructor from the same class or current running class.

Note:

=> Using 'super()' and 'this()' we can perform Constructor Interlinking process, which means Constructor Chaining process.

==========

sesssion 40

Example

ProjectName: Inheritance_App12

p1: C1.java

```
package p1;

public class C1 {

	public C1(int a,int b) {
		this(a);
		System.out.println("======PClass C1(a,b)=======");
		System.out.println("The value of b : "+b);
	}

	public C1(int a) {
		System.out.println("======PClass C1(a)=======");
		System.out.println("The value of a : "+a);
	}

}
```

---

p1: C2.java

```
package p1;

public class C2 extends C1 {

	public C2(int a,int b,int c,int d) {
		this(a, b, c);
		System.out.println("======CClass C2(a,b,c,d)=======");
```

```
		System.out.println("The value of d : "+d);
	}

	public C2(int a,int b,int c) {
		super(a,b);
		System.out.println("======CClass C2(a,b,c)=======");
		System.out.println("The value of c : "+c);
	}

}
```

p2: DemoInheritance12.java

```
package p2;

import java.util.Scanner;

import p1.C2;

public class DemoInheritance12 {

	@SuppressWarnings("resource")
	public static void main(String[] args) {
		Scanner s = new Scanner(System.in);
		 C2 ob = new C2(11,22,33,44); //Con_call_4_parameterized_Constructor
	}

}
```

==========

output

======PClass C1(a)=======

The value of a : 11

======PClass C1(a,b)=======

The value of b : 22

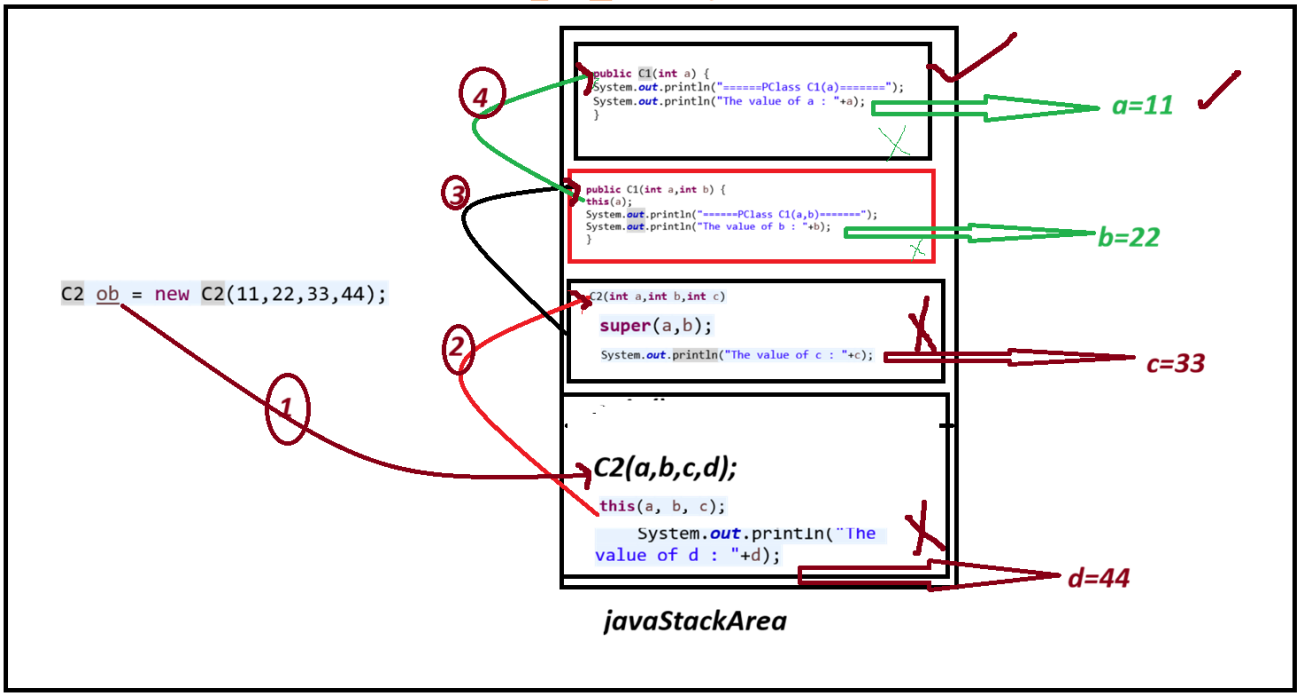
======CClass C2(a,b,c)=======

The value of c : 33

======CClass C2(a,b,c,d)=======

The value of d : 44

Diagram

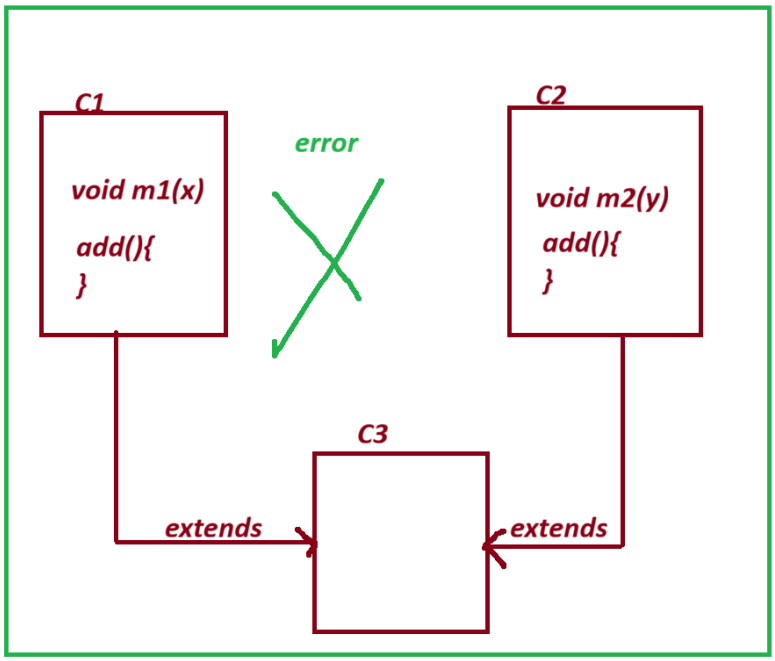


==========

define Multiple Inheritance?

=> The process of extracting the feature from more than one class at-a-time is known as Multiple Inheritance.

Diagram



Note:

=> Multiple inheritance process using Classes in java not available, becuase which loads to replication of programming components and raises ambiguity, the ambiguity state application will generate wrong results.

=> In java, we can perform Multiple Inheritance process using Interfaces.

==========

imp

Interface in java:

=> Interface is a collection of variable,abstract methods and Concrete methods from java8 version onwards.

(upto java7 version interface can hold only variable and absctract methods, but cannot hold Consrete methods).

define abstract methods?

=> The method whicha are declared without method_body are known as abstract methods.

structure of abstract_method

return_type method_name(para_list);

define Concrete methods?

=> The method which are declared with method_boyd are known as Conscrete method or Constructed method.

structure of Concrete_method?

```
return_type method_name(para_list)
{
    //method_body
}
```

---

session 41

==========

Coding Rules of Interface:

Rule 1: we use 'interface' keyword to declare interfaces

syntax:

```
interface Interface_name{
    //interface body
}
```

Rule 2: The programming components which are declared within the interface are automatically 'public'.

Note:

=>The programming components which are declared within the class without any access modifier are considered as 'default'.

Rule 3: The interface can be declared with both Primitive datatype variables and NonPrimitive datatypes variables.

Rule 4: The variables which are declared within the interface are automatically 'static' and 'final' variables.

Note:

(i) static variables will get the memory within the interface while interface loading and can be accessed with interface name.

(ii) The final variables must be initialized with values and once initialized cannot be modified.

(final variable are known as Secured variables or Constant variable).

Rule 5: The methods which are declared within the interface are automatically NonStatic abstract methods.

(There is no concept of static abstract methods)

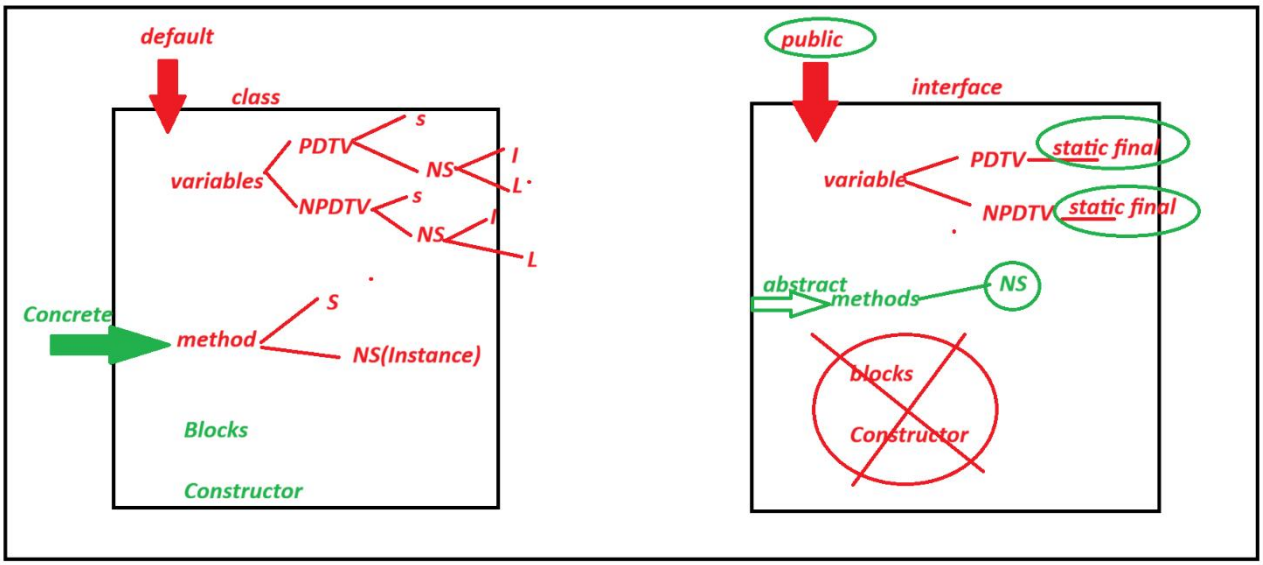
Rule 6: Interface cannot be instantiated, which means we cannot create objets for interface directly.

Rule 7: Interface must be implemented to class using 'implements' keyword and the classess are known as implementation clasess.

Rule 8: These implementation classes must construct body for all abstract methods of interface.

Rule 9: The implementation classess can also be declared with NonOverriding and NonImplemented methods.

Rule 10: There is no concept of Blocks and Constructors in interfaces, which means we cannot declare blocks and constructors within the interface.



==========

ProjectName: Interface_App1

p1: ITest.java

```
package p1;


public interface ITest {

    public static final int z=234;

  public abstract void dis(int k);


  public abstract void m1(int x);
```

```
}
==============
p1: IClass.java
package p1;

public class IClass implements ITest{


        public void dis(int k) {
          System.out.println("=====Implementation method dis(k)=====");
          System.out.println("The value k: "+k);
          System.out.println("The value z: "+z);

        }


        public void m1(int x) {
              System.out.println("=====Implementation method m1(x)=====");
               System.out.println("The value x: "+x);
               System.out.println("The value z: "+z);

        }

        public void m2(int y) {
              System.out.println("=====NonImplementation method m2(y)=====");
                System.out.println("The value y: "+y);
```

```
                System.out.println("The value z: "+z);
        }



}
=============
p2: DemoInterface1.java
package p2;

import p1.ITest;
import p1.IClass;
public class DemoInterface1 {

        public static void main(String[] args) {

                //ITest ob=new ITest();

                IClass ob=new IClass();

                ob.dis(12);
                ob.m1(13);
                ob.m2(14);
        }

}
```

output

=====Implementation method dis(k)=====

The value k: 12

The value z: 234

=====Implementation method m1(x)=====

The value x: 13

The value z: 234

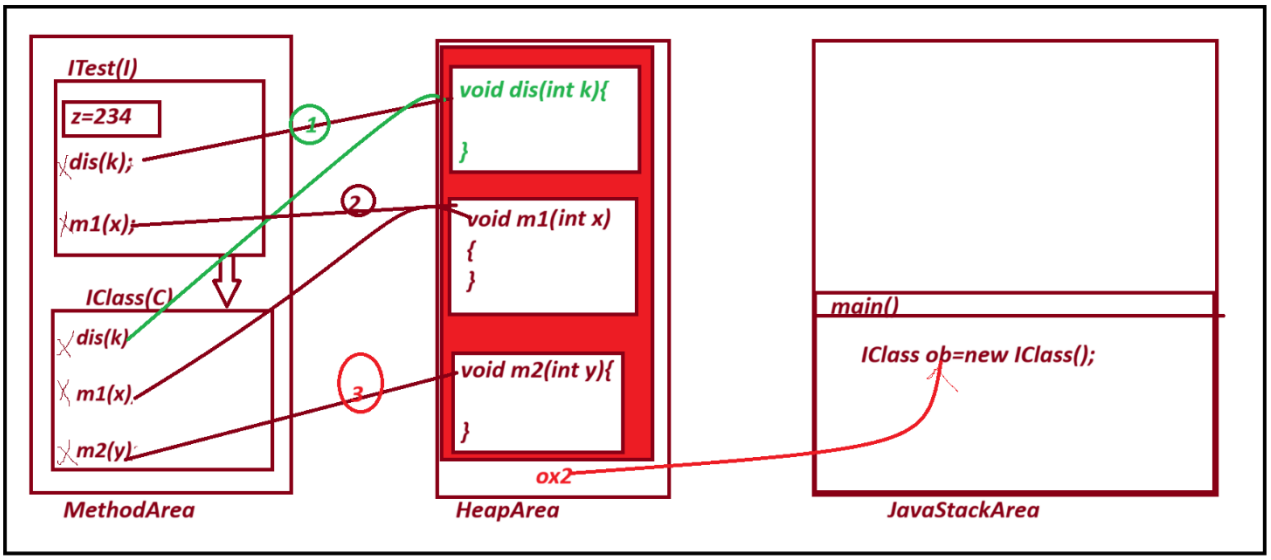
=====NonImplementation method m2(y)=====

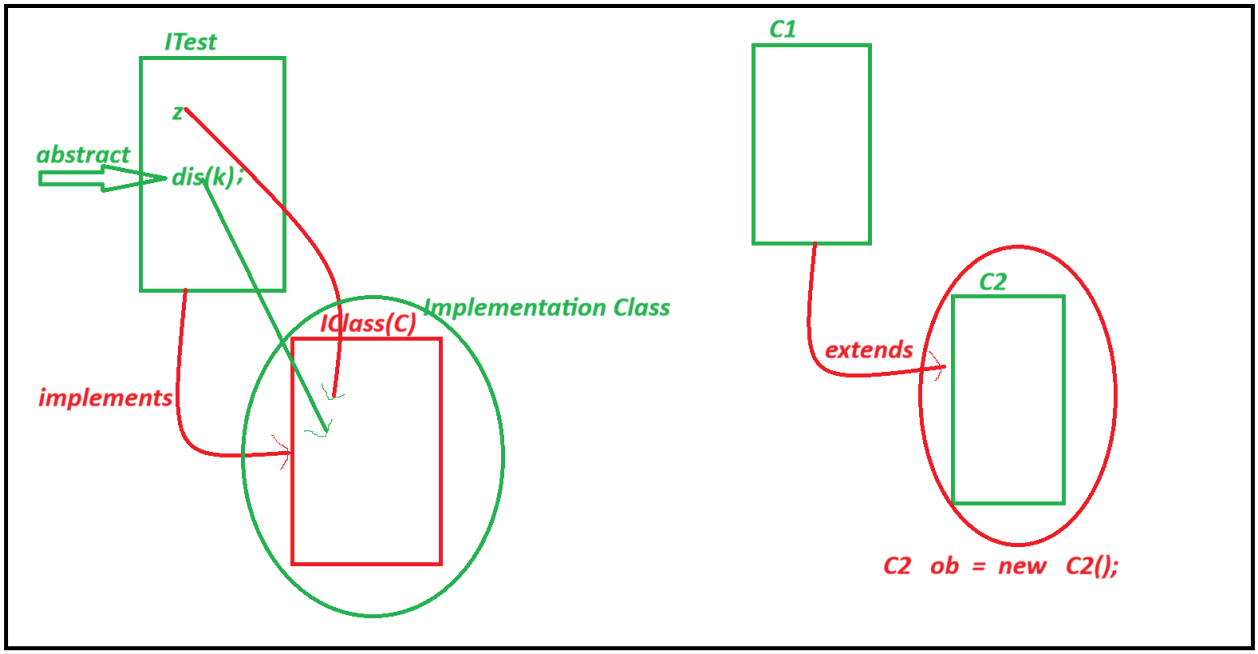
The value y: 14

The value z: 234

==========

Diagram





=========

session 42

=========